

射擊世界學習指南文件

學習目標

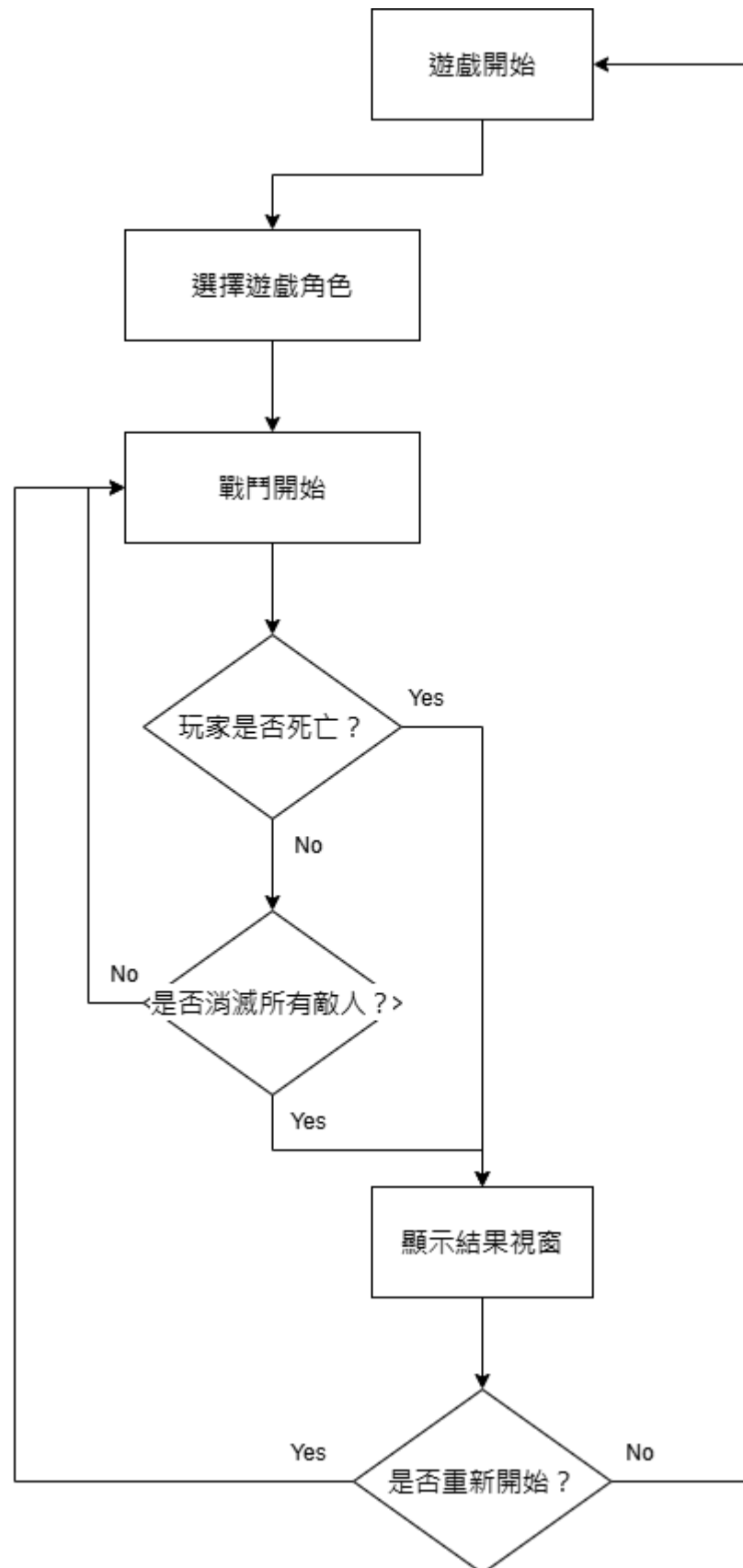
- 透過指南文件與範例世界，學習實作射擊遊戲核心功能的方法。
 - 學習在楓之谷世界中，依據欲製作的遊戲方式來設定地圖的方法。
 - 理解Logic，並能設計其使用方法。
 - 能將製作完成的世界加工成想要的形式並進行製作。
-

學習核心功能

- **DataSet管理**：學習使用DataTableLogic，讓一個Logic作為管理DataSet的Manager使用。
- **實體間碰撞管理**：利用TriggerComponent的Collision Group與Matrix設定碰撞條件並使用。
- **物件池**：學習為管理大量投射體的物件池原理，並能製作。



核心遊戲循環

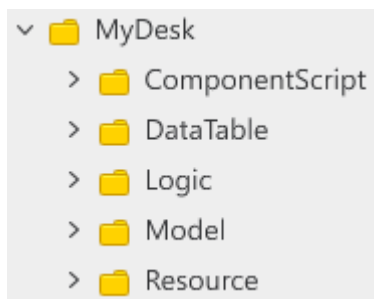


- 1.遊戲開始：**可在標題畫面中確認開始遊戲或遊玩方式。
- 2.玩家角色選擇：**在標題畫面按下開始遊戲按鈕時，會顯示選擇角色的UI。
- 3.戰鬥開始：**正式開始射擊遊戲。會發生如怪物出現、玩家攻擊等射擊遊戲互動。
- 4.檢查是否擊殺所有敵人：**確認當前遊玩地圖中是否擊殺了符合通關條件的怪物數量。

5.結果確認：根據玩家是否擊殺所有怪物，或玩家是否失敗，顯示結果畫面。

6.檢查是否重新開始：根據玩家是否要求重新開始或返回標題，移動至對應畫面。

資料夾構成



姓名	說明
ComponentScript	用於存放為了實作範例世界而附加在實體上的元件的資料夾。
DataTable	用於存放構成範例世界所需DataSet的資料夾。
Logic	用於存放以楓之谷世界功能Logic製作的腳本的資料夾。
Model	用於存放model化後實體的資料夾。
Resource	用於存放遊戲製作所需Resource的資料夾。 目前僅包含TileSet，之後可放入製作世界所需資源來使用。

實作順序

- 💡 **基礎學習：**包含楓之谷世界或該類型最基礎功能的學習項目。
- 📖 **進階學習：**學習並實作範例世界中的功能項目。透過該過程的學習，可以擴展類型功能並提升學習理解度。

[💡 基礎學習]1.設定地圖

- 學習楓之谷世界所提供的地圖種類。
- 學習BodyComponent與地圖種類之間的關係。
- 學習KinematicBodyComponent。
- 了解透過配置RectTile來形成地圖的方法。

[💡 基礎學習]2.宣告Logic

- 學習Logic扮演什麼角色。
- 理解Manager的概念並學習使用方法。
- 理解GameLogic、DataTableLogic、UIManageLogic的構成原因與目的。

- 學習Logic的優點與缺點。

[💡 基礎學習]3.製作Playercharacter

- 學習構成玩家的DataSet組成要素。
- 學習透過DataTableLogic載入DataSet並使用的方法。
- 了解製作並套用用於儲存資料的Component的方法。
- 學習賦予TriggerComponent並進行編輯的方法。

[💡 基礎學習]4.製作Monster

- 學習構成怪物的DataSet組成要素。
- 學習透過DataTableLogic載入DataSet並使用的方法。
- 了解製作並套用用於儲存資料的Component的方法。
- 學習賦予TriggerComponent並進行編輯的方法。
- 為了讓怪物追蹤玩家，賦予AIChaseComponent並學習使用方法。
- 學習將製作完成的怪物進行Model化以便重複使用的方法。

[💡 基礎學習]5.製作基本攻擊

- 學習構成投射體的DataSet組成要素。
- 學習透過DataTableLogic載入DataSet並使用的方法。
- 了解製作並套用用於儲存資料的Component的方法。
- 學習賦予TriggerComponent並進行編輯的方法。
- 學習Collision Group，並學習透過Matrix進行編輯的方法。
- 賦予投射體朝指定方向發射的功能。

[💡 基礎學習]6.建立物件池

- 學習物件池的概念並理解物件池的目的。
- 實作物件池Component。
- 透過物件池的建立與回收，直接確認其運作方式。
- 學習實作玩家與怪物的投射體發射系統的方法。

[📖 進階學習]7.實作技能

- 學習構成技能的技能DataSet組成要素。
- 學習透過DataTableLogic載入與技能相關的DataSet並使用的方法。
- 了解製作並套用用於儲存技能資料的Component的方法。
- 學習在技能實體上賦予TriggerComponent並進行編輯的方法。
- 學習將製作完成的實體轉換為Model並使用的方法。
- 賦予投射體型技能朝指定方向發射的功能。
- 學習為了實作投射體型技能所使用的TriggerEvent。

[📖 進階學習]8.實作道具

- 學習構成道具的技能DataSet組成要素。
- 學習透過DataTableLogic載入道具DataSet並使用的方法。
- 為了套使用道具資料與道具建立資料，進行元件新增與修改。

- 學習製作構成道具的Model的方法。
- 製作道具的建立機率與負責道具建立的ItemHelper Logic。
- 學習實作讓道具效果正常處理的方法。
- 學習為了套用道具建立機率而建立Monster的方法。

[📖 進階學習]9.實作MonsterSpawn

- 實作讓Monster依照DataSet中輸入的數值在InGame中出現的功能。
- 學習DataSet的結構並學習使用該結構的方法。
- 為每個出現的怪物賦予道具掉落機率，使其在被擊敗時依機率出現道具。

[💡 基礎學習]10.構成UI

- 學習UIGroup、其構成要素與編輯方法。
- 製作標題畫面並學習UI編輯方法。
- 製作控制標題畫面的Component，使其能處理該UI的功能。
- 學習製作讓玩家可以選擇角色的SelectCharacterUI的方法。
- 學習構成控制InGame資訊的DefaultUI的方法。
- 製作控制DefaultUI的Component，使體力與技能剩餘次數能實際更新。

[💡 基礎學習] 地圖設定

影片時間軸

- 可在課程影片中以下時間點進行學習。
- 理解地圖類型：00:01:29

學習目標

- 學習楓之谷世界的地圖方式，並在製作世界時選擇自己需要的地圖方式。
- 可說明依各地圖方式，與地面圖塊互動的元件是什麼。
- 學習為了與RectTileMap互動所需的KinematicBodyComponent。
- 學習在RectTileMap上直接配置圖塊的方法，並能形成地圖。

觀看該影片後可嘗試執行的內容

- 為了製作射擊世界，將Map的形式變更為RectTileMap。
- 製作TileSet。利用製作完成的TileSet實際製作可互動的Map。
- 利用製作完成的TileSet實際製作可互動的Map。

地圖種類

Key Point

遊戲視角會隨地圖類型而變化！

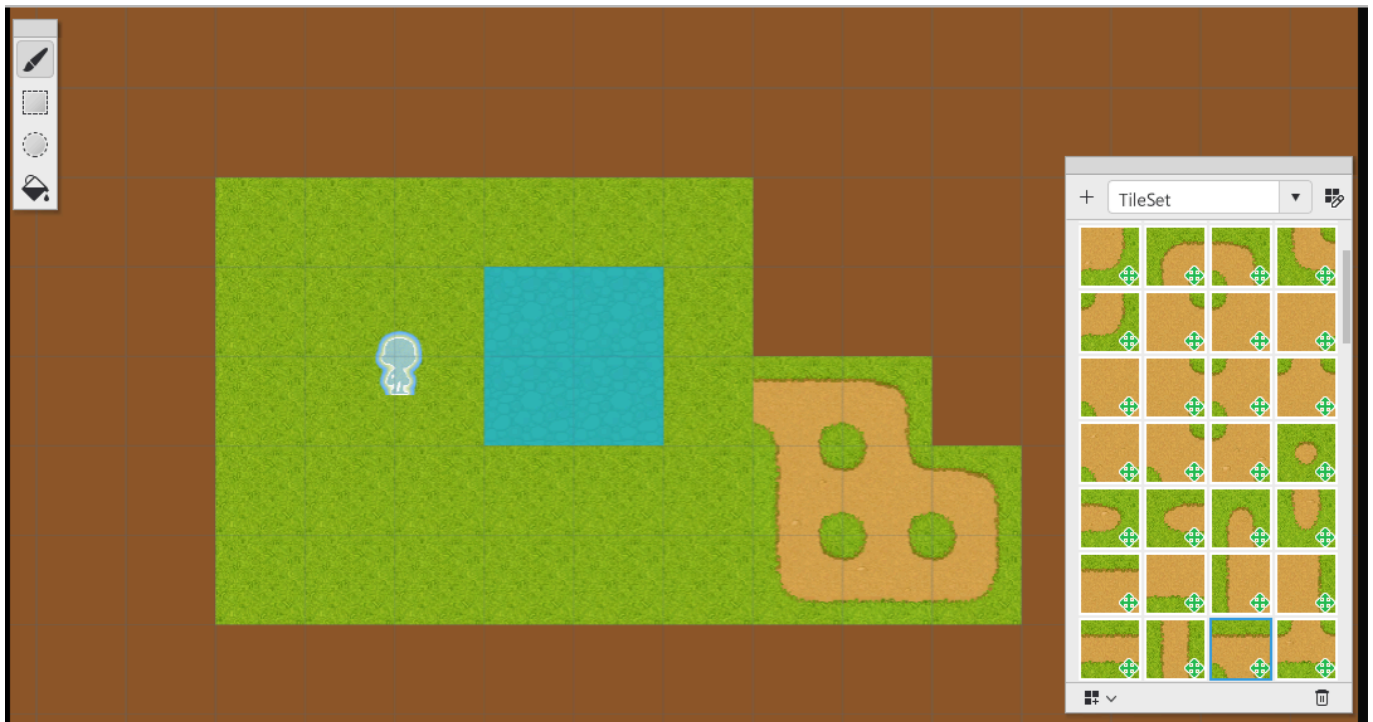


TileMap



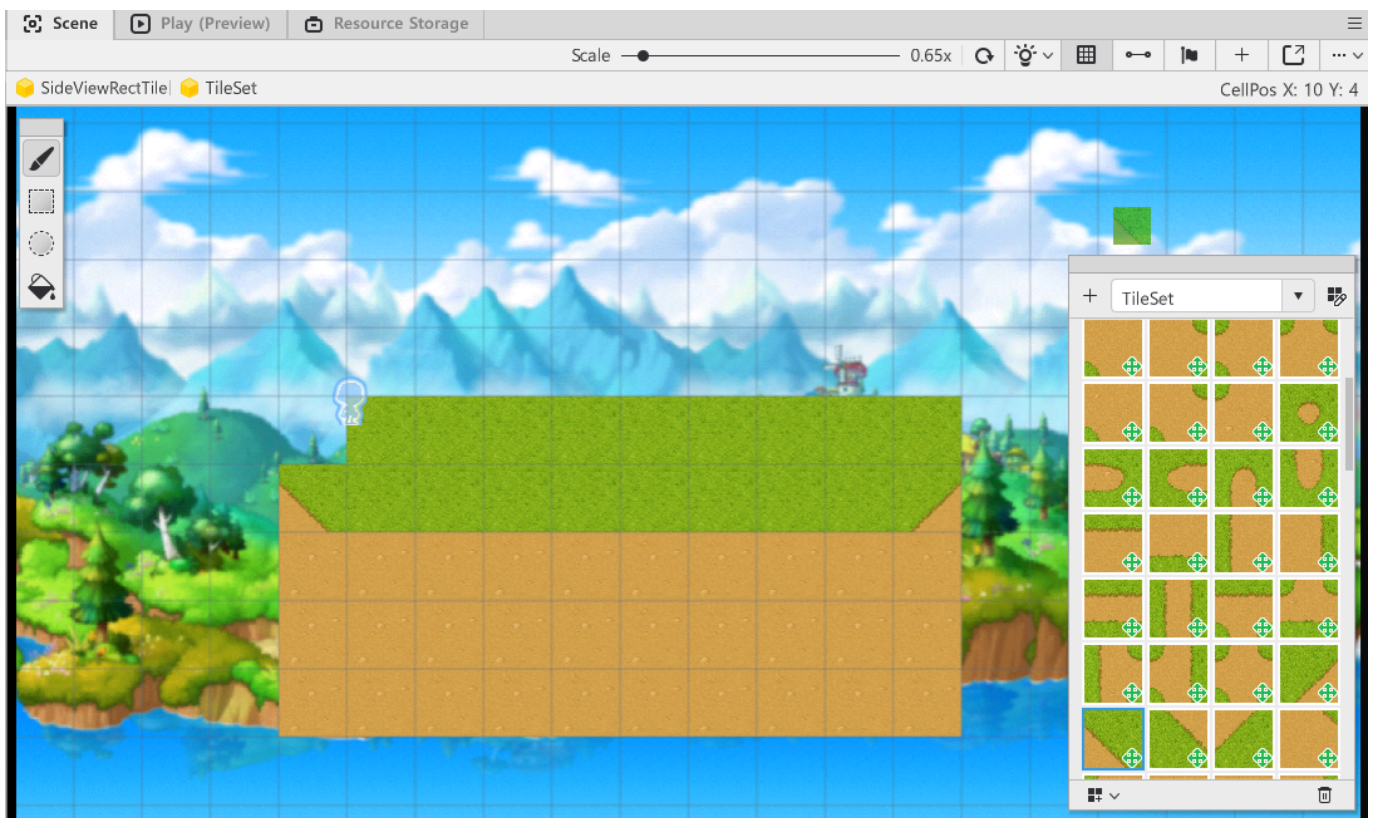
- 在楓之谷世界中初次建立世界時預設指定的地圖形式。
- 是最能展現楓之谷風格的世界方式。
- 利用楓之谷世界提供的Tile建立地面，並配置實體來形成地圖。
- 可依圖塊的配置製作斜坡。
- 在製作橫向捲軸形式的遊戲時，是能展現楓之谷風格的有用地圖方式。

RectTileMap



- 在製作俯視視角形式的遊戲時，是有效的地圖形式。
- 可自行製作圖塊集，或使用楓之谷世界提供的圖塊集資源來形成地圖。
- 可透過棋盤格形式配置圖塊來形成地圖。
- 可為每個圖塊賦予可移動與不可移動的性質。
- 與TileMap、SideViewRectTileMap方式不同，可製作4方向、8方向移動形式的遊戲。

SideViewRectTileMap



- 在製作橫向捲軸形式遊戲時，是有效的地圖形式。
- 可自行製作圖塊集，或使用楓之谷世界提供的圖塊集資源來形成地圖。

- 可透過棋盤格形式配置圖塊來形成地圖。
- 與RectTileMap不同，若不是不可移動的圖塊集，角色會**穿過**圖塊。
- 因此使用SideViewRectTileMap形成地圖時，必須將不可移動的圖塊集製作為**地面**。

BodyComponent

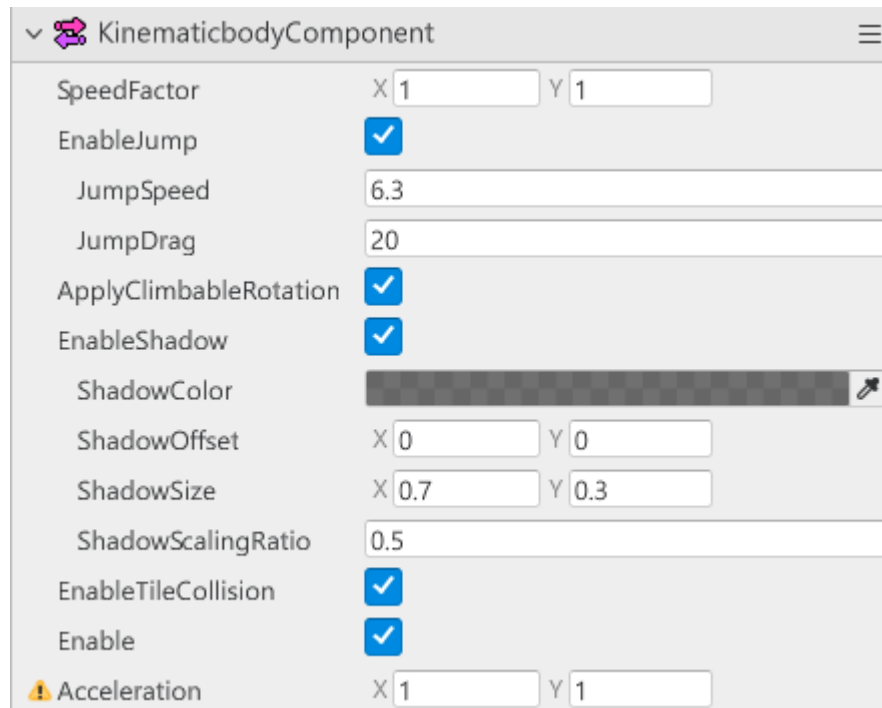
- 在楓之谷世界中，為了與作為地面的圖塊互動，需要BodyComponent。
- BodyComponent會依地圖形式使用不同的BodyComponent。
- BodyComponent大致分為3種類型。

RigidBodyComponent

Property	Value
Mass	100
Gravity	1
JumpBias	0.03
WalkJump	1.23
DownJumpSpeed	2
WalkSpeed	1.4
WalkAcceleration	1
WalkDrag	1
WalkSlant	0.9
AirAccelerationX	1
AirDecelerationX	1
FallSpeedMaxX	1
FallSpeedMaxY	1
LayerSettingType	All
IsBlockVerticalLine	☐
IgnoreMoveBoundary	☐
ApplyClimbableRotation	☒
IsolatedMove	☐
KinematicMove	☐
Enable	☒

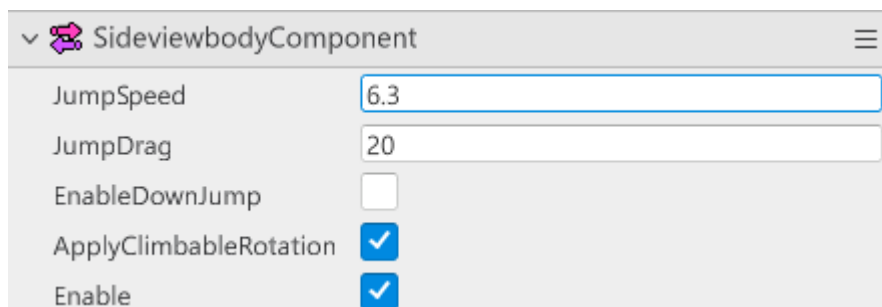
- 在楓之谷世界的TileMap方式中控制角色的物理。
- 若地圖為TileMap但沒有RigidBodyComponent，角色無法與圖塊互動。
- 在RigidBodyComponent中提供角色跳躍控制、重力控制等多種物理屬性值。
- 可透過變更該數值來改變物理方式。

KinematicBodyComponent



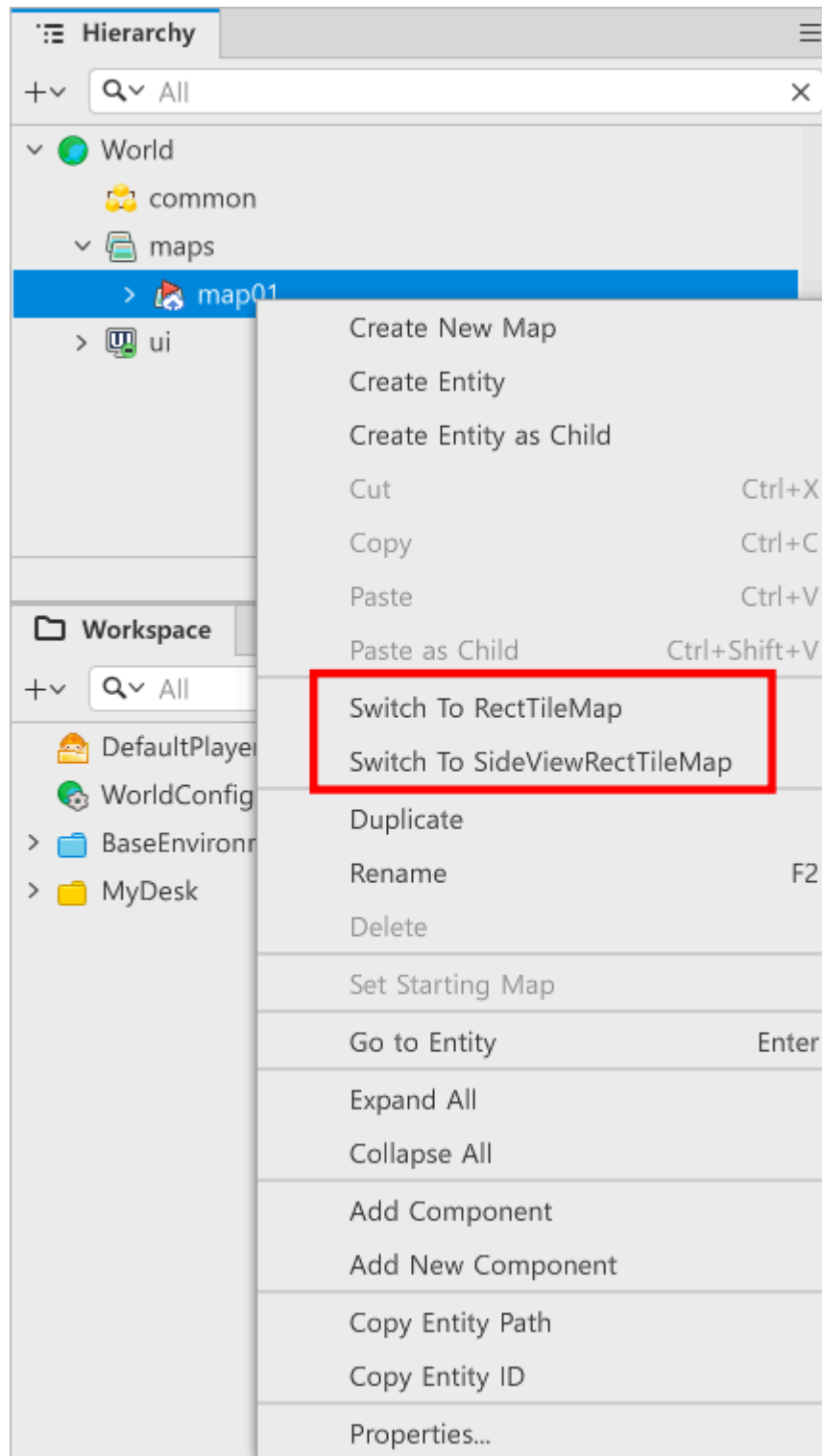
- 在楓之谷世界的RectTileMap方式中控制角色的物理。
- 當地圖為RectTileMap時，若沒有KinematicBodyComponent，將不會與RectTileMap的圖塊產生互動。
- 在KinematicBodyComponent中可調整角色的移動速度、是否可跳躍、是否顯示陰影等。

SideViewRectTileMap



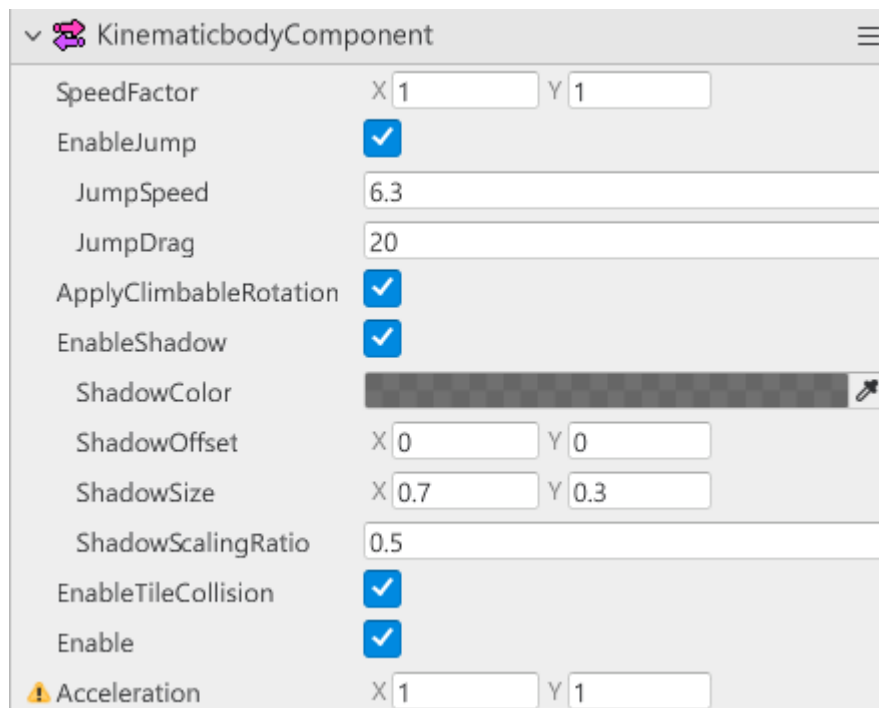
- 在楓之谷世界中使用SideViewRectTileMap方式時控制角色的物理。
- 若沒有該元件，角色無法與地圖圖塊互動。
- 可調整角色的跳躍速度與重力加速度等。

變更地圖形式



- 在Hierarchy視窗中對要變更的地圖按右鍵。
- 點擊Switch To RectTileMap或Switch To SideViewRectTileMap其中之一。
 - 若目前地圖不是TileMap，則會顯示Switch To TileMap，且切換到目前地圖狀態的選項會被隱藏。
- 為了製作射擊世界，點擊Switch To RectTileMap將地圖轉換為RectTileMap。

KinematicBodyComponent要素查看

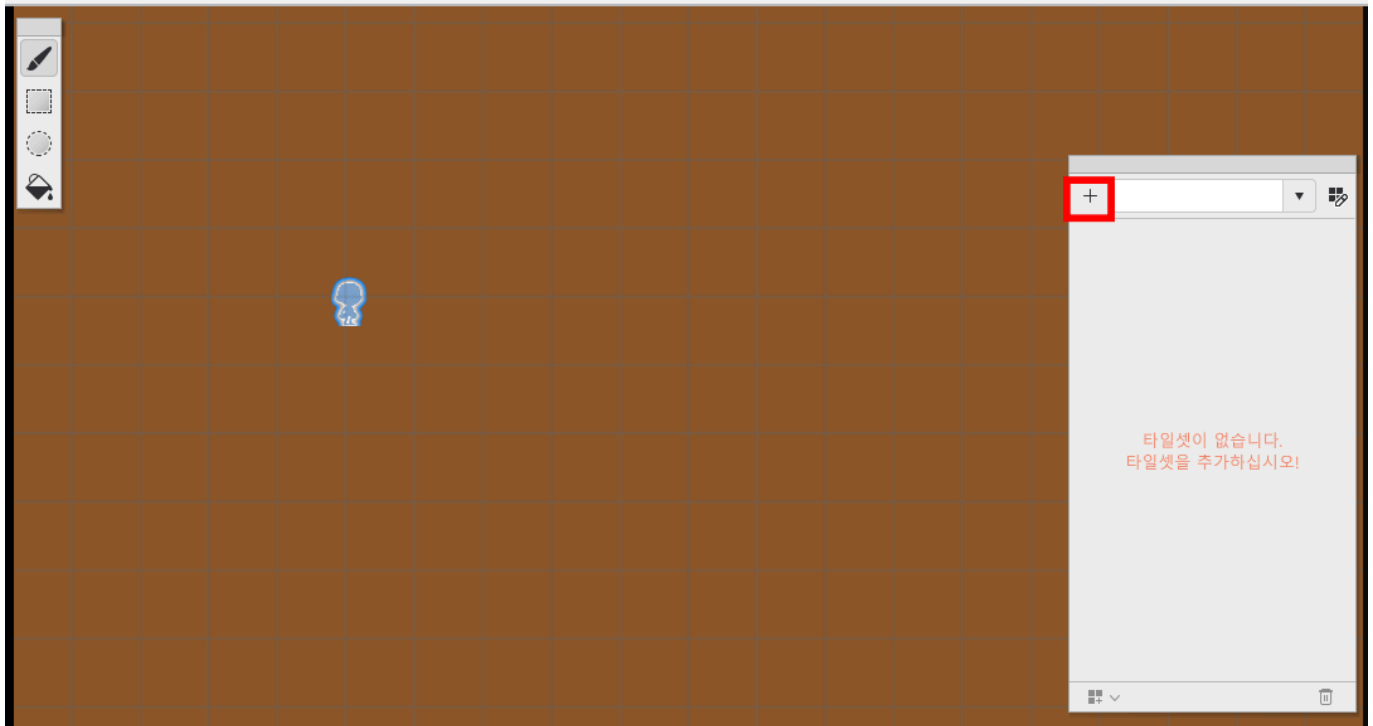


- **SpeedFactor**: 可調整角色移動速度的數值。X、Y的數值可分別調整左右或上下移動速度。
- **EnableJump**: 可決定是否允許角色跳躍。當該值未勾選時，角色無法跳躍。
- **JumpSpeed**: 可調整角色跳躍速度。數值越大，跳躍速度越快且可跳得更高。
- **JumpDrag**: 可調整角色的重力加速度。數值越小，重力影響越小，角色會在空中較慢下降。
- **ApplyClimbableRotation**: 當角色與梯子、繩索等物件互動時，決定是否依其旋轉來讓角色旋轉。
- **EnableShadow**: 決定是否顯示角色的陰影。當該值取消勾選時，將不顯示角色陰影。
- **ShadowColor**: 可變更陰影顏色。
- **ShadowOffset**: 可變更陰影顯示位置。
- **ShadowSize**: 可設定陰影大小。
- **ShadowScalingRatio**: 當角色跳躍時，可變更陰影縮放比例。數值越小，縮放比例越小。
- **EnableTileCollision**: 可讓角色在RectTileMap中無視不可移動的圖塊進行移動。

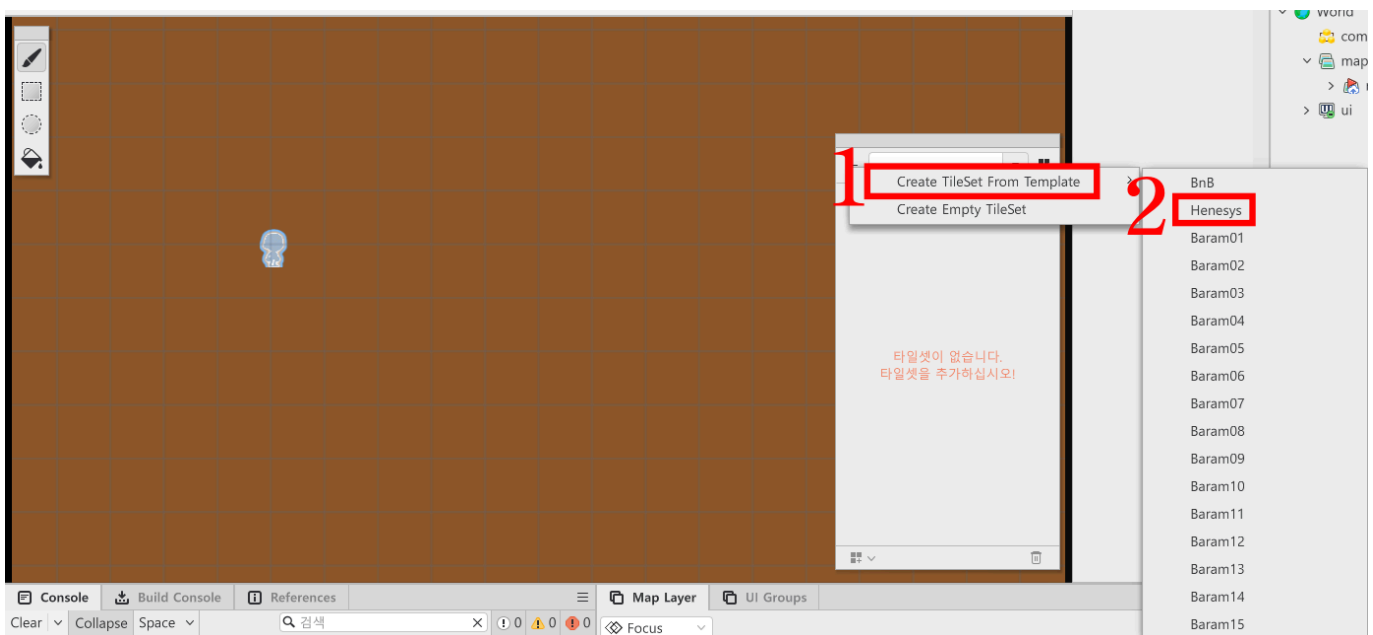
在RectTileMap配置圖塊

- 為了指定射擊世界的移動區域，需要配置RectTileMap。
- 若初次變更為RectTileMap，則需要用於配置圖塊的圖塊集。
- 在範例世界中，將使用楓之谷世界提供的資源來製作圖塊集。
- 若已經有要使用的資源，也可以自行製作圖塊集使用。

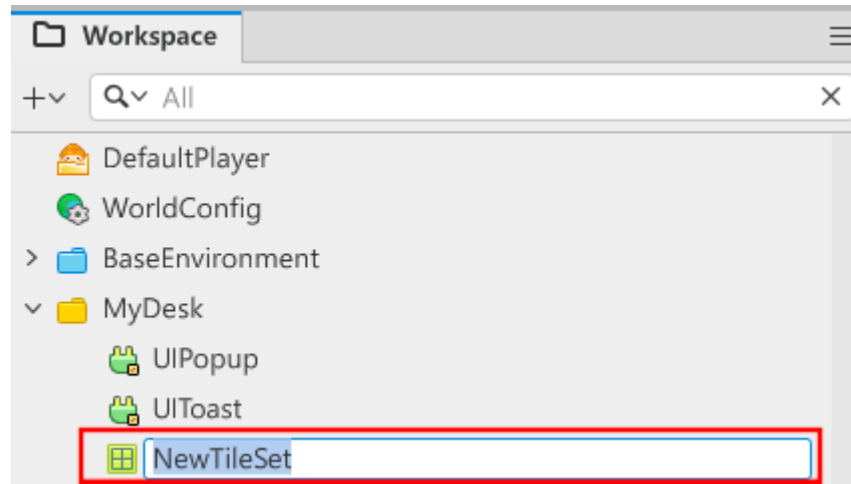
製作圖塊集



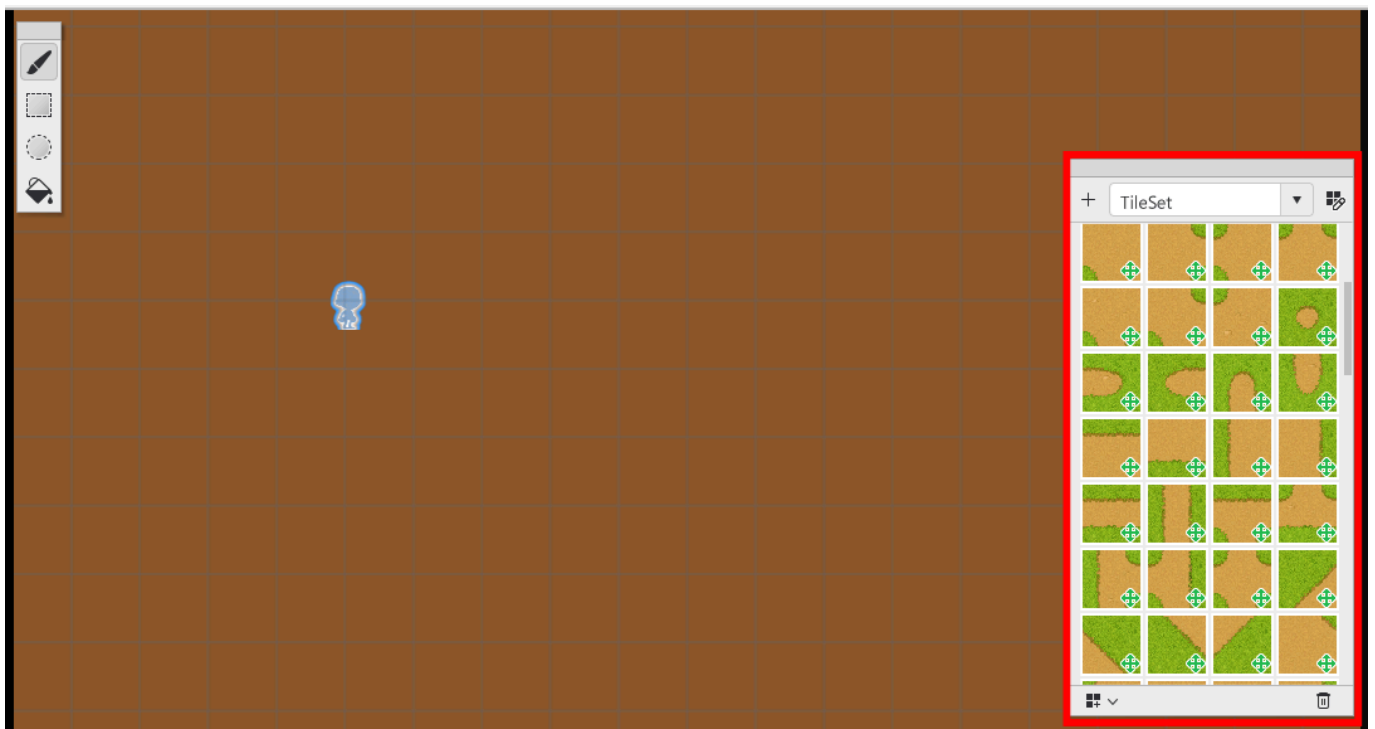
- 在RectTileMap狀態下，點擊圖塊集編輯器畫面的「+」按鈕。



- 點擊Create TileSetFrom Template。
- 接著選擇想要的資源。
 - 在範例世界中使用Henesys。
- 若有要使用的其他資源，可透過Create Empty TileSet自行製作圖塊集。

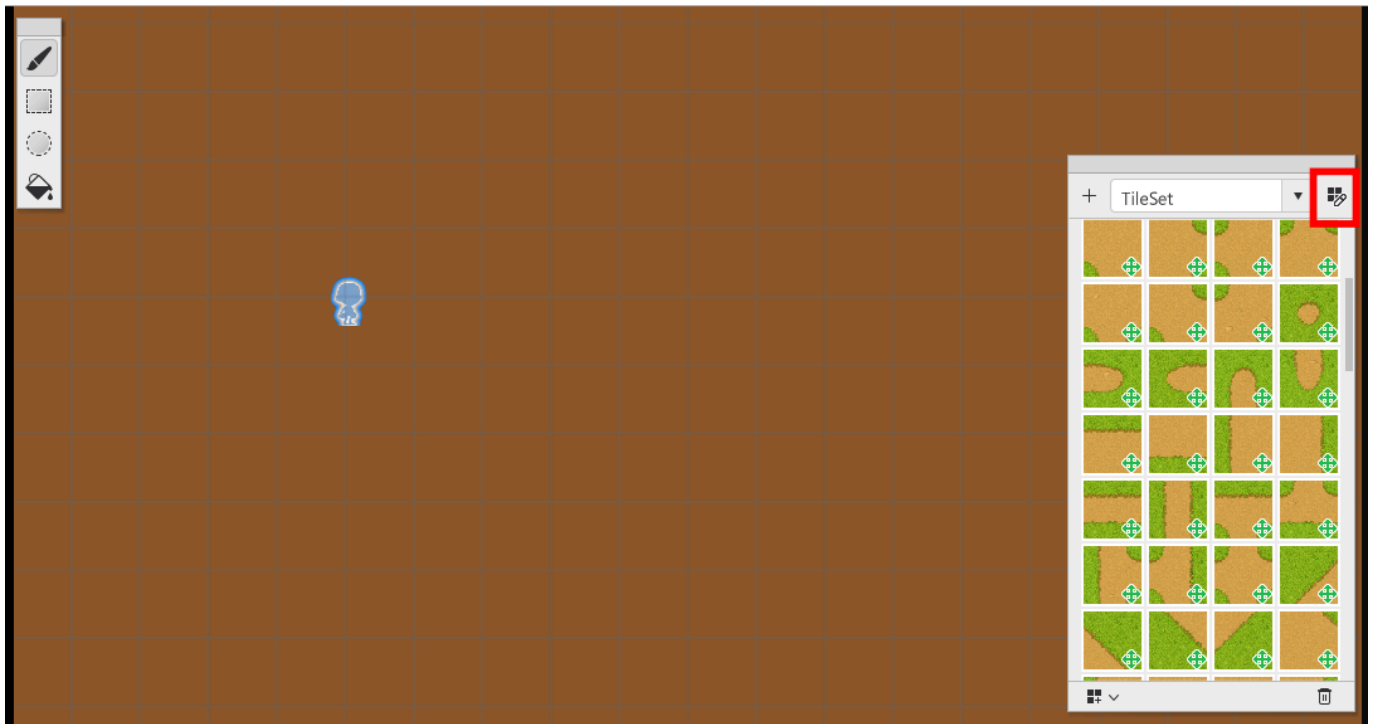


- 變更Workspace中新增的NewTileSet名稱以完成圖塊集製作。

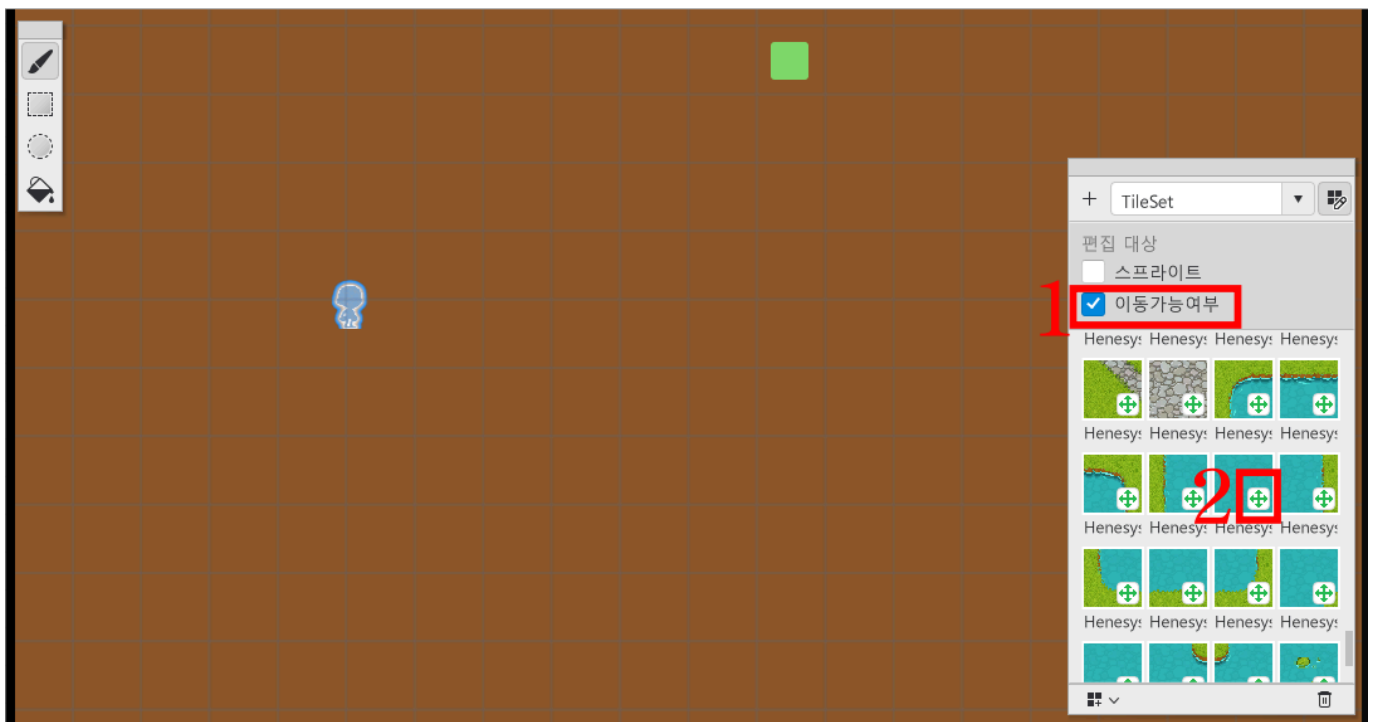


- 若圖塊集已正常製作完成，圖塊集編輯器畫面中會顯示可配置的圖塊。

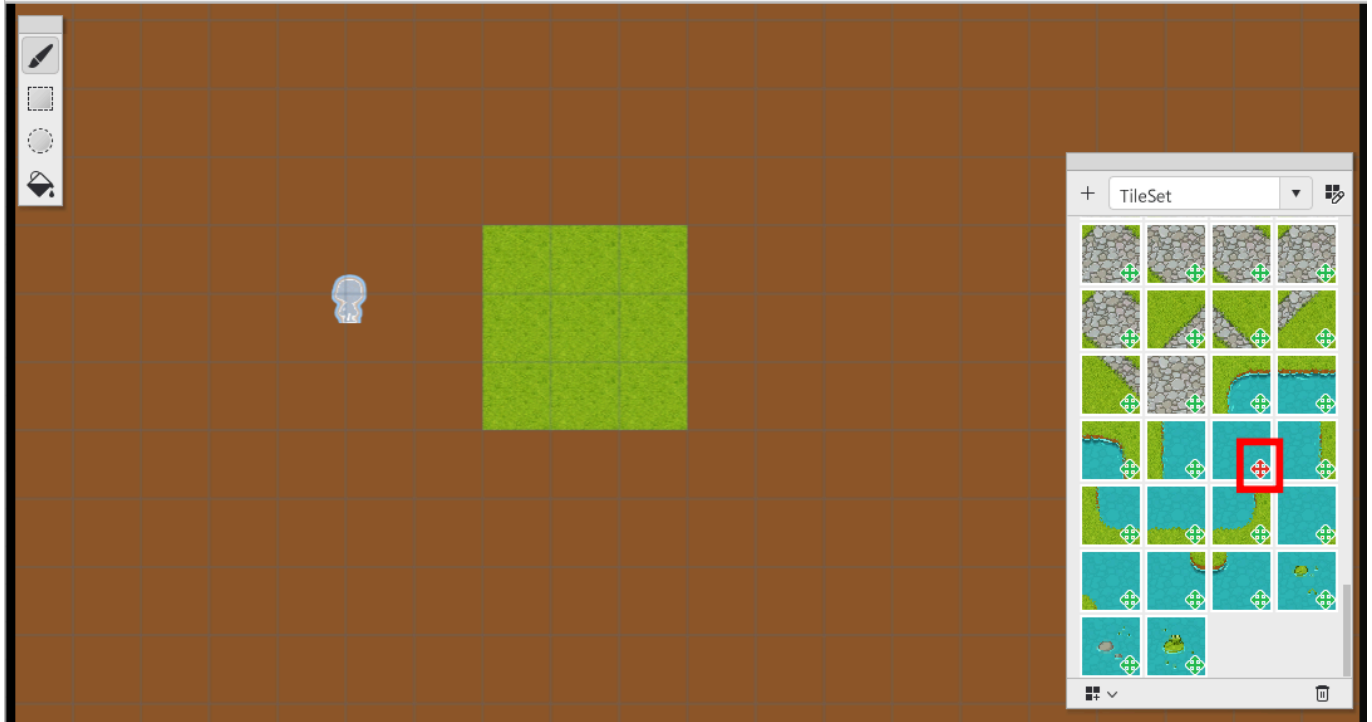
變更圖塊是否可移動



- 點擊圖塊集編輯器右側的圖塊編輯按鈕。

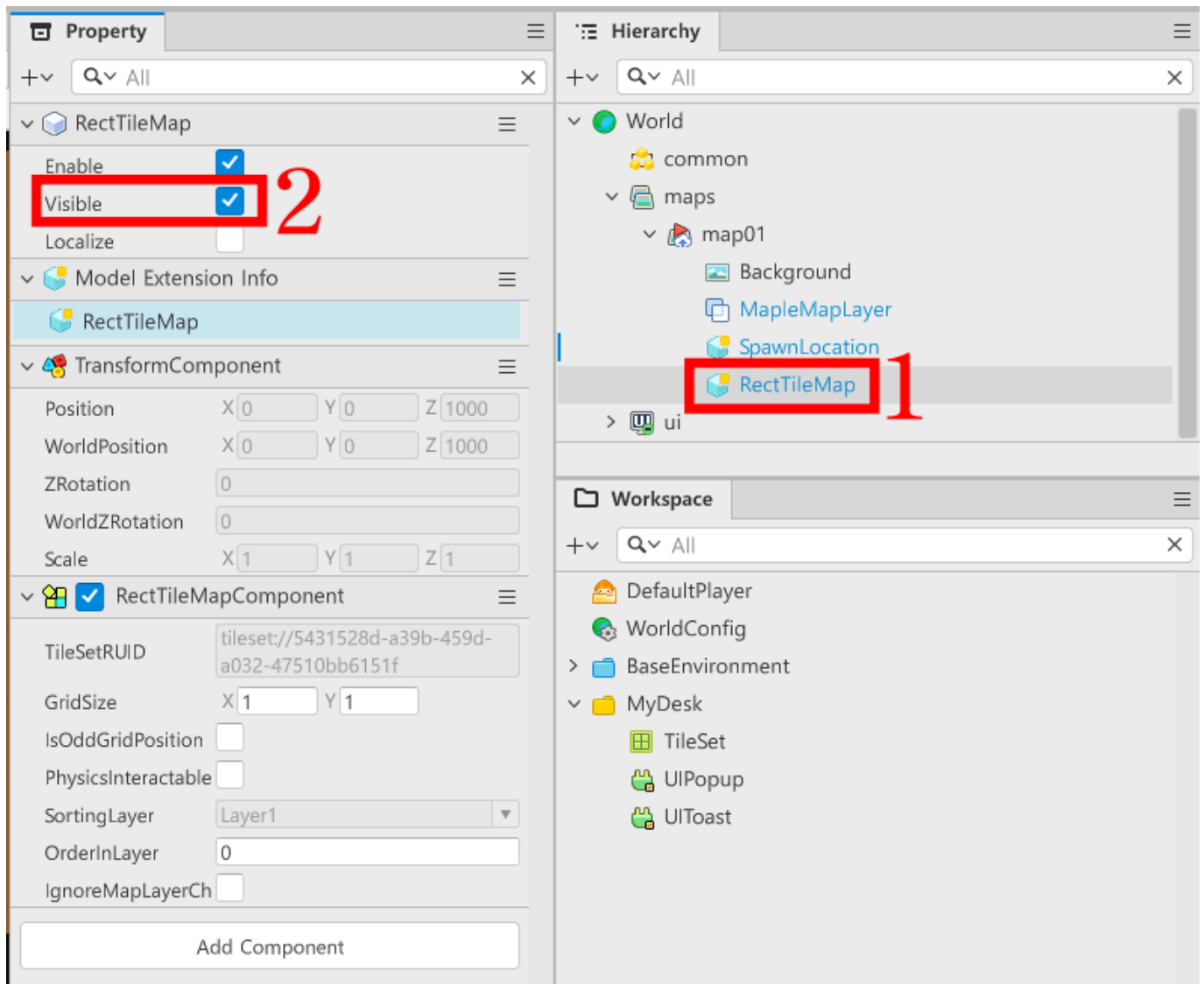


- 在編輯對象中點擊是否可移動。
- 點擊要改為不可移動的圖塊的綠色十字UI，將其變為紅色十字UI。

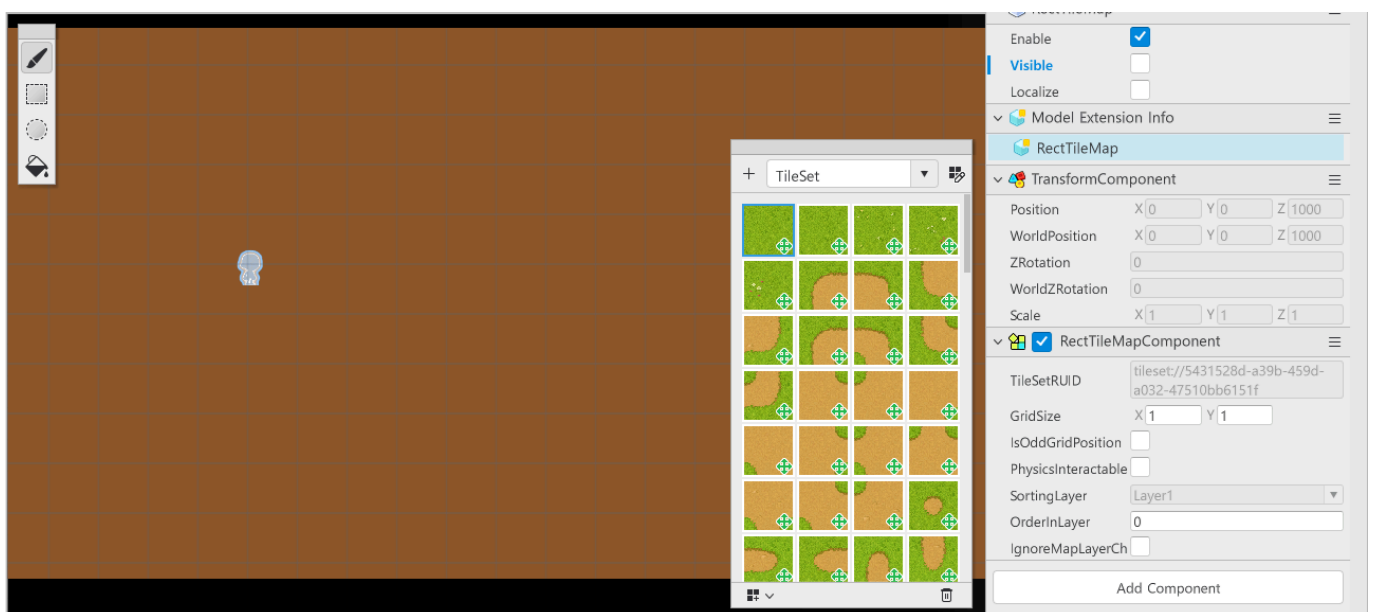


- 若已正常變更，在圖塊配置狀態下，UI會變更為紅色十字。
- 將該圖塊配置到地圖後，透過測試確認角色無法移動。

在已配置的圖塊地圖中隱藏

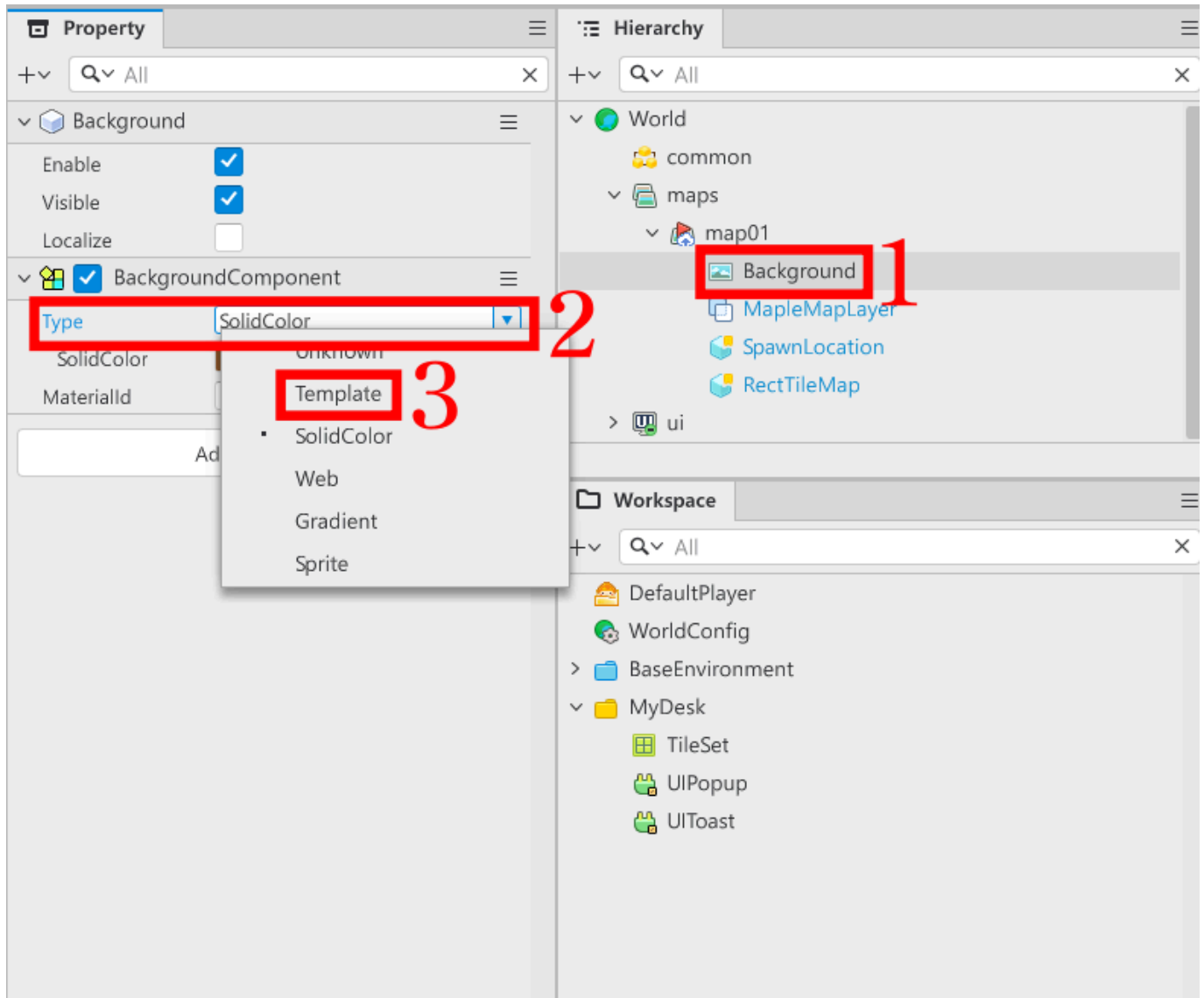


- 在Hierarchy視窗中找到RectTileMap並點擊。
- 在Property視窗中點擊Visible並取消勾選。

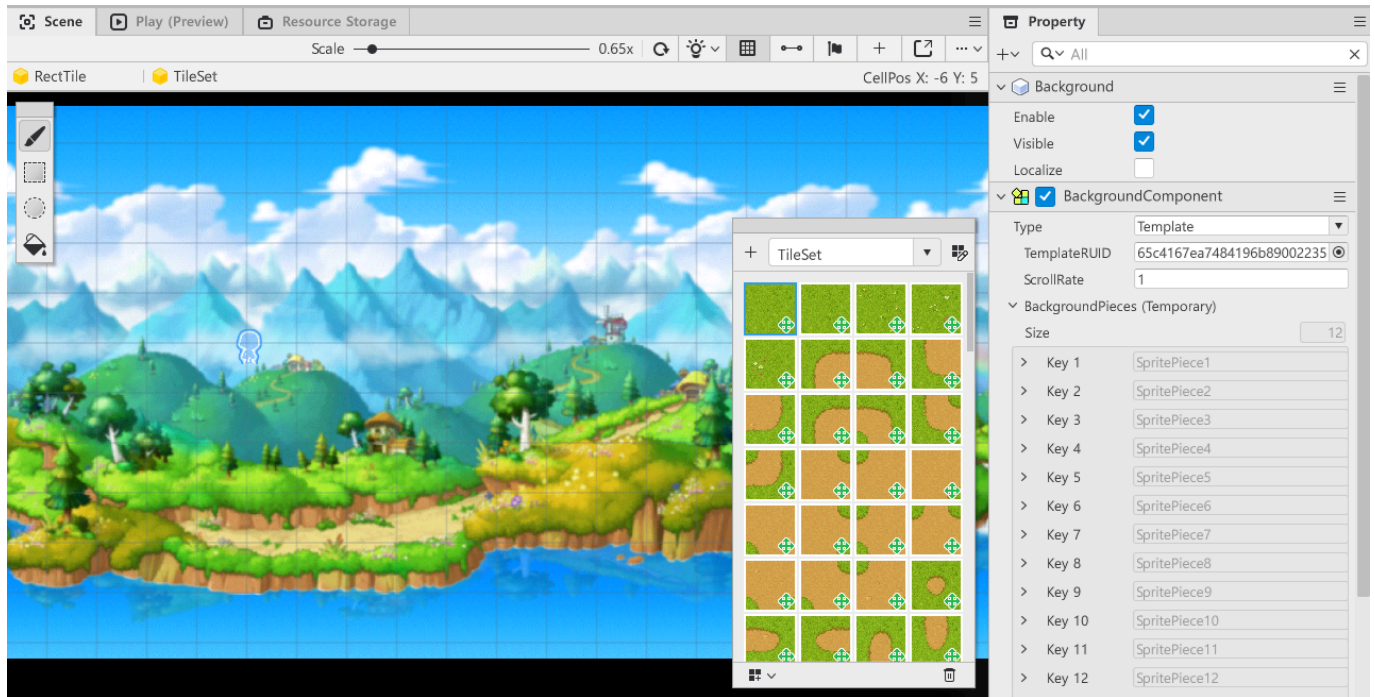


- 若成功取消Visible，Scene畫面會顯示隱藏Tile的畫面。

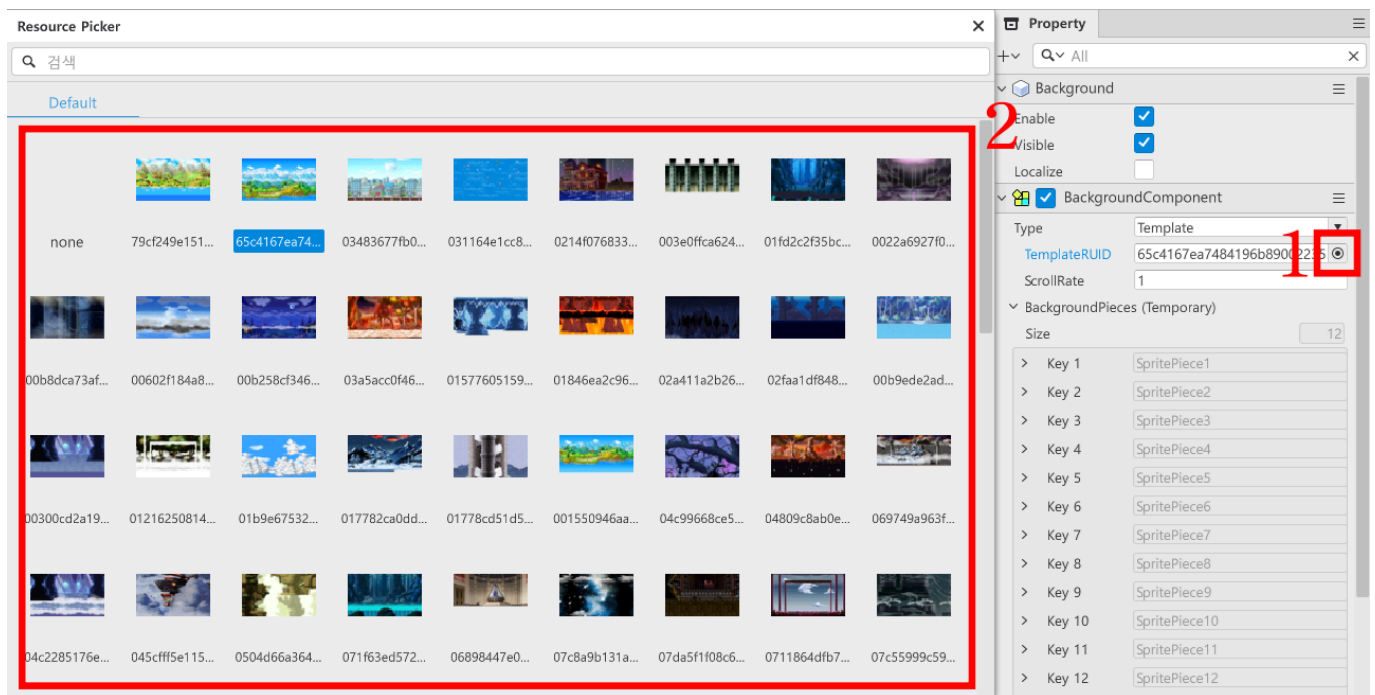
變更背景



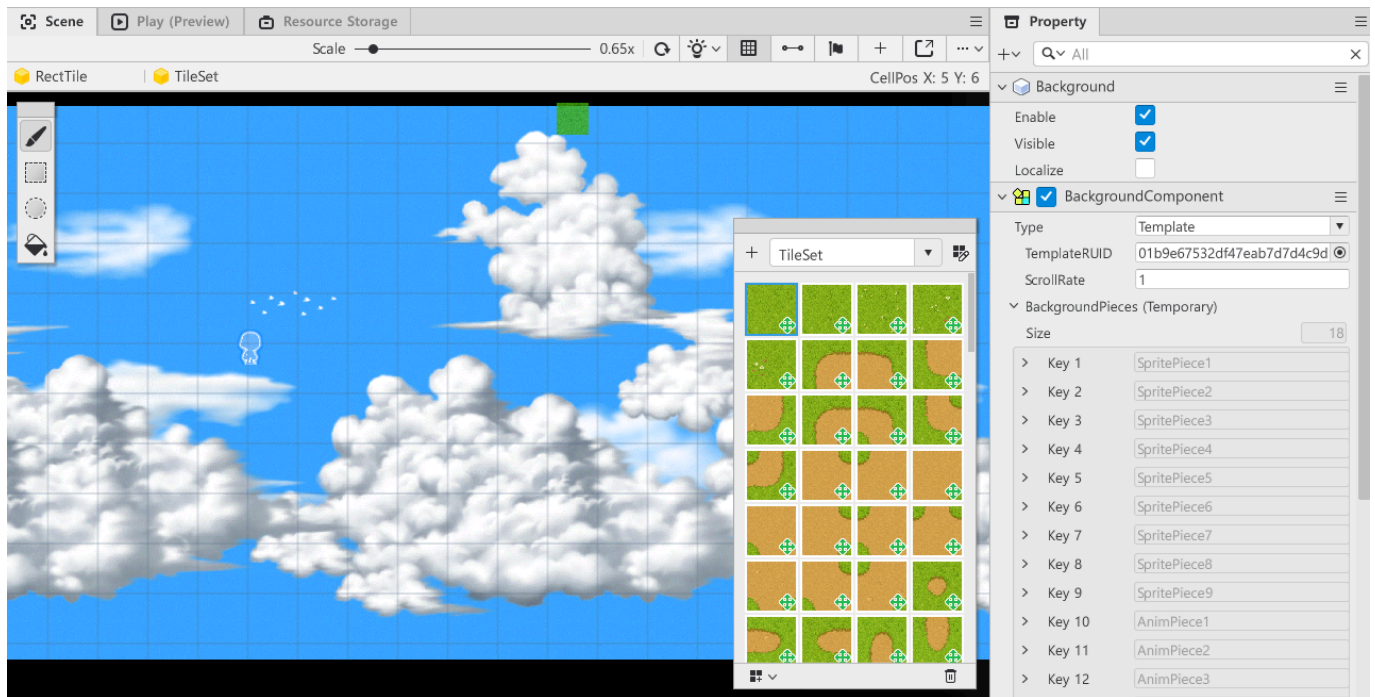
- 在Hierarchy視窗中找到Background並點擊。
- 在Property視窗中找到BackgroundComponent。
- 點擊Type。
- 從SolidColor變更為Template。



- 正常變更完成後，Scene畫面會從單色背景變為楓之谷世界提供的背景。



- 若要使用其他背景，在Property視窗中找到BackgroundComponent。
- 點擊TemplateRUID的Resource Picker按鈕。
- 找到並點擊想要的背景進行更換。



- 正常變更完成後，可在Scene畫面確認背景已變更。

最低實作標準

- 製作實際可使用的射擊世界地圖。
 - 必須利用TileSet區分可移動區域與不可移動區域。

應用與擴展方向

- 若使用楓之谷世界提供的資源，可自行製作資源後，利用自己的資源製作TileSet。
- 利用製作完成的TileSet，可以使用非楓之谷世界資源，而是實際可用的自製地圖。
- 學習CameraComponent，找出並套用讓視角不跟隨角色的方法。
 - 楓之谷世界官方文件，玩家相機控制文件[連結](#)

[💡 基礎學習]宣告Logic

影片時間軸

- 可在課程影片中以下時間點進行學習。
- Logic理解：00:23:57

學習目標

- 學習楓之谷世界提供的Logic功能。
- 理解Manager概念並設計以功能為單位的Logic。
- 利用Logic製作控制遊戲系統的Logic、管理資料的Logic等。

- 理解Logic的優缺點，並在之後製作Logic時加以考量設計。

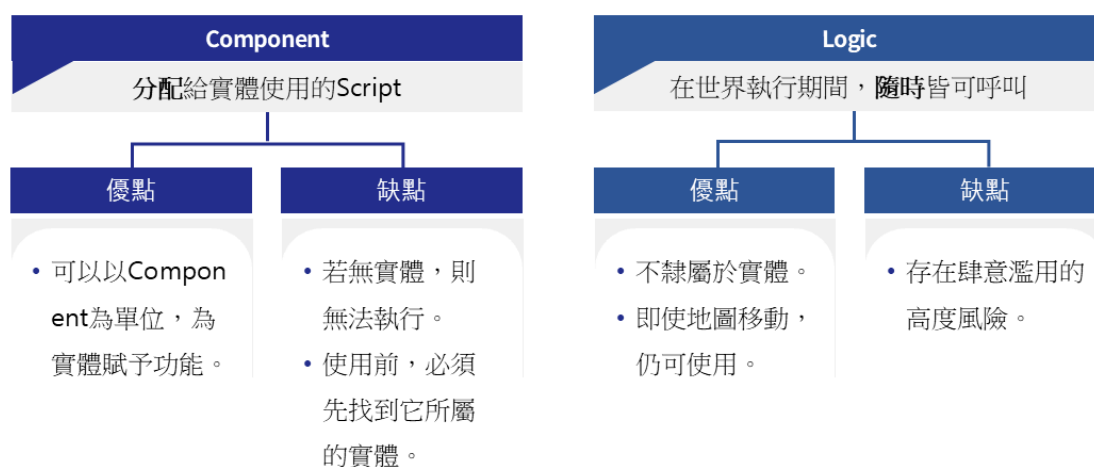
觀看該影片後可嘗試執行的內容

- 在空的實體上新增Component後，呼叫製作好的Logic，直接體驗Logic的特性。
- 親自製作具有Manager性質的Logic，並思考如何管理該Manager並進行設計。

Logic是什麼？

Key Point

依據實體從屬關係而產生的Script差異

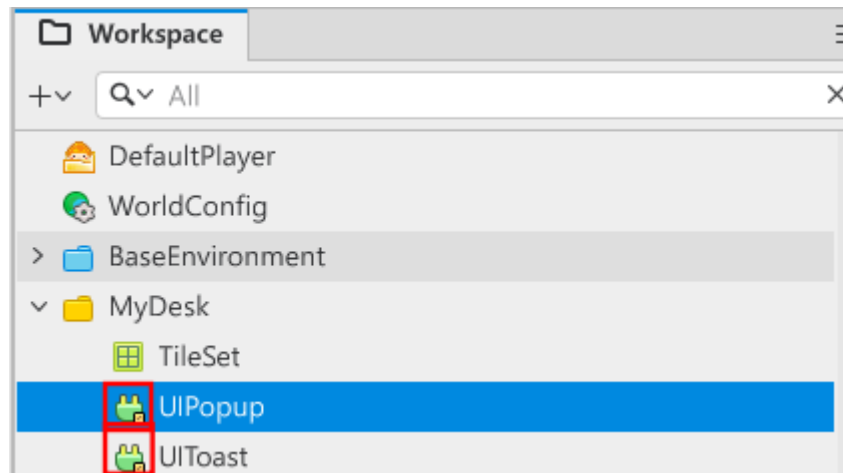


- 是楓之谷世界提供的腳本形式之一。
- 建立Logic時，會在Workspace中以綠色插頭圖示顯示。
- 與Component不同，即使不附加在實體上也可以存取腳本。

Manager概念

- Logic可以指定特定功能或角色，並以負責該功能的形式製作。
- 負責單一功能或角色的單位稱為Manager。
- 若特定功能發生問題，只需修改Manager內容即可快速解決。
 - Case 1：角色計算傷害並傳遞給怪物的情況
 - 每次變更傷害公式或新增計算要素時，都需要修改公式。
 - 需要逐一找到所有角色並修改輸入的程式碼。
 - Case 2：將傷害計算交由Logic製作的Manager處理
 - 在Logic中修改傷害計算公式。
 - 因為所有角色與怪物的傷害計算都由Logic負責，不需要再修改其他部分。
 - 即使之後傷害計算出現問題，也只需修改Manager即可解決。

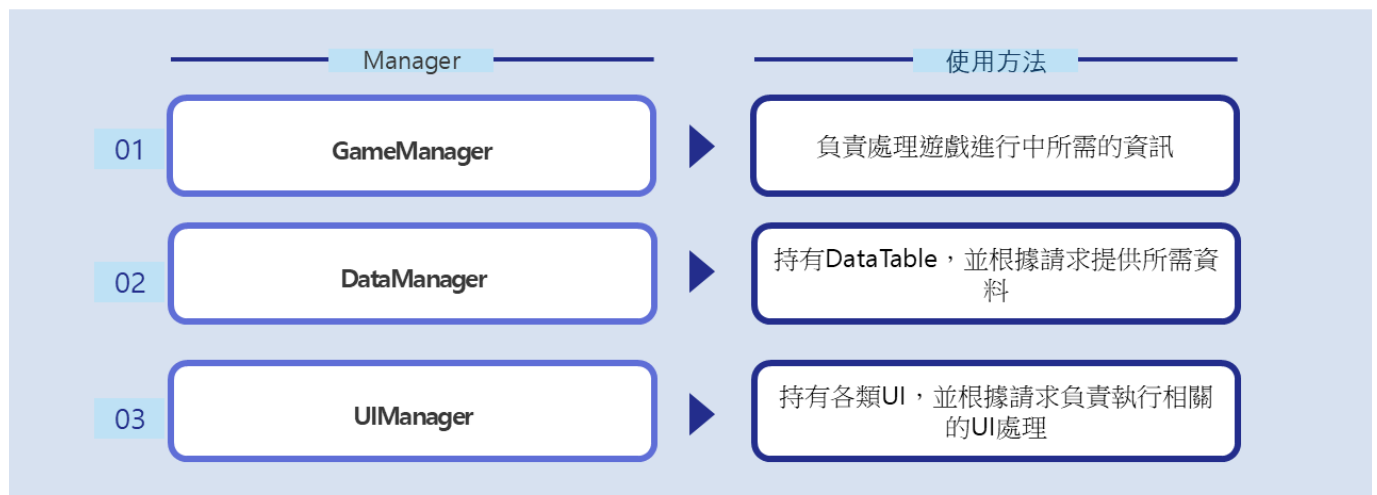
Logic製作方法



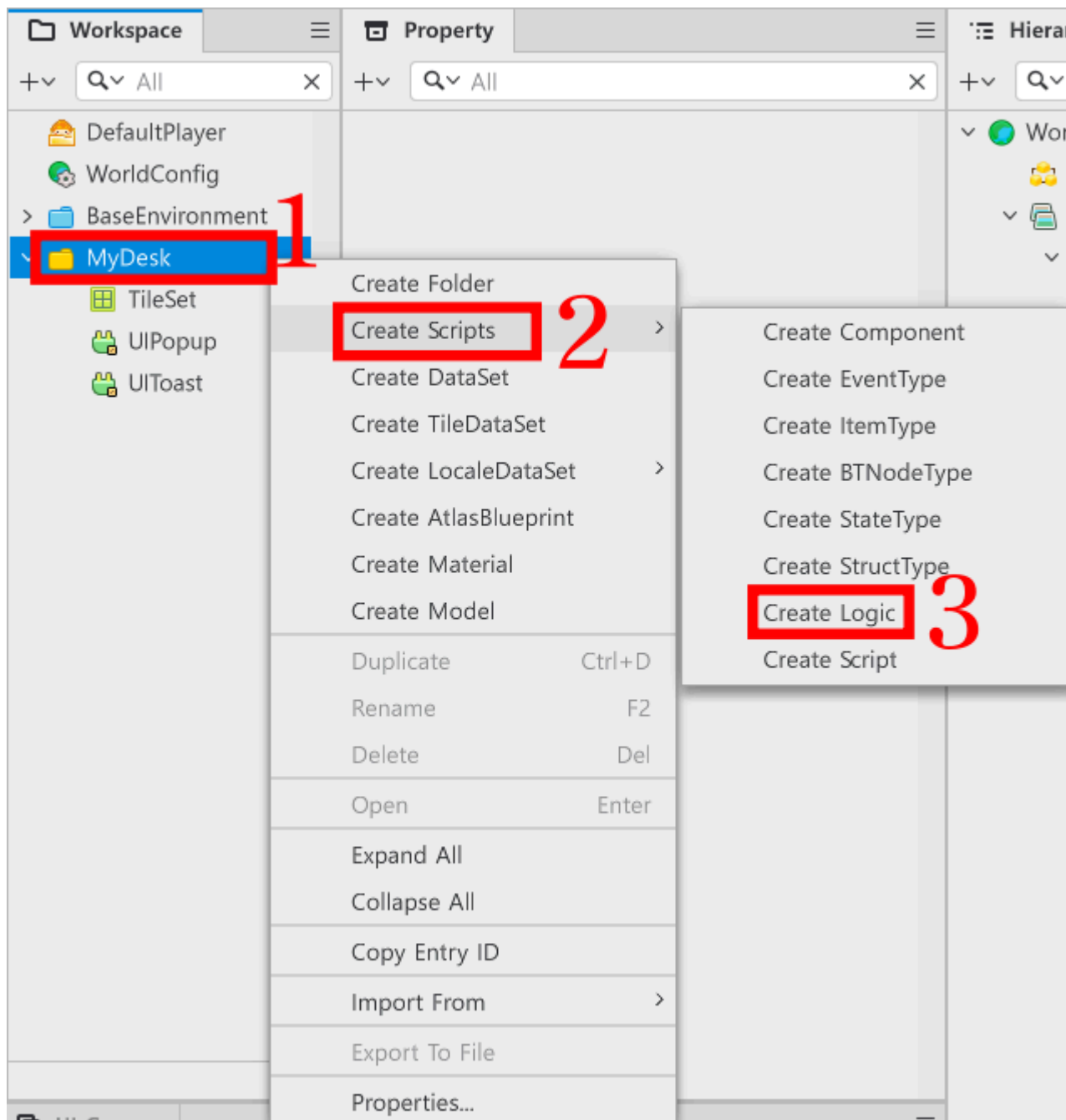
- 在Workspace中對MyDesk按右鍵。
- 在選單畫面中點擊Create Scripts。
- 點擊Create Logic以建立新的Logic。

Key Point

透過Manager配置以負責特定功能



- 將建立的Logic名稱修改為想要的名稱。



- 完成名稱修改後，可在Workspace中看到Logic已建立。

範例世界中Manager Logic的使用用途

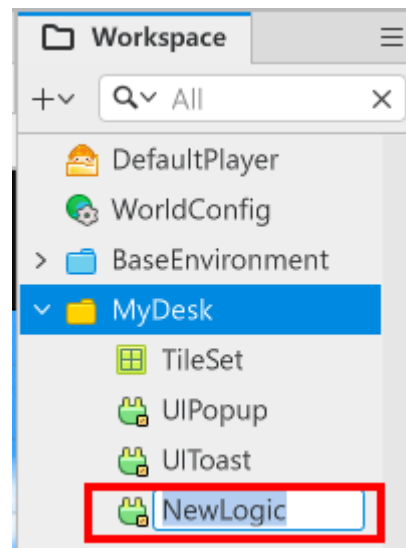
姓名	使用用途
GameLogic	為了管理隨著遊戲進行需要處理的功能而製作的Manager Logic。 負責遊戲通關判定、UI呼叫、角色瞬間移動等遊戲進行所需處理。
DataTableLogic	用於管理與使用DataSet的Logic。 每次需要資料時逐一載入DataSet會很麻煩。 當遊戲開始時，DataTableLogic會將所有DataSet進行快取，並在需要時回傳已快取的資料。

姓名	使用用途
UIManageLogic	用於管理UIGroups的Manager Logic。 若元件或GameLogic需要處理與UI相關的內容，就會將請求傳遞給該Manager Logic。 由於UIManageLogic持有所有UIGroups的元件，因此會專責處理所有與UI相關的請求。

- 像上述這樣，範例世界會建立並使用Logic。
- 當需要製作或擴充其他功能時，也可以宣告並製作屬於你們自己的Logic來使用。

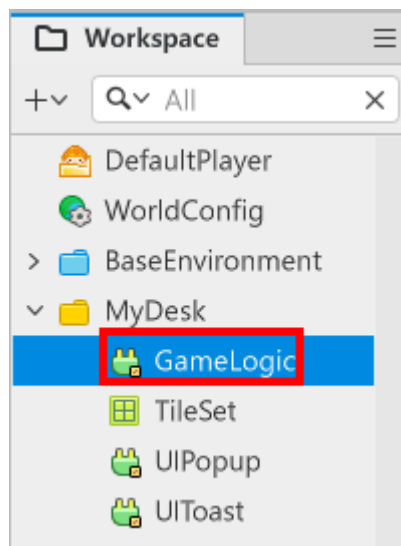
範例世界內Logic使用範例

GameLogic的情況



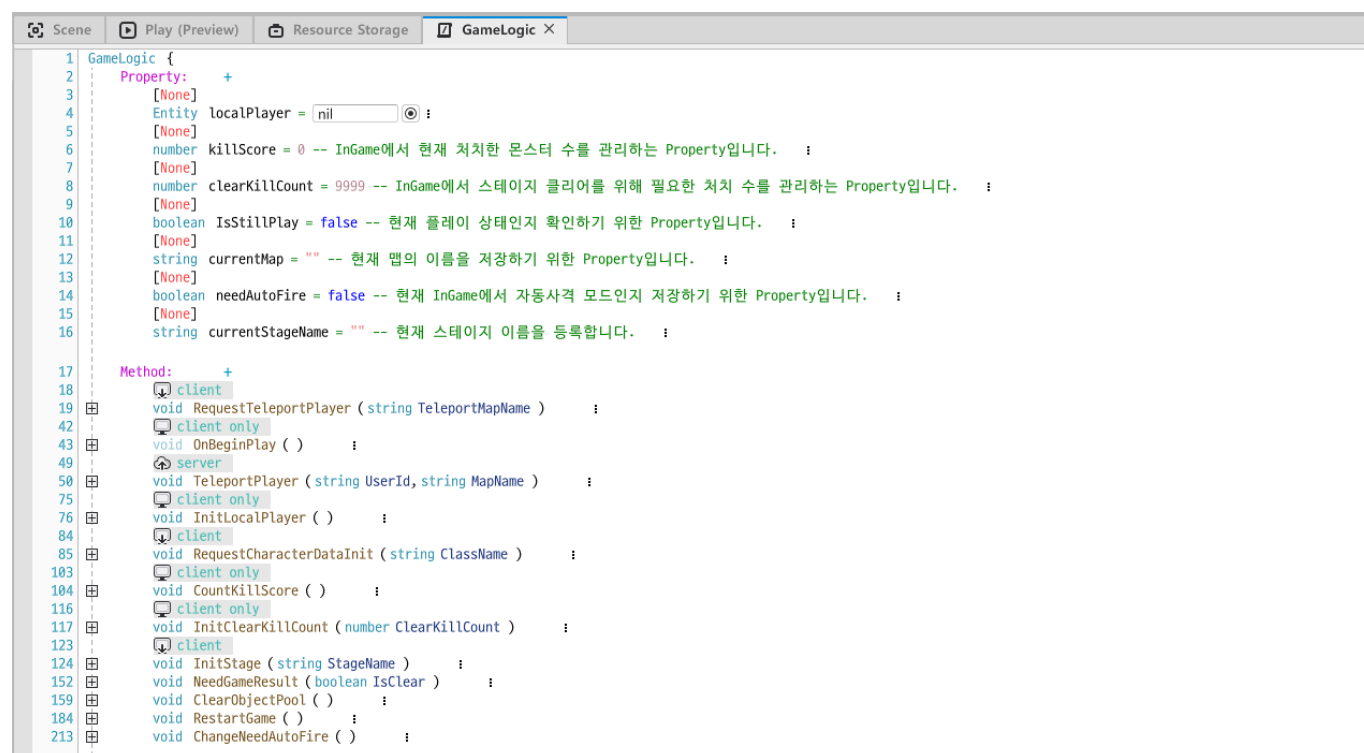
- 正在處理用於變更使用者實體所在地圖位置的功能。
- 當遊戲開始時，會計算目前Stage還需要再處理多少敵人，並計算已處理的數量。
- 當遊戲通關或GameOver時，會判斷是否呼叫對應的UI演出。
- 可以宣告並變更玩家角色是否需要自動攻擊。

DataTableLogic的情況



- 當遊戲開始時，會自行快取所有DataSet。
- 會依照請求，將已快取的DataSet以加工後的形式再次回傳。

UIManageLogic的情況



- 當遊戲開始時，會把各個UIGroups中負責自行管理的Component全部註冊起來。
- 在遊戲進行期間，若有與UI相關的處理(顯示UI、隱藏UI、更新UI等)，便會接收請求並指示對應UI執行作業。

撰寫GameLogic程式碼

```
@Logic
script GameLogic extends Logic
```

```
property Entity localPlayer = nil

property number killScore = 0 -- 用來管理InGame中目前已擊殺怪物數量的Property。

property number clearKillCount = 9999 -- 用來管理InGame中Stage通關所需擊殺數的Property。

property boolean IsStillPlay = false -- 用來確認目前是否為遊玩中的Property。

property string currentMap = "" -- 用來儲存目前地圖名稱的Property。

property boolean needAutoFire = false -- 用來儲存目前InGame是否為自動射擊模式的Property。

property string currentStageName = "" -- 登錄目前Stage名稱。

@ExecSpace("Client")
method void RequestTeleportPlayer(string TeleportMapName)
log("GameManager RequestTeleportPlayer 開始")

-- 若目前沒有Player, 則尋找LocalPlayer。
if not invalid(self.localPlayer) then
    self:InitLocalPlayer()
    -- 若沒有LocalPlayer, 則中斷程式執行。
    if not invalid(self.localPlayer) then return end
end

-- 讓UI能依目前地圖進行變更。
if TeleportMapName == "Title" then
    _UIManageLogic:ShowTitle()
elseif TeleportMapName == "InGame" then
    _UIManageLogic:ShowInGame()
end

self.currentMap = TeleportMapName
-- 在楓之谷世界中, 玩家實體的Name與UserId相同。
self:TeleportPlayer(self.localPlayer.Name, TeleportMapName)
end

@ExecSpace("ClientOnly")
method void OnBeginPlay()
-- 當遊戲開始時, 尋找並登錄本機玩家。
self:InitLocalPlayer()
end

@ExecSpace("Server")
method void TeleportPlayer(string UserId, string MapName)
-- 1.透過UserId再次從伺服器尋找實體。
log("成功進入TeleportPlayer")
local playerEntity = _UserService:GetUserEntityByUserId(UserId)
-- 若找不到playerEntity, 則輸出錯誤日誌。
if not invalid(playerEntity) then
    log("瞬間移動失敗: 找不到使用者。")
    return
```

```

end

-- 指定要移動的路徑。
local targetPath = ""
if MapName == "Title" then
    targetPath = "/maps/Title"
elseif MapName == "InGame" then
    targetPath = "/maps/InGame"
end
-- 若移動路徑已正常登錄，則將玩家角色瞬間移動到該路徑。
if targetPath ~= "" then
    _TeleportService:TeleportToEntityPath(playerEntity, targetPath)
    log("瞬間移動執行完成: " ..targetPath)
end
end

@ExecSpace("ClientOnly")
method void InitLocalPlayer()
-- 尋找並登錄本機玩家。
if not isValid(self.localPlayer) then
    self.localPlayer = _UserService.LocalPlayer
end
end

@ExecSpace("ClientOnly")
method void RequestCharacterDataInit(string ClassName)
-- 以傳入的CharacterDefaultData的ID值為基準，初始化PlayerState。
if ClassName ~= "" and ClassName ~= nil then
    -- 在DataTableManager中以ClassName尋找對應的資料表。
    local findPlayerData =
_DataTableLogic:GetCharacterDefaultDataByCharacterID(ClassName)
    -- 若找到，則以找到的資料初始化PlayerState。
    if isValid(findPlayerData) and next(findPlayerData) ~= nil then
        log("完成CharacterDefaultData Init請求")
        self.localPlayer.PlayerState:InitPlayerData(findPlayerData)
    -- 若找不到，則留下log。
    else
        log("非正常的CharacterDefaultData Init失敗! ")
    end
end
end

@ExecSpace("ClientOnly")
method void CountKillScore()
-- 若怪物死亡，killScore就增加1。
self.killScore = self.killScore + 1
-- 若killScore高於clearKillCount，則視為Stage通關。
if self.killScore >= self.clearKillCount then
    self.IsStillPlay = false
    self:NeedGameResult(true)
    log("遊戲通關")
end
end

```

```

@ExecSpace("ClientOnly")
method void InitClearKillCount(number ClearKillCount)
-- 以計算出的怪物數初始化Stage通關條件。
self.clearKillCount = ClearKillCount
end

@ExecSpace("ClientOnly")
method void InitStage(string StageName)
-- 以StageName值為基準，指定該地圖MonsterSpawnManager的路徑。
local spawnManagerPath = "/maps/"..StageName.."/MonsterSpawnManager"
-- 從預先指定的路徑中尋找MonsterSpawnManager實體。
local stageSpawnManager = _EntityService:GetEntityByPath(spawnManagerPath)
-- 若尋找MonsterSpawnManager失敗，則留下log並return。
if not isValid(stageSpawnManager) then
    log(StageName.."中找不到MonsterSpawnManager物件! ")
    return
end
-- 呼叫方法，讓ItemHelper登錄目前地圖的ItemSpawnManager。
_ItemHelper:InitData()
-- 若目前狀態不是遊戲進行中，則執行遊戲並將IsStillPlay的值改為true。
if not self.IsStillPlay then
    stageSpawnManager.MonsterSpawnManager:StartGame()
    self.IsStillPlay = true
end
-- 將目前怪物擊殺計數初始化為0。
self.killScore = 0
-- 由於已完成所有程序，因此登錄傳入的Stage名稱。
self.currentStageName = StageName
-- 暫時中斷所有BGM。
_SoundService:StopBGM(true)
-- 播放InGame中應輸出的BGM。
_SoundService:PlayBGM("d155c5573a3f46a7b2ffe38629441132", 1.0)
end

@ExecSpace("ClientOnly")
method void NeedGameResult(boolean IsClear)
-- 傳遞遊戲通關結果，並播放依結果對應的UI演出。
self.IsStillPlay = false
_UIManageLogic:ShowResultUI(IsClear)
end

@ExecSpace("ClientOnly")
method void ClearObjectPool()
-- 若在非InGame的其他地圖中呼叫該方法，則忽略以下程式碼。
if not self.currentMap == "InGame" then
    return
end
-- 指定InGame地圖內各ObjectPool管理Manager的路徑。
local missilePoolPath = "/maps/"..self.currentMap.."/ProjectileManager"
local monsterPoolPath = "/maps/"..self.currentMap.."/MonsterSpawnManager"

-- 依據上面指定的路徑尋找Manager。
local missilePoolEntity = _EntityService:GetEntityByPath(missilePoolPath)
local monsterPoolEntity = _EntityService:GetEntityByPath(monsterPoolPath)

```

```

-- 若成功找到，則將各物件池的實體設為非啟用狀態。
if invalid(missilePoolEntity) then
    missilePoolEntity.ProjectileManager:ResetPool()
end
if invalid(monsterPoolEntity) then
    monsterPoolEntity.MonsterSpawnManager:ClearAllChild()
end

-- 防止Helper類別在非InGame地圖中被呼叫而引發錯誤。
_ItemHelper:ReleaseData()
end

@ExecSpace("ClientOnly")
method void RestartGame()
-- 尋找本機玩家。
local findLocalPlayer = _UserService.LocalPlayer
-- 若找不到本機玩家，則輸出log後return。
if not invalid(findLocalPlayer) then
    log("找不到LocalPlayer! ")
    return
end
-- 若本機玩家實體上沒有PlayerState元件，則輸出log後return。
if not invalid(findLocalPlayer.PlayerState) then
    log("找不到LocalPlayer的PlayerState元件! ")
    return
end

--取得PlayerState後，尋找目前角色的ID值。
local currentPlayerCharacter = findLocalPlayer.PlayerState.characterID
-- 為了重新開始，初始化ObjectPool。
self:ClearObjectPool()
-- 初始化欲重新開始的Stage資料。
self:InitStage(self.currentStageName)
-- 為了將角色資訊恢復為初始狀態，再次初始化資料。
self:RequestCharacterDataInit(currentPlayerCharacter)
-- 將UI狀態切換為InGame開始狀態。
_UIManageLogic:ShowInGame()
-- 由於遊戲已重新開始，因此將IsStillPlay狀態切換為true。
self.IsStillPlay = true
end

@ExecSpace("ClientOnly")
method void ChangeNeedAutoFire()
-- 若呼叫該函式，則將目前needAutoFire的值改為相反值。
self.needAutoFire = not self.needAutoFire
end

end

```

撰寫DataTableLogic程式碼

```

@Logic
script DataTableLogic extends Logic

property table missileData = {} -- 用來管理MissileData DataSet的Property。

property table characterDefaultData = {} -- 用來管理CharacterDefaultData DataSet的Property。

property table characterSelectUIData = {} -- 用來管理CharacterSelectUIData DataSet的Property。

property table monsterDefaultData = {} -- 用來管理MonsterDefaultData DataSet的Property。

property table monsterSpawnData = {} -- 用來管理MonsterSpawnData DataSet的Property。

property table skillData = {} -- 用來管理SkillData DataSet的Property。

property table itemData = {} -- 用來管理ItemData DataSet的Property。

property table itemDropData = {} -- 用來管理ItemDropData DataSet的Property。

@ExecSpace("ClientOnly")
method void OnBeginPlay()
-- 初始化MissileData DataSet。
self:InitMissileData()
-- 初始化CharacterSelectUIData DataSet。
self:InitCharacterSelectUIData()
-- 初始化MonsterSpawnData DataSet。
self:InitMonsterSpawnData()
-- 初始化MonsterDefaultData DataSet。
self:InitMonsterDefaultData()
-- 初始化CharacterDefaultData DataSet。
self:InitCharacterDefaultData()
-- 初始化SkillData DataSet。
self:InitSkillData()
-- 初始化ItemData DataSet。
self:InitItemData()
-- 初始化ItemDropData DataSet。
self:InitItemDropData()
end

@ExecSpace("ClientOnly")
method void InitMissileData()
-- 檢查是否已完成投射體資料表解析。
if isvalid(self.missileData) == false or next(self.missileData) == nil then
-- 因為需要解析投射體資料，所以尋找MissileData資料表。
local missileDefaultData = _DataService:GetTable("MissileData")
if isvalid(missileDefaultData) == nil then
-- 由於找不到MissileData表，因此輸出日誌後Return。
log("找不到MissileData資料表！")

```

```

        return
    end
    -- 計算全部Row數量。
    local rowCount = missileDefaultData:GetRowCount()

    -- 先依序從資料表Row中由1開始代入作為Key的MissileID。
    for i = 1, rowCount do
        local missileID = missileDefaultData:GetCell(i, "MissileID")

        -- 檢查是否為有效ID。
        if missileID ~= nil and missileID ~= "" then

            -- 將該列的所有資訊綁成一個table。
            local rowData =
            {
                MissileID = missileID,
                AtkValue = tonumber(missileDefaultData:GetCell(i, "AtkValue")),
                MoveSpeed = tonumber(missileDefaultData:GetCell(i, "MoveSpeed")),
                LifeTime = tonumber(missileDefaultData:GetCell(i, "LifeTime")),
                HitEffectRUID = tostring(missileDefaultData:GetCell(i,
"HitEffectRUID"))),
                ModelRUID = tostring(missileDefaultData:GetCell(i, "ModelRUID"))
            }

            -- 登錄到missileData Property中。(Key: MissileID, Value: 資料)
            self.missileData[missileID] = rowData
        end
    end
    -- 輸出表示資料表快取完成的Log。
    log("MissileData資料表快取完成! ")
    return
end

-- 由於尚未進行資料表快取，因此輸出提示該情況的日誌。
log("MissileData資料表已經快取過，或是找不到MissileData資料表。")
end

@ExecSpace("ClientOnly")
method void InitCharacterDefaultData()
    -- 用於檢查玩家角色解析是否完成的項目
    if isValid(self.characterDefaultData) == false or next(self.characterDefaultData)
    == nil then
        local getTableData = _DataService:GetTable("CharacterDefaultData")
        --若取得資料表失敗則輸出警告訊息
        if getTableData == nil then
            log("取得CharacterDefaultData資料表失敗! ")
            return
        end
        --開始解析
        local rowCount = getTableData:GetRowCount() -- 計算全部列數。

        for i = 1, rowCount do
            -- 先從資料表Row中代入一個作為Key的CharacterID。
            local characterID = tostring(getTableData:GetCell(i, "CharacterID"))

```

```

-- 檢查是否為有效ID。
if characterID ~= nil and characterID ~= "" then

    -- 將該列的所有資訊綁成table。
    local rowData =
    {
        CharacterID = characterID,
        StartHP = tonumber(getTableData:GetCell(i, "StartHP")),
        StartSkillCount = tonumber(getTableData:GetCell(i,
"StartSkillCount")),
        StartMoveSpeed = tonumber(getTableData:GetCell(i,
"StartMoveSpeed")),
        DefaultMissile = tostring(getTableData:GetCell(i,
"DefaultMissile")),
        DefaultSkill = tostring(getTableData:GetCell(i, "DefaultSkill")),
        AttackInterval = tonumber(getTableData:GetCell(i,
"AttackInterval"))
    }

    -- 儲存到快取table中(Key: MissileID, Value: 資料)
    self.characterDefaultData[characterID] = rowData
    end
end
-- 輸出表示資料表快取完成的Log。
log("CharacterDefaultData資料表快取完成！")
return
end
-- 由於尚未進行資料表快取，因此輸出提示該情況的日誌。
log("CharacterDefaultData資料表已經快取過，或是找不到CharacterDefaultData資料表。")
end

@ExecSpace("ClientOnly")
method void InitCharacterSelectUIData()
-- 檢查角色選擇視窗資訊表是否已快取。
if isValid(self.characterSelectUIData) == false or
next(self.characterSelectUIData) == nil then
    -- 因為角色選擇視窗資訊表尚未登錄，所以尋找資料表以快取該部分。
    local characterSelectData = _DataService:GetTable("CharacterSelectUIData")
    -- 若找不到角色選擇視窗資訊表，則以log輸出錯誤。
    if characterSelectData == nil then
        log("找不到CharacterSelectUIData資料表！")
        return
    end
    -- 若正常找到，則計算資料表的RowCount。
    local rowCount = characterSelectData:GetRowCount()
    -- 先依序從資料表Row中由1開始代入作為Key的CharacterID。
    for i = 1, rowCount do
        local characterID = characterSelectData:GetCell(i, "CharacterID")

        -- 檢查是否為有效ID。
        if characterID ~= nil and characterID ~= "" then

            -- 將該列的所有資訊綁成一個table。
            local rowData =

```

```

        {
            CharacterID = characterID,
            LocalizeNameID = tostring(characterSelectData:GetCell(i,
"LocalizeNameID")),
            SpriteRUID = tostring(characterSelectData:GetCell(i,
"SpriteRUID"))
        }

        -- 儲存到快取table中(Key: MissileID, Value: 資料)
        self.characterSelectUIData[characterID] = rowData
    end
end
-- 輸出表示資料表快取完成的Log。
log("CharacterSelectUIData資料表快取完成! ")
return
end
-- 由於尚未進行資料表快取，因此輸出提示該情況的日誌。
log("CharacterSelectUIData資料表已經快取過，或是找不到MissileData資料表。")
end

@ExecSpace("ClientOnly")
method void InitMonsterDefaultData()
-- 檢查怪物資料解析是否完成。
if isValid(self.monsterDefaultData) == false or next(self.monsterDefaultData) ==
nil then
    local getTableData = _DataService:GetTable("MonsterDefaultData")
    --若取得資料表失敗則輸出警告訊息。
    if getTableData == nil then
        log("取得MonsterDefaultData資料表失敗! ")
        return
    end
    -- 計算全部列數。
    local rowCount = getTableData:GetRowCount()

    for i = 1, rowCount do
        -- 先從資料表Row中代入一個作為Key的MissileID。
        local monsterID = getTableData:GetCell(i, "MonsterID")

        -- 檢查是否為有效ID。
        if monsterID ~= nil and monsterID ~= "" then

            -- 將該列的所有資訊綁成table。
            local rowData =
            {
                monsterID = monsterID,
                StartHP = tonumber(getTableData:GetCell(i, "StartHP")),
                StartMoveSpeed = tonumber(getTableData:GetCell(i,
"StartMoveSpeed")),
                DefaultMissile = getTableData:GetCell(i, "DefaultMissile"),
                AttackInterval = tonumber(getTableData:GetCell(i,
"AttackInterval")),
                ModelRUID = tostring(getTableData:GetCell(i, "ModelRUID"))
            }

```

```

        -- 儲存到快取table中(Key: MissileID, Value: 資料)
        self.monsterDefaultData[monsterID] = rowData
    end
end
-- 輸出表示資料表快取完成的Log。
log("MonsterDefaultData資料表快取完成! ")
return
end
-- 由於尚未進行資料表快取，因此輸出提示該情況的日誌。
log("MonsterDefaultData資料表已經快取過，或是找不到MonsterDefaultData資料表。")
end

@ExecSpace("ClientOnly")
method void InitMonsterSpawnData()
-- 檢查怪物產生資料解析是否完成。
if isvalid(self.monsterSpawnData) == false or next(self.monsterSpawnData) == nil
then
    local getTableData = _DataService:GetTable("MonsterSpawnData")
    --若取得資料表失敗則輸出警告訊息
    if getTableData == nil then
        log("取得MonsterSpawnData資料表失敗! ")
        return
    end
    -- 計算全部列數。
    local rowCount = getTableData:GetRowCount()

    for i = 1, rowCount do
        -- 先從資料表Row中代入一個作為Key的MissileID。
        local stageID = getTableData:GetCell(i, "StageID")

        -- 檢查是否為有效ID。
        if stageID ~= nil and stageID ~= "" then

            -- 將該列的所有資訊綁成table。
            local rowData =
            {
                StageID = getTableData:GetCell(i, "StageID"),
                MonsterID = tostring(getTableData:GetCell(i, "MonsterID")),
                SpawnPosX = tonumber(getTableData:GetCell(i, "SpawnPosX")),
                SpawnPosY = tonumber(getTableData:GetCell(i, "SpawnPosY")),
                EntranceTime = tonumber(getTableData:GetCell(i, "EntranceTime")),
                ItemDropRate = tonumber(getTableData:GetCell(i, "ItemDropRate")),
                MustDropItem = tostring(getTableData:GetCell(i, "MustDropItem"))
            }
            -- 將要登錄到key值的stageID轉換為number形式。
            stageID = tonumber(stageID)
            -- 儲存到快取table。(Key: MissileID, Value: 資料)
            if self.monsterSpawnData[stageID] == nil then
                self.monsterSpawnData[stageID] = {}
            end
            table.insert(self.monsterSpawnData[stageID], rowData)
        end
    end
end
-- 輸出表示資料表快取完成的Log。

```

```

    log("MonsterSpawnData資料表快取完成! ")
    return
end
-- 由於尚未進行資料表快取, 因此輸出提示該情況的日誌。
log("MonsterSpawnData資料表已經快取過, 或是找不到MonsterSpawnData資料表。")
end

@ExecSpace("ClientOnly")
method void InitSkillData()
-- 若沒有已快取的skillData, 則嘗試快取資料表。
if not isvalid(self.skillData) or not next(self.skillData) then
    local findData = _DataService:GetTable("SkillData")
    --若找不到SkillData, 則留下日誌並離開程式碼。
    if not isvalid(findData) then
        log("SkillData載入失敗! ")
        return
    end
    -- 計算全部列數。
    local rowCount = findData:GetRowCount()

    for i = 1, rowCount do
        -- 先從資料表Row中代入一個作為Key的MissileID。
        local skillID = findData:GetCell(i, "SkillID")

        -- 檢查是否為有效ID。
        if skillID ~= nil and skillID ~= "" then

            -- 將該列的所有資訊綁成table。
            local rowData =
            {
                SkillID = findData:GetCell(i, "SkillID"),
                SkillType = findData:GetCell(i, "SkillType"),
                IsPierce = findData:GetCell(i, "IsPierce"),
                AtkValue = tonumber(findData:GetCell(i, "AtkValue")),
                MoveSpeed = tonumber(findData:GetCell(i, "MoveSpeed")),
                LifeTime = tonumber(findData:GetCell(i, "LifeTime")),
                MaxHitCount = tonumber(findData:GetCell(i, "MaxHitCount")),
                HitInterval = tonumber(findData:GetCell(i, "HitInterval")),
                ModelRUID = findData:GetCell(i, "ModelRUID"),
                HitEffectRUID = findData:GetCell(i, "HitEffectRUID")
            }

            -- 儲存到快取table。(Key: MissileID, Value: 資料)
            self.skillData[skillID] = rowData
        end
    end
    -- 輸出表示資料表快取完成的Log。
    log("SkillData資料表快取完成! ")
    return
end
-- 由於尚未進行資料表快取, 因此輸出提示該情況的日誌。
log("SkillData資料表已經快取過, 或是找不到SkillData資料表。")
end

```

```

@ExecSpace("ClientOnly")
method void InitItemData()
-- 檢查是否已完成ItemData資料表解析。
if not isvalid(self.itemData) or not next(self.itemData) then
-- 因為需要解析ItemData資料表，所以尋找ItemData資料表。
local itemDefaultData = _DataService:GetTable("ItemData")
if not isvalid(itemDefaultData) then
-- 由於找不到ItemData資料表，因此輸出日誌後Return。
log("找不到ItemData資料表! ")
return
end
-- 計算全部Row數量。
local rowCount = itemDefaultData:GetRowCount()

-- 先依序從資料表Row中由1開始代入作為Key的ItemID。
for i = 1, rowCount do
local itemID = itemDefaultData:GetCell(i, "ItemID")

-- 檢查是否為有效ID。
if itemID ~= nil and itemID ~= "" then

-- 將該列的所有資訊綁成一個table。
local rowData =
{
ItemID = itemID,
ValueType = tonumber(itemDefaultData:GetCell(i, "ValueType")),
TargetPlayerStateName = tostring(itemDefaultData:GetCell(i,
"TargetPlayerStateName")),
NumberValue = tonumber(itemDefaultData:GetCell(i, "NumberValue")),
StringValue = tostring(itemDefaultData:GetCell(i, "StringValue")),
SpriteRUID = tostring(itemDefaultData:GetCell(i, "SpriteRUID")),
GainEffect = tostring(itemDefaultData:GetCell(i, "GainEffect"))
}

-- 儲存到快取table。(Key: ItemID, Value: 資料)
self.itemData[itemID] = rowData
end
end
-- 輸出表示資料表快取完成的Log。
log("ItemData資料表快取完成! ")
return
end
-- 由於尚未進行資料表快取，因此輸出提示該情況的日誌。
log("ItemData資料表已經快取過，或是找不到ItemData資料表。")
end

@ExecSpace("ClientOnly")
method void InitItemDropData()
-- 檢查是否已完成ItemDropData資料表解析。
if not isvalid(self.itemDropData) or not next(self.itemDropData) then
-- 因為需要解析ItemDropData資料表，所以尋找ItemDropData資料表。
local itemDropDefaultData = _DataService:GetTable("ItemDropData")
if not isvalid(itemDropDefaultData) then
-- 由於找不到ItemDropData資料表，因此輸出日誌後Return。

```

```

        log("找不到ItemDropData資料表! ")
        return
    end
    -- 計算全部Row數量。
    local rowCount = itemDropDefaultData:GetRowCount()

    -- 先依序從資料表Row中由1開始代入作為Key的ItemID。
    for i = 1, rowCount do
        local itemID = itemDropDefaultData:GetCell(i, "ItemID")

        -- 檢查是否為有效ID。
        if itemID ~= nil and itemID ~= "" then

            -- 將該列的所有資訊綁成一個table。
            -- 注意：用GetCell取得的數字可能是字串，因此若為數值建議使用tonumber()
            local rowData =
            {
                ItemID = itemID,
                DropRate = tonumber(itemDropDefaultData:GetCell(i, "DropRate")),
            }

            -- 儲存到快取table。(Key: ItemID, Value: 資料)
            self.itemDropData[itemID] = rowData
        end
    end
    -- 輸出表示資料表快取完成的Log。
    log("ItemDropData資料表快取完成! ")
    return
end
-- 由於尚未進行資料表快取，因此輸出提示該情況的日誌。
log("ItemDropData資料表已經快取過，或是找不到ItemDropData資料表。")
end

@ExecSpace("ClientOnly")
method table GetMissileDataByMissileID(string MissileID)
    -- 檢查missileData是否已正常快取。
    if invalid(self.missileData) or next(self.missileData) ~= nil then
        -- 若存在以MissileID為Key的Value，則回傳該值。
        if self.missileData[MissileID] then
            return self.missileData[MissileID]
        end
        -- 若missileData尚未快取，則輸出錯誤日誌。
    else
        log("目前沒有投射體資料解析! ")
        return nil
    end
end

@ExecSpace("ClientOnly")
method table GetCharacterDefaultDataByCharacterID(string CharacterID)
    -- 若characterDefaultData尚未正常快取，則嘗試進行快取。
    if self.characterDefaultData == nil then
        self:InitCharacterDefaultData()
    end
end

```

```

-- 檢查目前CharacterDefaultData是否已正常登錄。
if invalid(self.characterDefaultData) and next(self.characterDefaultData) ~= nil
then
    -- 若存在擁有CharacterID值的資料，則回傳該資料。
    if self.characterDefaultData[CharacterID] then
        return self.characterDefaultData[CharacterID]
    end
-- 若沒有對應Key的資料，則輸出錯誤日誌。
else
    log("沒有PlayerDefaultData資料，或是不存在"..CharacterID.."值! ")
    return nil
end
end

@ExecSpace("ClientOnly")
method table GetCharacterSelectUIDataTable()
-- 直接回傳整個角色選擇UI資料表。
if invalid(self.characterSelectUIData) and next(self.characterSelectUIData) ~= nil
then
    return self.characterSelectUIData
else
    -- 若沒有可回傳的值，則輸出錯誤日誌。
    log("CharacterSelectUIData資料表不存在，或是未正常完成快取! ")
    return nil
end
end

@ExecSpace("ClientOnly")
method table GetMonsterSpawnDataTableByStageID(number StageID)
-- 若傳入的StageID不是1以上的正常值，則留下日誌並結束載入。
if StageID <= 0 then
    log("為了在DataTableManager中呼叫SpawnData而傳入的值不是1以上! ")
    return
end
-- 若可正常傳遞怪物產生資料，則連同日誌一起回傳資料。
if self.monsterSpawnData[StageID] ~= nil then
    log("收到的產生資料值: "..self.monsterSpawnData[StageID][1].MonsterID)
    return self.monsterSpawnData[StageID]
end
-- 若無法回傳產生資料，則輸出錯誤日誌。
log(StageID.."這個StageID值的資料無法在MonsterSpawnData資料表中找到! ")
return nil
end

@ExecSpace("ClientOnly")
method table GetMonsterDefaultDataByMonsterID(string MonsterID)
-- 檢查monsterDefaultData是否已快取。
if invalid(self.monsterDefaultData) or next(self.monsterDefaultData) ~= nil then
    -- 若能回傳擁有MonsterID值的資料，則回傳該資料。
    if self.monsterDefaultData[MonsterID] then
        return self.monsterDefaultData[MonsterID]
    end
-- 若無法回傳資料，則輸出錯誤日誌。
else

```

```

        log("目前沒有怪物資料解析! ")
        return nil
    end
end

@ExecSpace("ClientOnly")
method table GetSkillDataBySkillID(string SkillID)
    -- 若skillData尚未快取, 則嘗試進行快取。
    if not isvalid(self.skillData) or not next(self.skillData) then
        self:InitSkillData()
    end
    -- 若可使用skillData, 且存在以SkillID為Key的值, 則回傳該值。
    if isvalid(self.skillData[SkillID]) and next(self.skillData[SkillID]) then
        return self.skillData[SkillID]
    end
    -- 若無法回傳資料, 則輸出錯誤日誌。
    log("找不到名稱為"..SkillID.."的SkillData! ")
    return nil
end

@ExecSpace("ClientOnly")
method table GetItemDataByItemID(string ItemID)
    -- 若itemData尚未快取, 則嘗試進行快取。
    if not isvalid(self.itemData) or not next(self.itemData) then
        self:InitItemData()
    end
    -- 若存在itemData中以ItemID為Key的Value, 則回傳該值。
    if isvalid(self.itemData[ItemID]) and next(self.itemData[ItemID]) then
        return self.itemData[ItemID]
    end
    -- 若無法回傳ItemID的值, 則輸出錯誤日誌。
    log("找不到名稱為"..ItemID.."的ItemData! ")
    return nil
end

@ExecSpace("ClientOnly")
method table GetItemDropData()
    -- 若itemDropData尚未快取, 則嘗試進行快取。
    if not isvalid(self.itemDropData) or not next(self.itemDropData) then
        self:InitItemDropData()
    end
    -- 若可以回傳itemDropData, 則全部回傳itemDropData。
    if isvalid(self.itemDropData) and next(self.itemDropData) then
        return self.itemDropData
    end
    -- 若無法回傳, 則連同錯誤日誌一起回傳nil。
    log("DataTableLogic中未登錄ItemDropData值! ")
    return nil
end

end

```

撰寫UIManageLogic程式碼

```

@Logic
script UIManageLogic extends Logic

property TitleUIController titleUIController = nil

property SelectCharacterUIController selectCharacterUIController = nil

property ResultUIController resultUIController = nil

property DefaultUIController defaultUIController = nil

@ExecSpace("ClientOnly")
method void OnBeginPlay()
-- 當遊戲開始時，尋找並登錄TitleUIController。
self:InitTitleUIController()
-- 當遊戲開始時，尋找並登錄SelectCharacterUIController。
self:InitSelectCharacterUIController()
-- 當遊戲開始時，尋找並登錄ResultUIController。
self:InitResultUIController()
-- 當遊戲開始時，尋找並登錄DefaultUIController。
self:InitDefaultUIController()
end

@ExecSpace("ClientOnly")
method void InitTitleUIController()
--若未登錄TitleUIController，則登錄該元件。
if not isvalid(self.titleUIController) then
    local titleUIEntity = _EntityService:GetEntityByPath("/ui/TitleUI")
    --若成功找到titleUIGroup，則登錄該實體的元件。
    if isvalid(titleUIEntity) then
        local component = titleUIEntity.TitleUIController
        -- 搜尋是否已正常登錄元件後，將其指派。
        if isvalid(component) then
            self.titleUIController = component
        end
    end
end
end

@ExecSpace("ClientOnly")
method void InitSelectCharacterUIController()
--若未登錄SelectCharacterUIController，則登錄該元件。
if not isvalid(self.selectCharacterUIController) then
    local selectCharacterUIEntity =
_EntityService:GetEntityByPath("/ui/SelectCharacterUI")
    --若成功找到SelectCharacterUIController，則登錄該實體的元件。
    if isvalid(selectCharacterUIEntity) then
        local component = selectCharacterUIEntity.SelectCharacterUIController
        -- 搜尋是否已正常登錄元件後，將其指派。
        if isvalid(component) then

```

```

        self.selectCharacterUIController = component
    end
end
end

@ExecSpace("ClientOnly")
method void ShowTitle()
-- 顯示標題畫面。
self.titleUIController:Show()
-- 其餘UI全部設為隱藏。
self.selectCharacterUIController:Hide()
self.resultUIController:Hide()
self.defaultUIController:Hide()
end

@ExecSpace("ClientOnly")
method void CallSelectCharacterUI()
-- 顯示角色選擇視窗。
self.selectCharacterUIController:Show()
-- 嘗試初始化角色選擇視窗內的角色清單。
self.selectCharacterUIController:InitCharacterSelectFrame()
end

@ExecSpace("ClientOnly")
method void ShowInGame()
-- 初始化作為遊戲畫面UI的defaultUI。
self.defaultUIController:Show()
-- 請求初始化自動攻擊UI。
self.defaultUIController:UpdateAutoUI()
-- 將其餘UI設為隱藏。
self.titleUIController:Hide()
self.selectCharacterUIController:Hide()
self.resultUIController:Hide()
end

@ExecSpace("ClientOnly")
method void InitResultUIController()
--若尚未登錄ResultUIController, 則登錄該元件。
if not isvalid(self.resultUIController) then
    local resultUIEntity = _EntityService:GetEntityByPath("/ui/ResultUI")
    --若成功找到ResultUIController, 則登錄該實體的元件。
    if isvalid(resultUIEntity) then
        local component = resultUIEntity.ResultUIController
        -- 搜尋是否已正常登錄元件後, 將其指派。
        if isvalid(component) then
            self.resultUIController = component
        end
    end
end
end
end

@ExecSpace("ClientOnly")
method void ShowResultUI(boolean IsClear)

```

```

-- 顯示結果UI。
self.resultUIController:Show(IsClear)
if IsClear then
    --播放祝賀效果音
    _SoundService:PlaySound("5d57b617cfc242cca347925e209c8e05", 1.0)
else
    --播放失敗效果音
    _SoundService:PlaySound("52dccfb8889b4181b9181780dc7e9040", 1.0)
end
end

@ExecSpace("ClientOnly")
method void InitDefaultUIController()
--若尚未登錄defaultUIController, 則登錄該元件。
if not isvalid(self.defaultUIController) then
    local titleUIEntity = _EntityService:GetEntityByPath("/ui/DefaultUI")
    --若成功找到DefaultUIGroup, 則登錄該實體的元件。
    if isvalid(titleUIEntity) then
        local component = titleUIEntity.DefaultUIController
        -- 搜尋是否已正常登錄元件後, 將其指派。
        if isvalid(component) then
            self.defaultUIController = component
        end
    end
end
end

@ExecSpace("ClientOnly")
method void UpdateLifeCountUI(number PlayerLife)
-- 依照PlayerLife數量更新DefaultUI的體力UI。
self.defaultUIController:UpdateLifeUI(PlayerLife)
end

@ExecSpace("ClientOnly")
method void UpdateSkillCountUI(number LeftSkillCount)
-- 依照LeftSkillCount的數量更新DefaultUI的技能UI。
self.defaultUIController:UpdateSkillUI(LeftSkillCount)
end

end

```

Logic的優點

- 具有可在任何地方方便存取的優點。
- 不需要像元件那樣去尋找特定實體, 也不需要確認找到的實體是否持有該元件。
- 只要先建立好Logic, 就可以明確地信任並使用該Logic。
- 此外, 由於功能由Logic負責, 因此當出現責任歸屬問題時, 只要修改具有該責任的Logic即可解決問題。

使用Logic時的注意事項

- 有可能發生一個Logic負責所有功能的God Class化。
 - EX: 負責傷害計算的Logic同時負責傷害計算、傷害值處理、聲音輸出、Hit特效輸出等
- 一旦發生God Class化, 該Logic除了自己需要的資訊之外, 還必須知道更多資訊。
- 這種方式偏離物件導向設計理論, 也會成為大幅增加程式碼行數的原因。
- 增加的程式碼行數會降低可讀性, 並且在進行除錯時, 會產生無法快速找出問題原因的缺點。

最低實作標準

- 製作用來管理並使用之後要使用的DataSet的Logic。
 - 以範例世界為基準, 製作DataTableLogic來管理DataSet。
- 製作可負責依遊戲進行而處理內容的Logic。
 - 以範例世界為基準, 製作GameLogic來實作依遊戲狀態而處理的內容。
- 製作可管理所有UI的Logic。
 - 以範例世界為基準, 製作UIManageLogic來實作可管理多個UI的功能。

應用與擴展方向

- 製作CalcDamageLogic, 在接收到傷害值後, 依攻擊計算公式計算最終傷害, 以管理傷害計算功能。
- 利用Logic製作並使用ConfigLogic, 以管理預定不會變更的固定值。
 - EX: 若是生命不會超過5個的遊戲, 可以製作並使用可登錄MaxHP以固定最大體力的Logic。

[💡 基礎學習]製作PlayerCharacter

影片時間軸

- 可在課程影片中以下時間點進行學習。
- 製作PlayerCharacter: 00:49:40

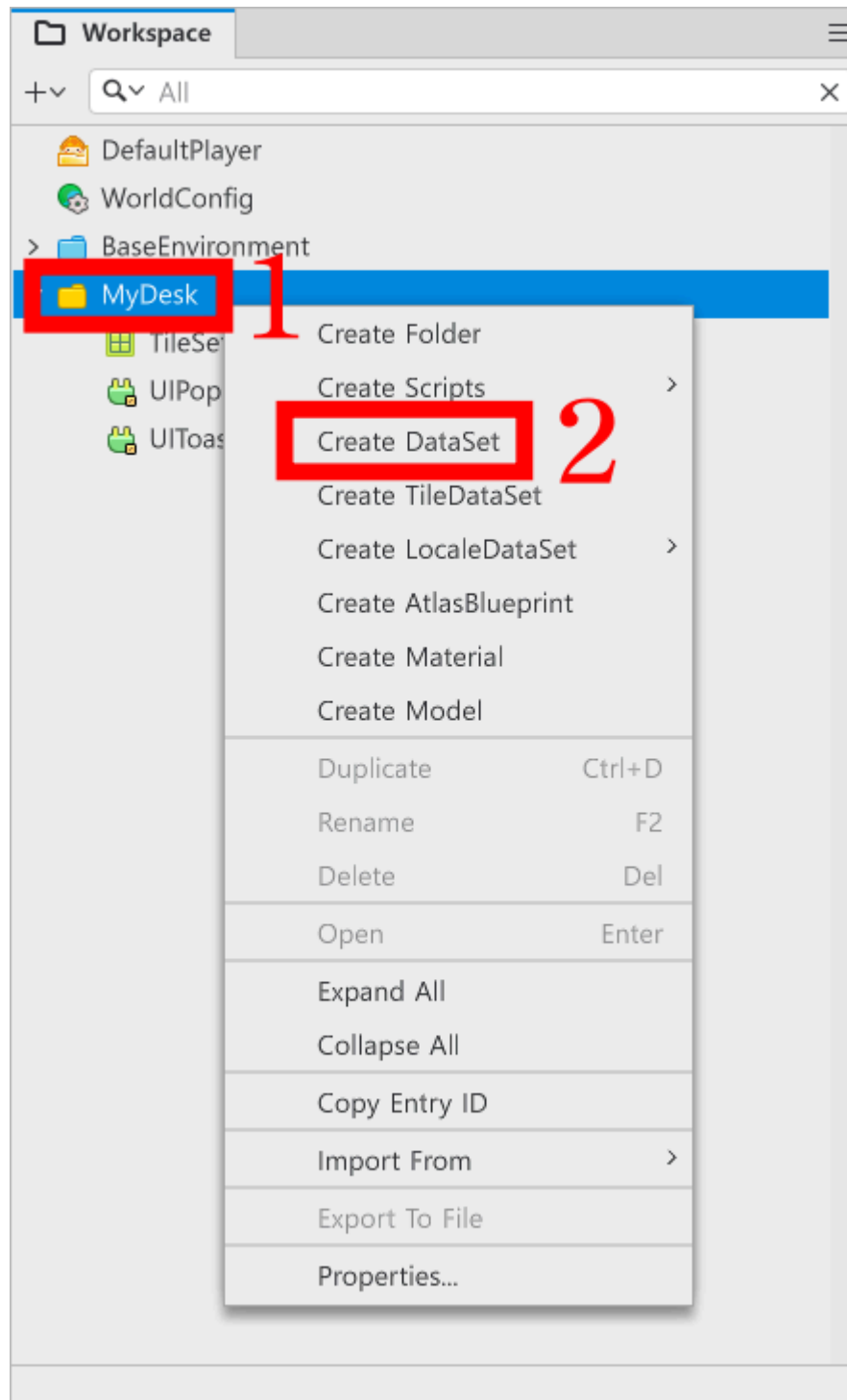
學習目標

- 學習構成玩家角色的DataSet, 並可修改DataSet。
- 可利用DataTableLogic直接實作Manager並加以使用。
- 可直接製作並使用Component。
- 可利用DataSet初始化Component中的值。

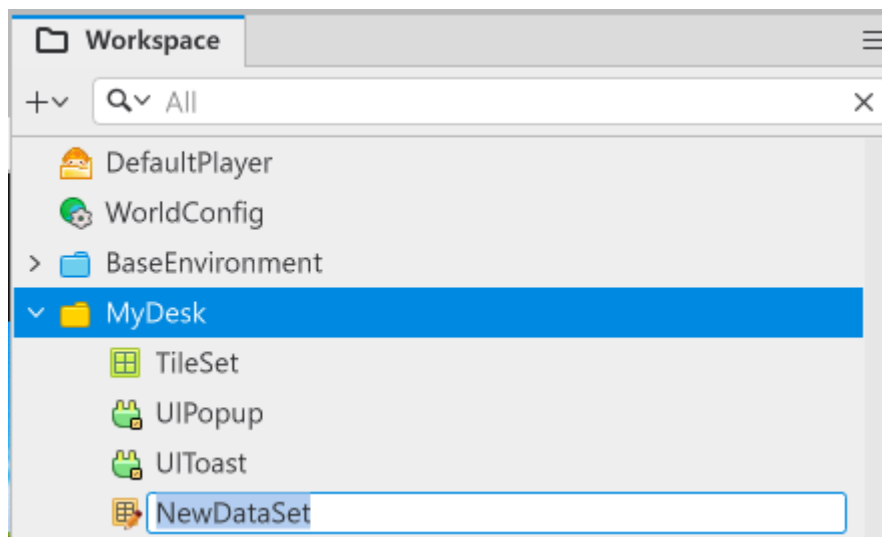
觀看該影片後可嘗試執行的內容

- 實作DataTableLogic, 製作用來管理DataSet的Logic。
- 直接製作使用者自訂元件的PlayerState元件, 並可將其新增到玩家角色上。
- 可利用TriggerComponent為玩家設定碰撞範圍。

製作DataSet



- 在Workspace中對MyDesk按右鍵。
- 按下Create DataSet來製作新的DataSet。



- 變更已建立的DataSet名稱。
- 在範例世界中使用的是CharacterDefaultData。
- 完成DataSet建立後，雙擊所建立的DataSet以移動到編輯視窗。

CharacterDefaultData							
ID	CharacterID	StartHP	StartSkillCount	StartMoveSpeed	DefaultMissile	DefaultSkill	AttackInterval
1	Pirate	3	2	5.0	Missile_Bullet	Skill_CannonBal	0.5

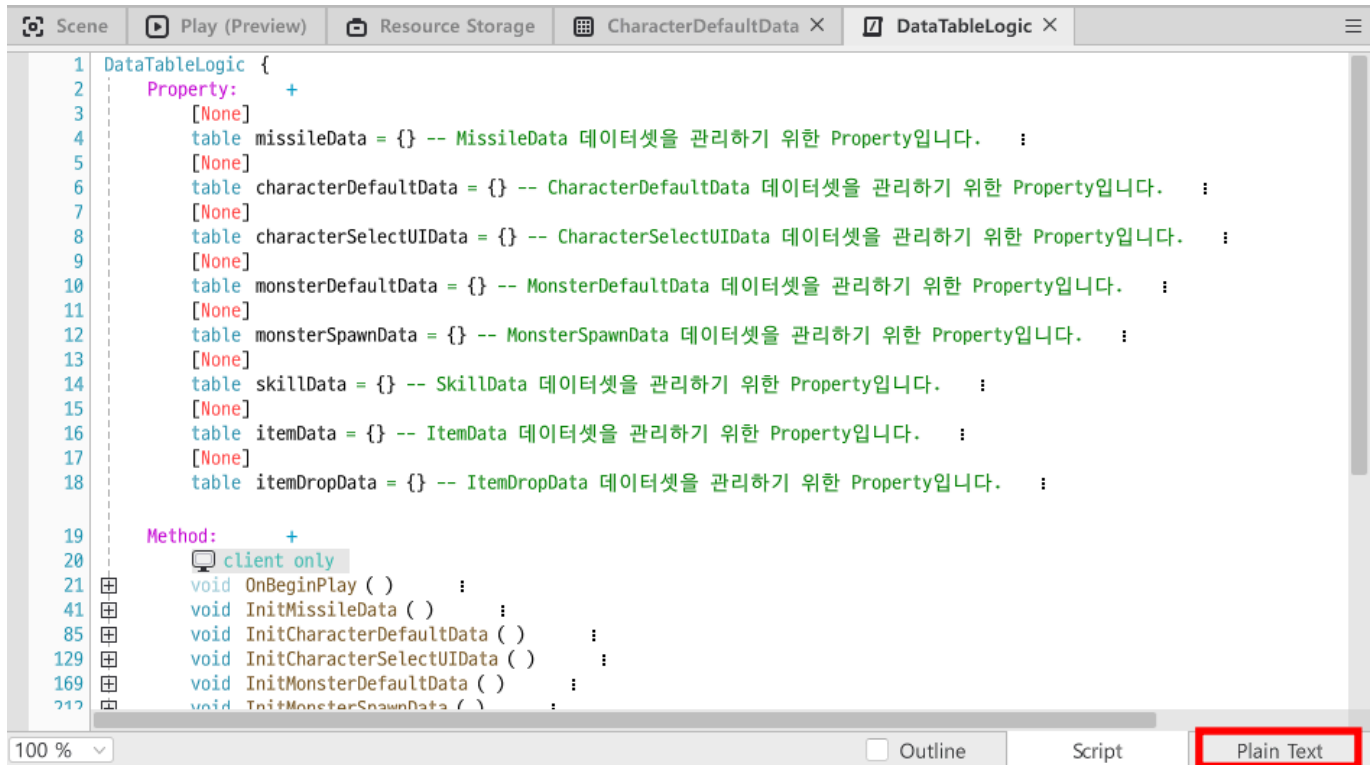
- 若需要新增各列、各欄，可按下+按鈕新增對應的列、欄。
- 在第1列輸入要使用的資料名稱。
- 從第2列開始輸入符合資料名稱的值。

範例世界的DataSet構成

姓名	說明
CharacterID	這是用來區分角色的ID值。可依據該值載入構成角色的資訊。
StartHP	這是遊戲開始時用來設定角色初始體力的值。
StartSkillCount	這是遊戲開始時用來設定角色可使用必殺技數量的值。
StartMoveSpeed	這是遊戲開始時用來設定角色移動速度的值。
DefaultMissile	這是用來決定該角色基本投射體的值。
DefaultSkill	這是用來指定角色必殺技的值。
AttackInterval	這是用來決定角色攻擊延遲的值。值越小，連射速度越快。

在DataTableLogic中登錄DataSet

- 在DataTableLogic中如下撰寫程式碼。
- 程式碼範例是以PlainText為基準撰寫。



- PlainText可透過程式碼編輯器視窗右下角的UI按鈕進行切換。

property table characterDefaultData = {} -- 用來管理CharacterDefaultData DataSet의 Property.

```

@ExecSpace("ClientOnly")
method void OnBeginPlay()
-- 初始化CharacterDefaultData DataSet.
self:InitCharacterDefaultData()
end

method void InitCharacterDefaultData()
-- 用於檢查玩家角色解析是否完成的項目
if isValid(self.characterDefaultData) == false or next(self.characterDefaultData)
== nil then
    local getTableData = _DataService:GetTable("CharacterDefaultData")
    --若取得資料表失敗則輸出警告訊息
    if getTableData == nil then
        log("無法取得CharacterDefaultData資料表! ")
        return
    end
    --開始解析
    local rowCount = getTableData:GetRowCount() -- 計算全部列數。
  
```

```

    for i = 1, rowCount do
        -- 先從資料表Row中代入一個作為Key的CharacterID。
        local characterID = tostring(getTableData:GetCell(i, "CharacterID"))

        -- 檢查是否為有效ID。
        if characterID ~= nil and characterID ~= "" then

            -- 將該列的所有資訊綁成table。
            local rowData =
            {
                CharacterID = characterID,
                StartHP = tonumber(getTableData:GetCell(i, "StartHP")),
                StartSkillCount = tonumber(getTableData:GetCell(i,
"StartSkillCount")),
                StartMoveSpeed = tonumber(getTableData:GetCell(i,
"StartMoveSpeed")),
                DefaultMissile = tostring(getTableData:GetCell(i,
"DefaultMissile")),
                DefaultSkill = tostring(getTableData:GetCell(i, "DefaultSkill")),
                AttackInterval = tonumber(getTableData:GetCell(i,
"AttackInterval"))
            }

            -- 儲存到快取table中(Key: MissileID, Value: 資料)
            self.characterDefaultData[characterID] = rowData
        end
    end

    -- 輸出表示資料表快取完成的Log。
    log("CharacterDefaultData資料表快取完成! ")
    return
end

-- 由於尚未進行資料表快取，因此輸出提示該情況的日誌。
log("CharacterDefaultData資料表已經快取，或找不到CharacterDefaultData資料表。")
end

method table GetCharacterDefaultDataByCharacterID(string CharacterID)
-- 若characterDefaultData尚未正常快取，則嘗試進行快取。
if self.characterDefaultData == nil then
    self:InitCharacterDefaultData()
end

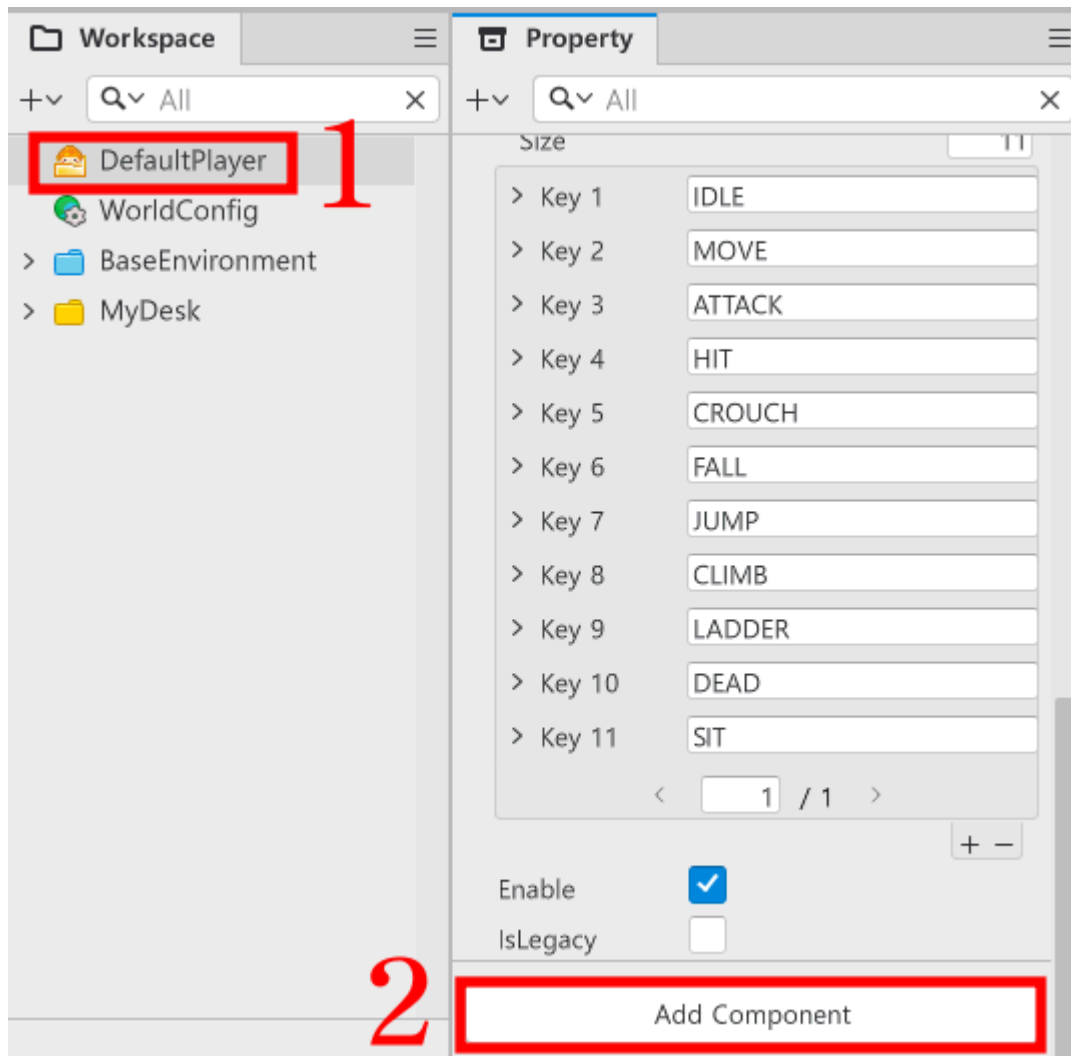
-- 檢查目前CharacterDefaultData是否已正常登錄。
if invalid(self.characterDefaultData) and next(self.characterDefaultData) ~= nil
then
    -- 當有具有CharacterID值的資料時，回傳該資料。
    if self.characterDefaultData[CharacterID] then
        return self.characterDefaultData[CharacterID]
    end
end

-- 若沒有對應Key的資料，則輸出錯誤日誌。
else
    log("沒有PlayerDefaultData資料，或是不存在"..CharacterID.."值! ")
    return nil
end
end
end

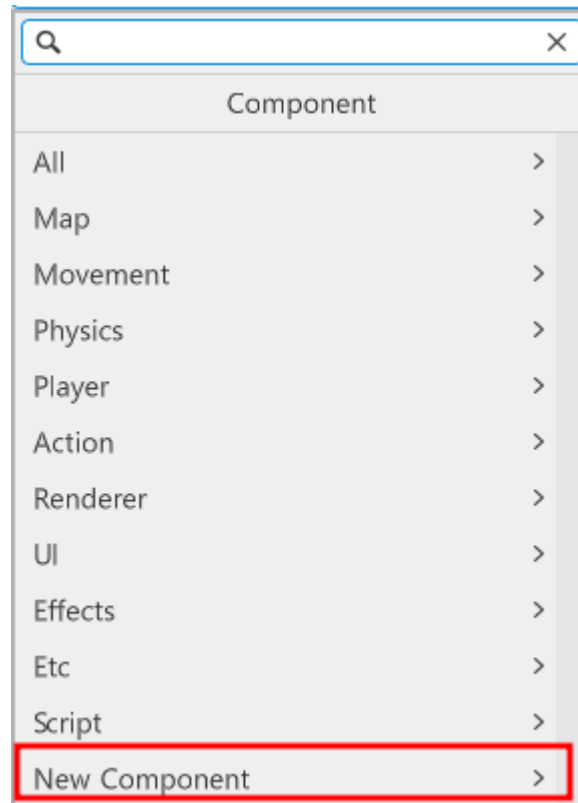
```

製作PlayerState Component

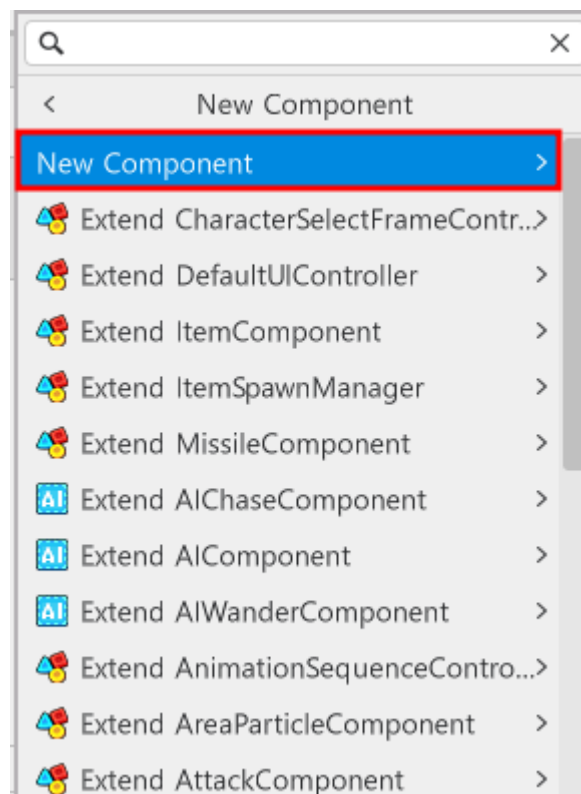
- 若已將玩家資訊儲存在DataSet中，就需要使用已儲存的資訊。



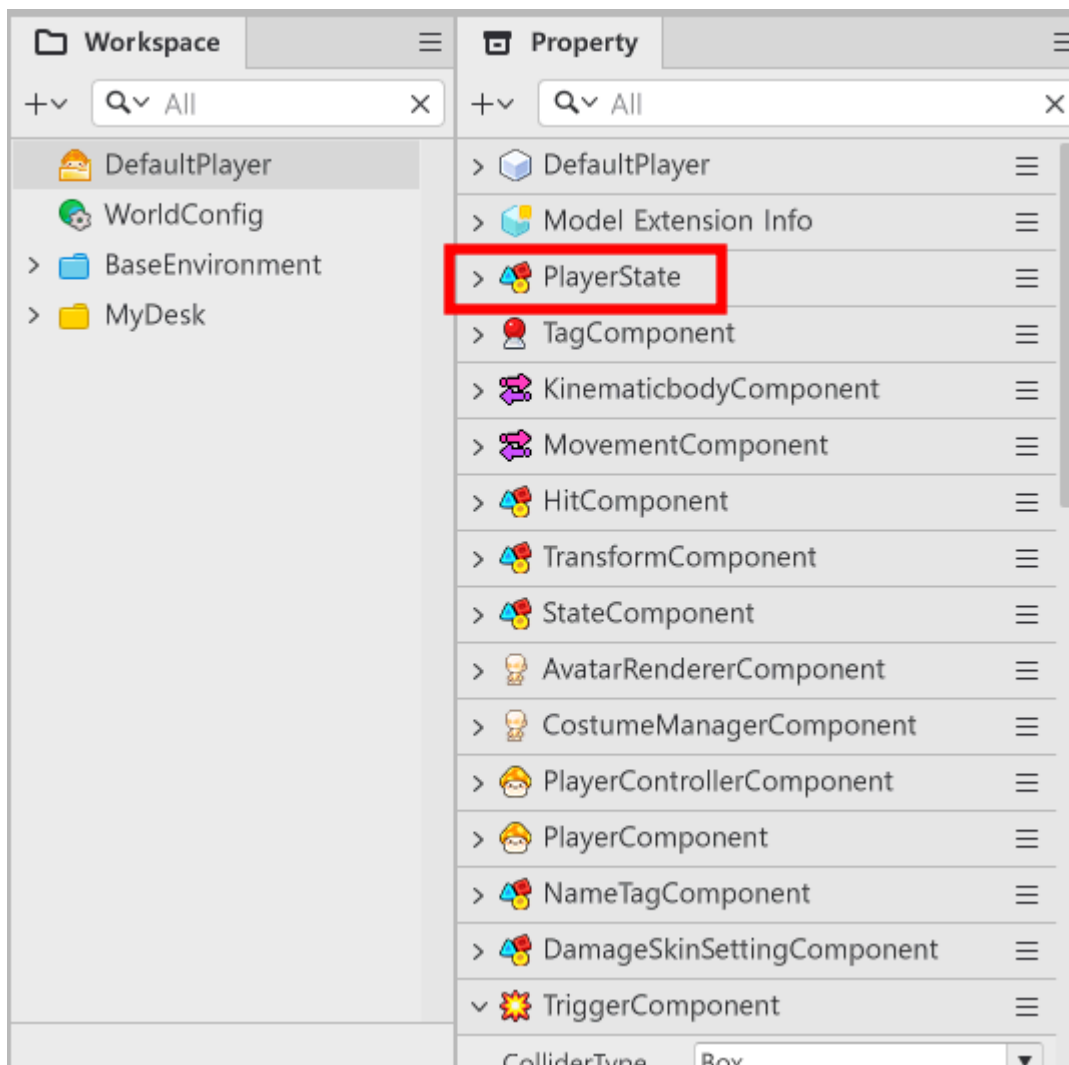
- 在Workspace中點擊DefaultPlayer。
- 將Property視窗拉到底後，點擊Add Component。



- 在新增的選單中點擊NewComponent。



- 點擊New Component後，輸入要建立的Component名稱，再點擊Create and Add。
- 在範例世界中，將該Component名稱設定為PlayerState。



- 若已正常登錄，點擊DefaultPlayer時，可在Property視窗中確認已登錄所輸入的Component。

初始化PlayerState的值

- 在PlayerState中如下輸入程式碼。
- 該程式碼是以Plain Text為基準撰寫。

```
@Component
script PlayerState extends Component

property number currentHP = 0 -- 用來管理玩家目前體力的Property。

property string currentMissile = "" -- 用來儲存玩家目前投射體ID的Property。

property string characterID = "" -- 用來儲存目前遊玩角色ID的Property。

property number attackInterval = 0.5 -- 用來管理玩家攻擊延遲的Property。

property number attackTimer = 0 -- 用來計算玩家是否可攻擊的Property。

property any projectilePoolManager = nil
```

property Vector3 firePos = Vector3(0.25,0.25,0) -- 用來調整玩家攻擊發射位置的Property。

property boolean isInvincibility = false -- 用來區分玩家是否處於無敵狀態的Property。

property number invincibilityDelta = 0 -- 用於計算無敵時間的Property。

property number invincibilityTime = 1.0 -- 無敵持續時間的基準Property。可透過修改該值來變更無敵時間。

property boolean isAlive = false -- 用來區分玩家是否存活的Property。

property string skillID = "" -- 用來儲存玩家使用技能ID的Property。

property number canUseSkillCount = 0 -- 用來管理目前玩家可使用技能次數的Property。

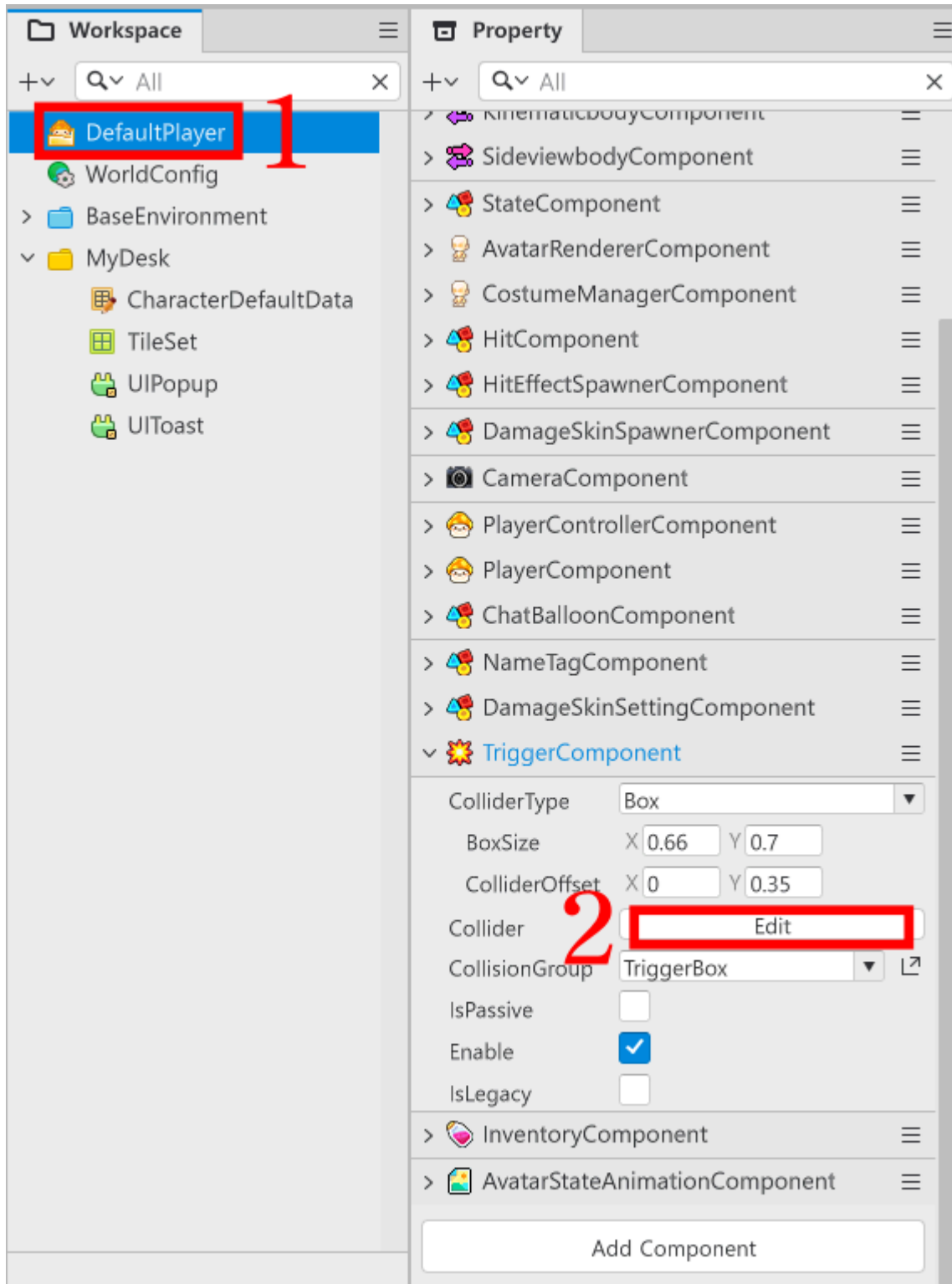
property number moveSpeed = 0 -- 用來登錄玩家移動速度的Property。

```
@ExecSpace("ClientOnly")
method void InitPlayerData(table PlayerDefaultData)
-- 若傳入的PlayerData不是正常資料，則輸出錯誤日誌並中止程式執行。
if PlayerDefaultData == nil or next(PlayerDefaultData) == nil then
    log("PlayerDefaultData不正常! ")
    return
end
-- 初始化玩家目前體力。
self.currentHP = PlayerDefaultData.StartHP
-- 初始化玩家可使用的技能ID。
self.canUseSkillCount = PlayerDefaultData.StartSkillCount
-- 初始化玩家移動速度。
self.moveSpeed = PlayerDefaultData.StartMoveSpeed
-- 為了正常套用初始化後的玩家移動速度，將其傳遞給MovementComponent。
self.Entity.MovementComponent.InputSpeed = self.moveSpeed
-- 初始化玩家基本投射體ID。
self.currentMissile = PlayerDefaultData.DefaultMissile
-- 初始化玩家角色ID值。
self.characterID = PlayerDefaultData.CharacterID
-- 初始化玩家攻擊延遲。
self.attackInterval = PlayerDefaultData.AttackInterval
-- 初始化玩家技能ID。
self.skillID = PlayerDefaultData.DefaultSkill
-- 初始化玩家起始位置。
self.Entity.TransformComponent.Position = Vector3.zero
-- 將玩家Entity的使用狀態改為true。
self.Entity.Enable = true
-- 將玩家角色的Visible狀態改為true。
self.Entity.Visible = true
end
```

```
@ExecSpace("ClientOnly")
method void OnBeginPlay()
    local getData = _DataTableLogic:GetCharacterDefaultDataByCharacterID("Pirate")
    self:InitPlayerData(getData)
```

```
self.isAlive = true  
end
```

修改TriggerComponent



- 在Workspace中點擊DefaultPlayer。
- 在Property視窗中找到TriggerComponent後，點擊Edit。



- 拖曳綠色區域並放下，將大小調整為符合角色圖片。

什麼是TriggerComponent?

- TriggerComponent是用來偵測實體之間碰撞的Component。
- 綠色區域是碰撞判定區域。
- 當該區域互相交錯的瞬間，就會被判定為發生碰撞。
- 因此若判定比圖片資源更大或更小，玩家可能會覺得是不合理的判定。
- 但依照遊戲方式，有時也會故意放寬判定，誘導較寬鬆的處理。
- 必須依照想要製作的遊戲性質，思考並設計要完全貼合判定，還是保留寬裕空間。

最低實作標準

- 可以製作DataSet。
- 可透過製作完成的DataTableLogic載入DataSet中想要的值。
- 可將載入的DataSet值傳遞到Component並進行指定。
- 可透過編輯TriggerComponent來設定碰撞範圍。

應用與擴展方向

- 可以將之後要製作的與玩家相關功能分離為Component單位。
 - EX: 若由攻擊負責，則可製作PlayerAttackComponent，並將與攻擊相關的功能實作在該Component中。
- 將遊戲製作所需的值新增至DataSet，並修改DataTableLogic以載入新增的值。
 - 透過學習並使用該方法，可依需求擴充或新增DataSet。

[💡 基礎學習]製作Monster

影片時間軸

- 可在課程影片中以下時間點進行學習。
- 製作Monster: 01:21:22

學習目標

- 可建立作為玩家威脅要素的敵人，為遊戲營造緊張感。

- 可建立實體並新增Component來構成敵人。
- 可利用楓之谷世界提供的Component，實作讓敵人追蹤玩家的功能。

觀看該影片後可嘗試執行的內容

- 建立空實體後，新增所需Component來製作Model。
- 可依需要製作不同外型的Monster。
- 將已製作Monster的數值交由DataSet管理，以製作出具有不同能力值的Monster。

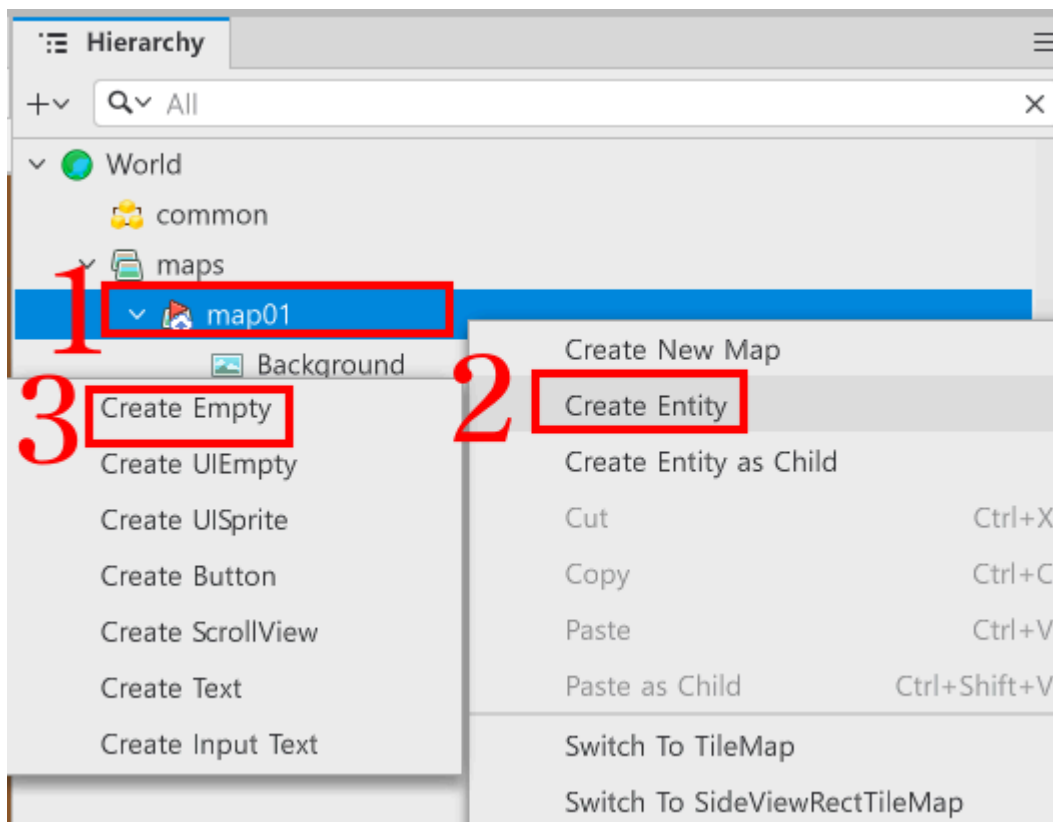
構成MonsterDataSet

- 建立用來儲存與Monster相關資訊的DataSet。
- 在範例世界中，將該DataSet名稱製作為MonsterDefaultData。
- 若不知道如何製作DataSet，請參考本文件的製作PlayerCharacter來進行製作。

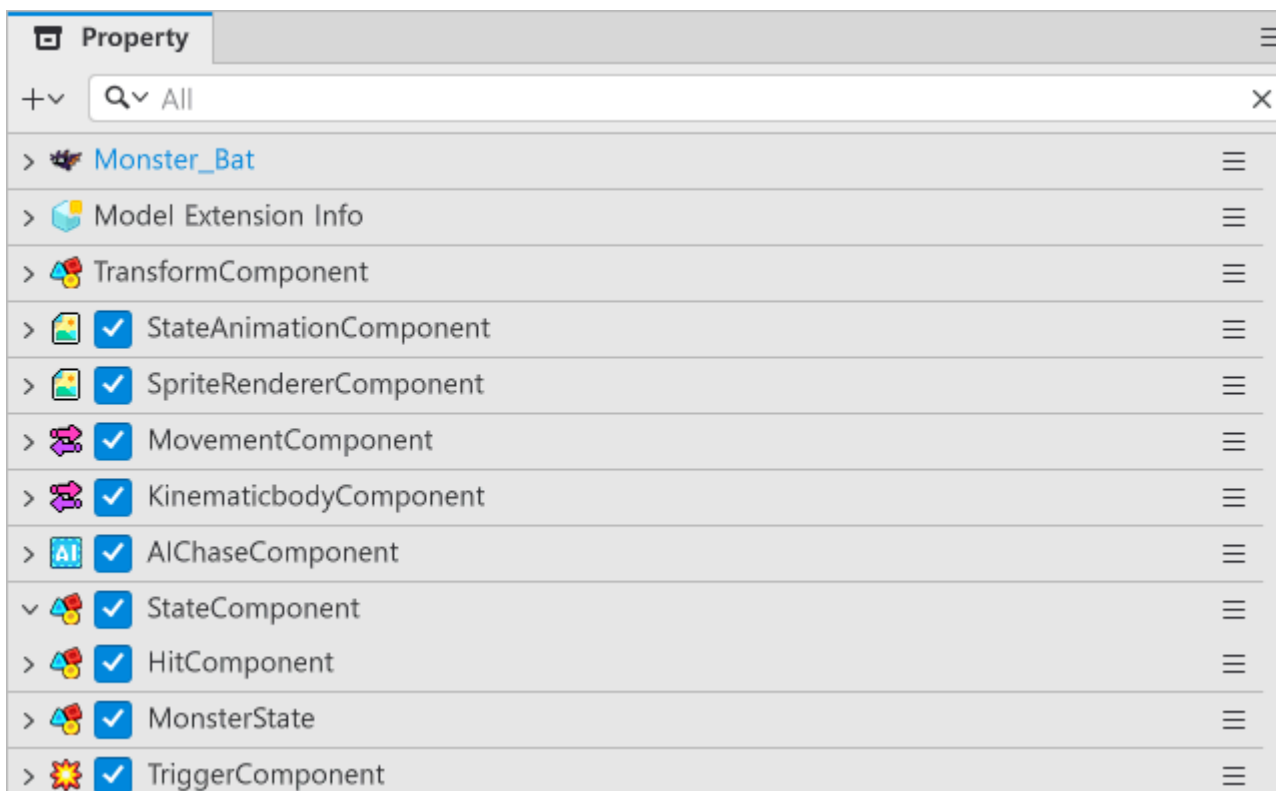
MonsterDefaultData構成要素

姓名	說明
MonsterID	這是用來區分Monster的ID值。可依據該值區分怪物並載入資料。
StartHP	這是Monster在遊戲中建立時，用來設定初始HP的值。
StartMoveSpeed	這是Monster建立時，用來指定基本移動速度的值。
DefaultMissile	這是用來指定Monster基本投射體的值。
AttackInterval	這是用來指定Monster攻擊延遲的值。該值越小，連射速度越快。
ModelRUID	這是用來指定已製作Monster Model的RUID的值。之後MonsterSpawnManager會依據該值複製Model。

構成怪物實體



- 在Hierarchy中對map01按右鍵。
- 點擊Create Entity。
- 點擊Create Empty，將空實體建立為map01的子實體。
- 可以變更已建立實體的名稱。
- 在範例世界中，為了示範，名稱製作為Monster_Bat。

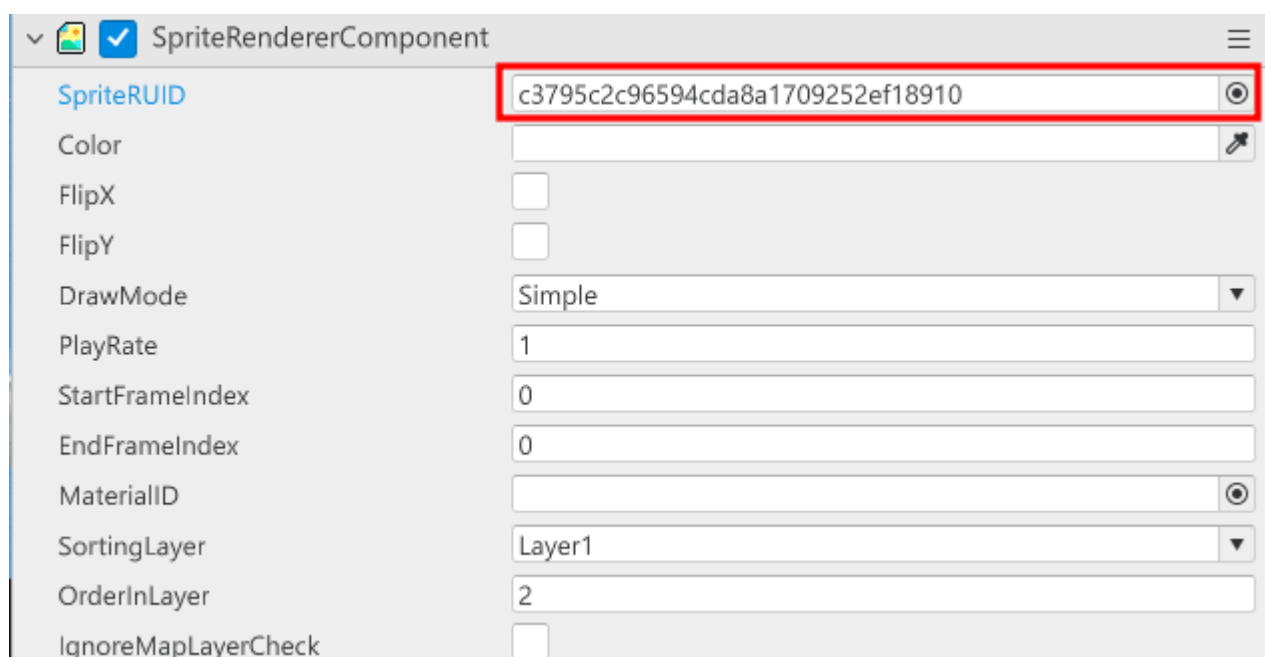


- 將以下Component新增到已製作的實體上。
- **SpriteRendererComponent**：用來輸出Monster圖片的Component。

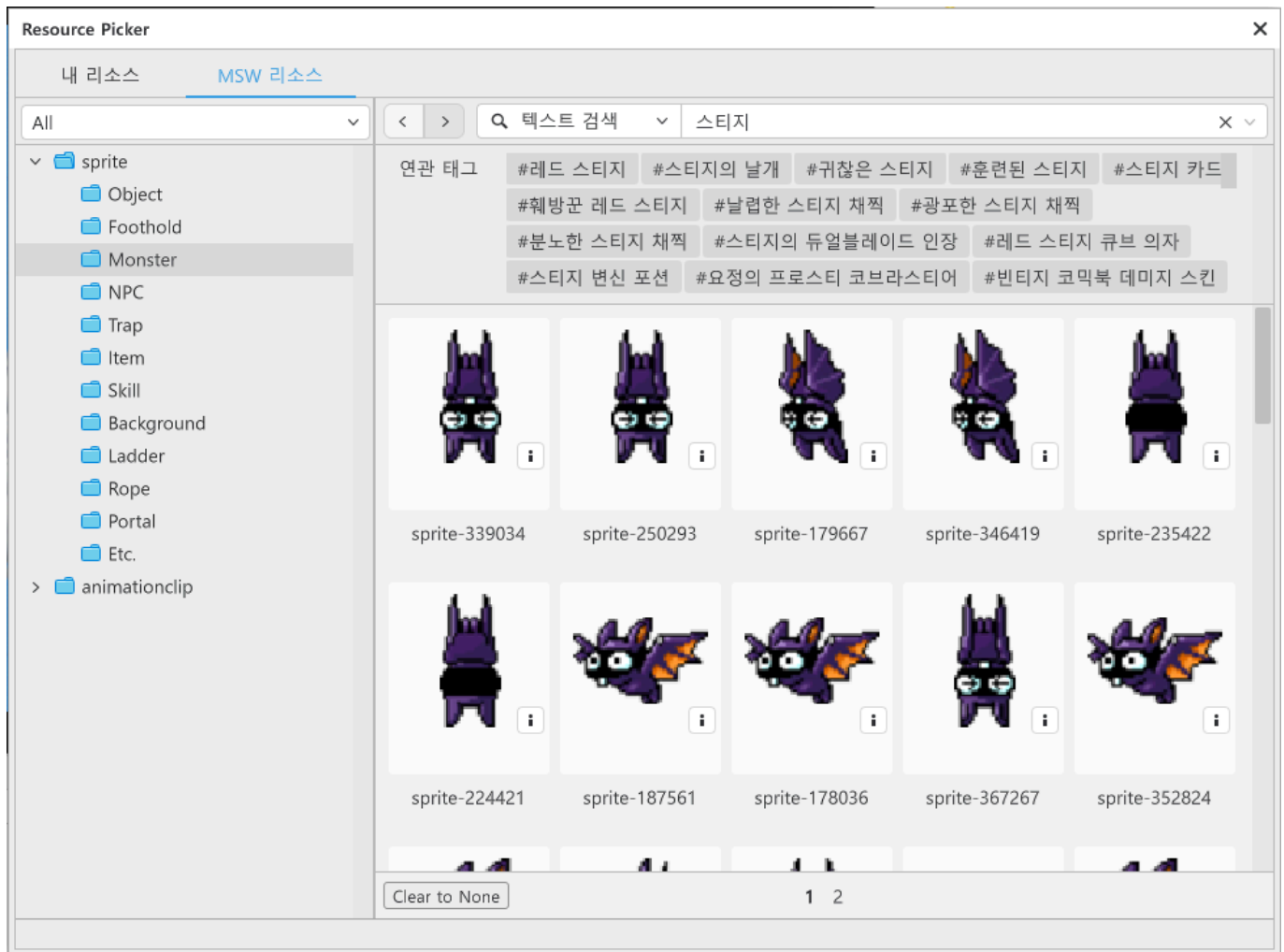
- **MovementComponent**: Monster移動所需的Component。AIChaseComponent會利用該Component追蹤玩家。
- **KinematicBodyComponent**: 讓怪物可在RectTileMap中移動。Monster需將EnableTileCollision值改為false，以便沒有不可移動區域。
- **AIChaseComponent**: 這是楓之谷世界提供的Component。具有該Component的實體可以追蹤玩家。
- **HitComponent**: 被投射體擊中時，用來計算傷害的Component。
- **MonsterState**: 這是需要直接製作的Component。該Component和PlayerState一樣，是用來承載Monster資料，或負責攻擊、死亡處理等功能的Component。
- **TriggerComponent**: 用來指定Monster碰撞區域的Component。可透過該Component指定怪物的碰撞區域，並檢查是否與投射體或技能發生碰撞。

變更怪物圖片

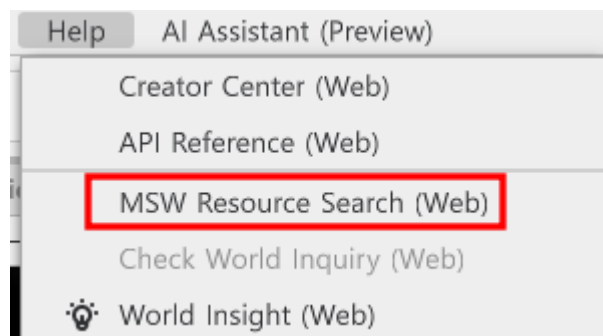
- 若要變更怪物圖片，可透過變更SpriteRendererComponent的SpriteRUID來套用。



- 可透過變更SpriteRUID的值來變更怪物圖片。
- 可利用右側的Resource Picker尋找並使用其他圖片。



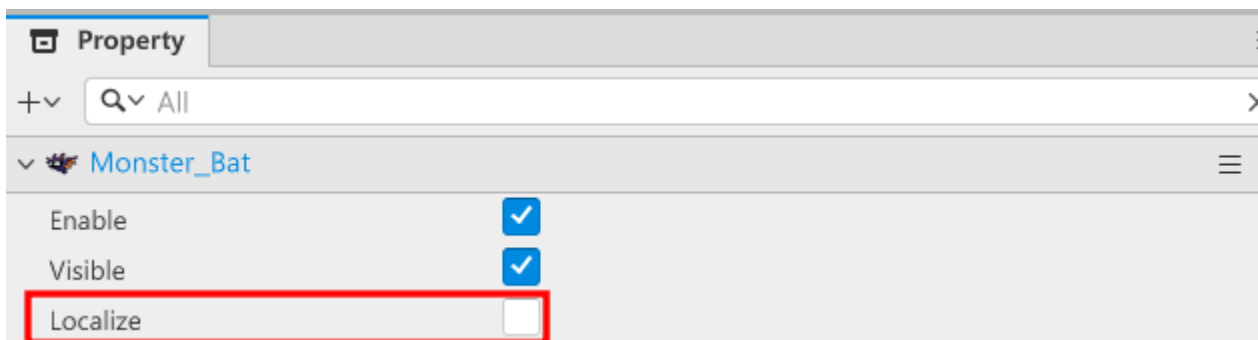
- 可在Resource Picker中搜尋資源，以使用楓之谷世界提供的資源。



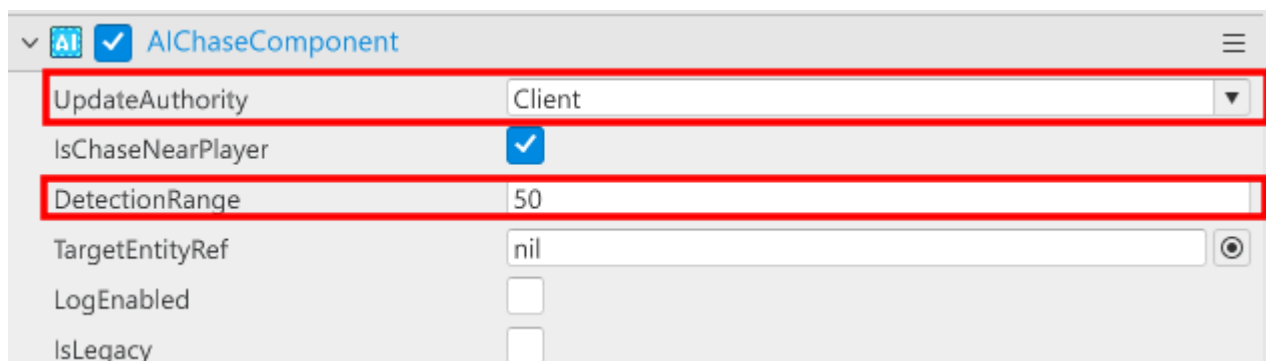
- 若覺得使用Resource Picker困難，也可以透過編輯器上方的Help來尋找資源。

設定讓怪物可以追蹤玩家

- 為了讓怪物追蹤玩家，必須套用以下要素。



- 必須在Property中啟用Localize。 - 範例世界是以客戶端驅動為基礎製作的。 - 因此必須勾選Localize，讓Entity可在本機端運作。



- 必須將AIChaseComponent內的UpdateAuthority變更為Client。
 - 若該值為Server，則在以客戶端驅動為基礎的範例世界中，可能無法正常追蹤玩家。
- 可依需要增加DetectionRange的值，以擴大敵人辨識玩家的範圍。

構成MonsterState Component

- 必須撰寫加在Model中的MonsterState Component程式碼。
- 透過該程式碼，Monster可找到MonsterDefaultData後取得數值。
- 如下撰寫程式碼。
- 該程式碼是以Plain Text為基準撰寫。

```
@Component
script MonsterState extends Component

property number currentHP = 0 -- 用來管理怪物目前體力的Property。

property number attackInterval = 0 -- 用來儲存怪物攻擊延遲的Property。

property boolean isAlive = true -- 用來檢查怪物是否存活的Property。

property string defaultMissile = "" -- 用來儲存怪物發射之投射體ID的Property。

property ProjectileManager projectileManager = nil

property Vector2 firePos = Vector2(0,0) -- 用來指定投射體發射位置的Property。

property number deltaTimer = 0 -- 用來計算怪物投射體發射延遲的Property。
```

```

property number itemDropRate = 0 --用來計算擊殺怪物時道具掉落機率的Property。

property string mustDropItemValue = "" -- 若怪物必定要掉落道具時，用來指定道具的Property。

@ExecSpace("Client")
method void InitData(table MonsterData, number ItemDropRate, string MustDropItem)
-- 檢查傳入的MonsterData是否為nil。
if MonsterData == nil or next(MonsterData) == nil then
    log("傳入的table MonsterData是nil! ")
    return
end

-- 檢查傳入的MonsterData中的必要要素是否具有正常的值。
if MonsterData.StartHP == nil or MonsterData.DefaultMissile == nil or
MonsterData.DefaultMissile == "" then
    log("怪物的初始體力、投射體類型尚未設定! ")
end
self:FindProjectileManager()

-- 若有填入必要值則開始初始化
-- 在怪物開始時，將目前體力初始化為最大體力。
self.currentHP = MonsterData.StartHP
-- 初始化怪物的初始投射體ID。
self.defaultMissile = MonsterData.DefaultMissile
-- 設定怪物的攻擊延遲。若接收到的值為nil，則初始化為1.0秒。
self.attackInterval = MonsterData.AttackInterval or 1.0
-- 將怪物的存活狀態初始化為true。
self.isAlive = true
-- 初始化道具掉落機率。
self.itemDropRate = ItemDropRate
-- 當存在必定要掉落的道具時，初始化該值。
self.mustDropItemValue = MustDropItem
-- 初始化怪物的移動速度。
self.Entity.MovementComponent.InputSpeed = MonsterData.StartMoveSpeed
end

method void ShowMonster()
-- 將怪物的Enable狀態變更為true。
self.Entity.Enable = true
end

@ExecSpace("ClientOnly")
method void OnUpdate(number delta)
-- 若遊戲狀態不是Play狀態，則立即結束程式碼。
if not _GameLogic.IsStillPlay then
    self.Entity.MovementComponent.Enable = false
    return
end
-- 若目前怪物存活，且遊戲狀態為Play狀態，則進行處理。
if self.isAlive and _GameLogic.IsStillPlay then
    -- 若怪物的MovementComponent為停用狀態，則將其啟用。
    if self.Entity.MovementComponent.Enable == false then

```

```

        self.Entity.MovementComponent.Enable = true
    end
    -- 將delta時間流逝量加到deltaTimer中。
    self.deltaTimer = self.deltaTimer + delta
    -- 若deltaTimer大於攻擊延遲，則發射投射體，並將deltaTimer初始化為0。
    if self.deltaTimer >= self.attackInterval then
        self:FireMissile()
        self.deltaTimer = 0
    end
end
end

@ExecSpace("Client")
method void FindProjectileManager()
    -- 尋找位於InGame地圖中的MissileManager實體。
    local currentMap = self.Entity.CurrentMapName
    local poolEntity =
_EntityService:GetEntityByPath("/maps/"..currentMap.."/ProjectileManager")
    -- 若正常找到poolEntity，則以poolEntity初始化missileManager。
    if isvalid(poolEntity) then
        self.projectileManager = poolEntity.ProjectileManager
        self.isAlive = true
    else
        -- 若在InGame中找不到ProjectileManager，則輸出錯誤日誌。
        log("錯誤：在InGame地圖中找不到ProjectileManager。")
    end
end

@ExecSpace("Client")
method void FireMissile()
    -- 若沒有projectileManager，則中止程式執行。
    if self.projectileManager == nil then return end

    -- 檢查projectileManager是否為nil，以及是否已登錄基本投射體。
    if self.projectileManager == nil or self.defaultMissile == nil or
self.defaultMissile == "" then
        -- 若其中任一條件未滿足，則留下錯誤日誌。
        log("玩家MissileData初始化失敗！")
    end

    -- 從ProjectileManager取得可使用的defaultMissile實體。
    local missileEntity = self.projectileManager:GetMissile(self.defaultMissile)

    -- 當projectileManager傳來的實體可使用時，執行程式碼。
    if isvalid(missileEntity) then
        -- 取得傳來實體的MissileComponent。
        local missileComp = missileEntity.MissileComponent
        if missileComp then
            -- 決定投射體的發射方向。
            local fireDir = 1
            -- 由於基本資源是朝左，因此若實體的FlipX為停用狀態，則設定為朝左飛行。
            if self.Entity.SpriteRendererComponent.FlipX == false then
                fireDir = -1
            end
        end
    end
end

```

```

        -- 取得目前怪物的位置。
        local monsterPos = self.Entity.TransformComponent.Position
        -- 將怪物目前位置傳遞給投射體。
        local firePos = Vector3(monsterPos.x, monsterPos.y, 0)
        -- 命令投射體發射。
        missileComp:FireMissile(Vector2(fireDir, 0), false, firePos)
    end
end
end

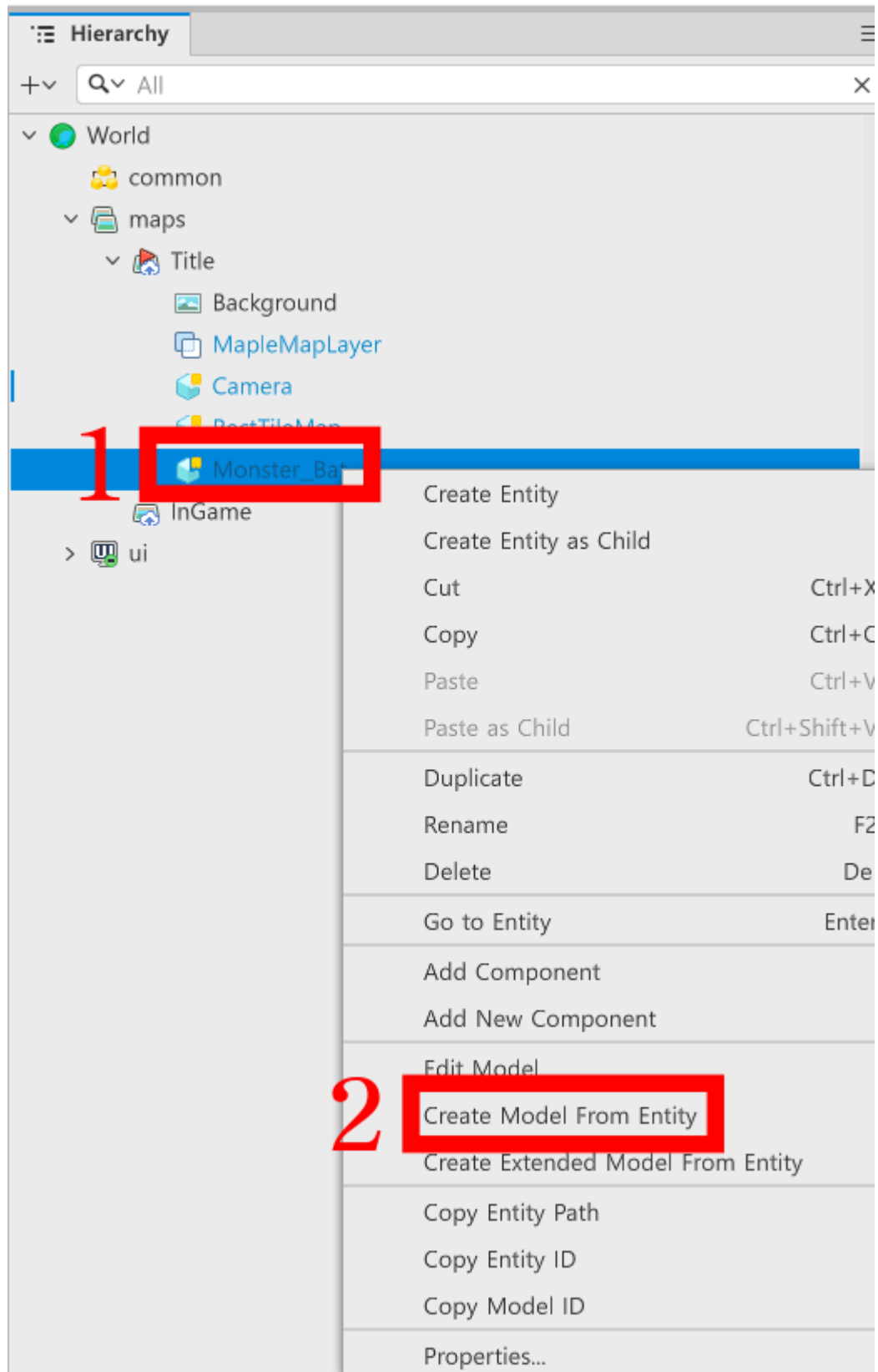
@ExecSpace("Client")
method void SendCalcDropItem()
    -- 當怪物死亡時，傳遞給ItemHelper以計算是否掉落道具。
    _ItemHelper:CalcDropItemData(self.itemDropRate, self.mustDropItemValue,
    self.Entity.TransformComponent.Position)
end

@ExecSpace("ClientOnly")
@EventSender("Self")
handler HandleHitEvent(HitEvent event)
    -- 若怪物狀態為死亡狀態，則不執行程式碼。
    if not self.isAlive then
        return
    end
    -- 取得發生碰撞的投射體實體。
    local attackerEntity = event.AttackerEntity
    -- 取得投射體傳遞的傷害。
    local totalDamage = event.TotalDamage
    -- 為了輸出Hit特效，登錄特效輸出位置。
    local effectSpawnPos = Vector3(attackerEntity.TransformComponent.Position.x,
    attackerEntity.TransformComponent.Position.y, 0)
    -- 進行傷害計算。
    self.currentHP = self.currentHP - totalDamage
    -- 若命中，則輸出Hit特效。
    _EffectService:PlayEffect(event.Extra, self.Entity, effectSpawnPos, 0,
    Vector3(0.5, 0.5, 0.5))
    -- 播放受擊音效。可透過變更第一個參數值來播放其他音效。
    _SoundService:PlaySound("5873f227d7604a63997546a92824c135", 1.0)
    -- 若體力變為0，則進行死亡處理。
    if self.currentHP <= 0 then
        -- 將存活狀態切換為false。
        self.isAlive = false
        -- 由於怪物已死亡，因此執行是否掉落道具。
        self:SendCalcDropItem()
        -- 輸出死亡時要顯示的特效。可透過變更第一個參數值來變更死亡時輸出的特效。
        _EffectService:PlayEffect("f67f25d5f8e14d2c9962be0fa42808f7", self.Entity,
        effectSpawnPos, 0, Vector3(0.5, 0.5, 0.5))
        -- 將怪物是否被擊殺傳遞給GameLogic，以檢查是否滿足條件。
        _GameLogic:CountKillScore()
        -- 刪除自己本身，讓實體消滅。
        _EntityService:Destroy(self.Entity)
    end
end
end

```

end

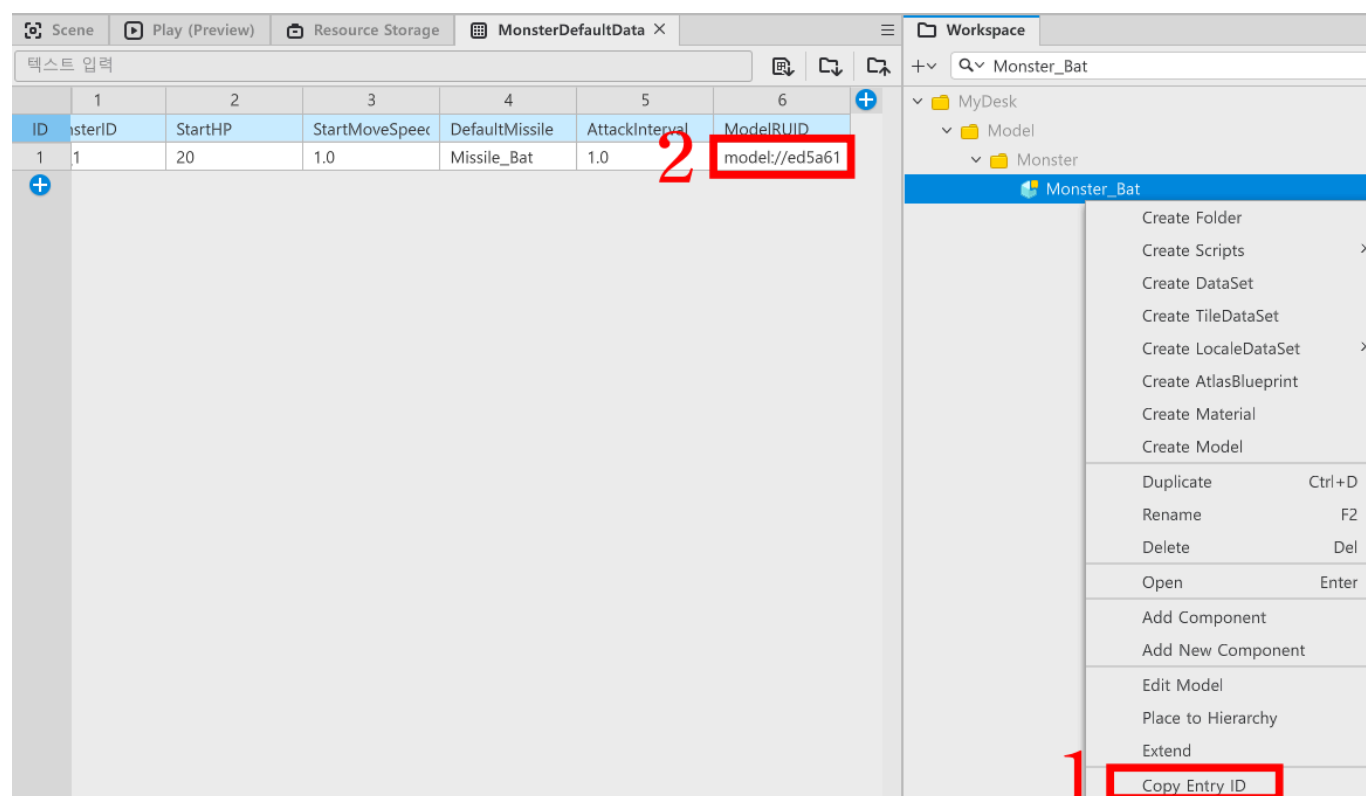
將製作完成的實體模型化



- 將製作完成的實體做成Model後，必須新增到WorkSpace中。

- 對要作成Model的實體按滑鼠右鍵後，點擊Create Model From Entity來製作Model。

在DataSet中新增模型ID



- 對在WorkSpace中建立的Model按滑鼠右鍵。
- 點擊Copy Entry ID以複製該模型的ID值。
- 將該值新增到MonsterDefaultData的ModelRUID中。

ModelRUID的作用是什麼？

- ModelRUID的作用是為找到WorkSpace中的Model，並複製該Model。
- 這部分會在進階學習項目實作道具中製作MonsterSpawnManager時使用。
- 學習透過ModelRUID的值來複製Model的方法後，就可以召喚各種Monster，或依需求複製其他Model。

最低實作標準

- 必須先製作構成Monster的Model。
- 製作完成的Monster Model必須能追蹤玩家。

應用與擴展方向

- 製作具有各種型態與數值的Monster Model，以多樣化準備Monster。
- 不使用AIChase來實作Monster的移動。
- 為每個Monster實作移動方式，以製作怪物的移動模式。

[💡 基礎學習]製作基本攻擊

影片時間軸

- 可在課程影片中以下時間點進行學習。 -製作基本攻擊：01:42:04

學習目標

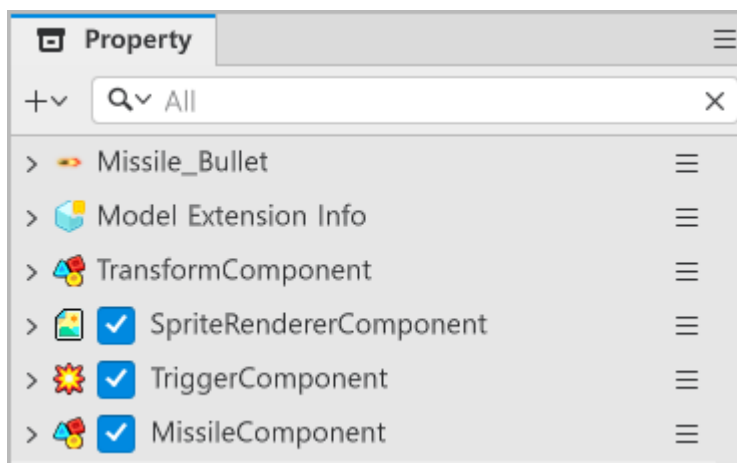
- 學習利用TriggerComponent來指定碰撞群組的Collision Group。
- 理解並可設定決定Collision Group之間碰撞關係的Matrix。

觀看該影片後可嘗試執行的內容

- 學習TriggerComponent的Collision Group之Matrix。
- 建立屬於自己的Collision Group後，進行透過Matrix建立關係的實作練習。

構成投射體實體

- 在Hierarchy中建立空實體，並如下構成Component。



- **SpriteRendererComponent**：用於輸出投射體圖片的Component。指定並使用想要的投射體圖片。
- **TriggerComponent**：用於指定投射體碰撞的Component。
- **MissileComponent**：為了控制投射體資料與行為而製作的Component。

撰寫MissileComponent腳本

- 如下撰寫腳本。
- 該腳本是以Plain Text為基準撰寫。

```
@Component
script MissileComponent extends Component

property number moveSpeed = 0 -- 用於儲存投射體速度的Property。

property number atkValue = 0 -- 用於儲存投射體攻擊力的Property。
```

```

property number lifeTime = 0 -- 用於儲存投射體持續時間的Property。

property boolean isShotPlayer = false -- 用於區分投射體發射主體的Property。

property string hitEffectRUID = "" -- 用於儲存投射體命中時要輸出的特效的Property。

property Vector2 fireDirection = Vector2(0,0) -- 用於儲存投射體發射方向的Proeprty。

property number deltaTimer = 0 -- 用於計算投射體產生時間的Property。

property string missileID = "" -- 用於儲存目前投射體ID的Property。

property any poolManager = nil

@ExecSpace("ClientOnly")
method void InitData(table MissileData)
-- 若傳入的MissileData是不可使用的值，則輸出錯誤日誌並結束程式碼。
if MissileData == nil or next(MissileData) == nil then
    log("MissileData是空的。")
    return
end

-- 從傳入的資料中儲存投射體的ID。
self.missileID = MissileData.MissileID
-- 從傳入的資料中儲存投射體的攻擊力。
self.atkValue = MissileData.AtkValue
-- 從傳入的資料中儲存投射體的移動速度。
self.moveSpeed = MissileData.MoveSpeed
-- 從傳入的資料中儲存投射體的持續時間。
self.lifeTime = MissileData.LifeTime
-- 從傳入的資料中儲存投射體碰撞時要輸出的特效ID。
self.hitEffectRUID = MissileData.HitEffectRUID
end

@ExecSpace("ClientOnly")
method void OnUpdate(number delta)
-- 若實體為不可使用狀態，則立即結束程式碼。
if self.Entity.Enable == false then
    return
end
-- 若遊戲狀態不是進行中狀態，則立即結束程式碼。
if not _GameLogic.IsStillPlay then
    return
end

-- 取得投射體目前位置。
local transform = self.Entity.TransformComponent
-- 計算依移動速度與時間應移動的距離。
local moveVec = Vector3(self.fireDirection.x, self.fireDirection.y, 0) *
self.moveSpeed * delta
-- 將投射體目前位置移動到應移動的位置。
transform.Position = transform.Position + moveVec

-- 將delta時間加入deltaTimer。

```

```

self.deltaTimeTimer = self.deltaTimeTimer + delta
-- 若deltaTimer大於lifeTime, 則回收投射體。
if self.deltaTimeTimer >= self.lifeTime then
    self:ReturnSelf()
end
end

@ExecSpace("ClientOnly")
method void ReturnSelf()
    -- 將投射體回收到ProjectileManager。
    if self.poolManager and isValid(self.poolManager) then
        self.poolManager:ReturnMissile(self.Entity)
    else
        -- 若poolManager為nil, 則銷毀投射體。
        _EntityService:Destroy(self.Entity)
    end
end

@ExecSpace("Client")
method void FireMissile(Vector2 FireDirection, boolean IsShotPlayer, Vector3
FireStartPos)
    -- 將deltaTimer初始化為0。
    self.deltaTimeTimer = 0
    -- 初始化投射體的發射方向。
    self.fireDirection = FireDirection
    -- 初始化投射體的發射主體。若投射體的發射主體是玩家, 則儲存為true。
    self.isShotPlayer = IsShotPlayer
    -- 將投射體目前位置變更為發射位置。
    self.Entity.TransformComponent.Position = FireStartPos
    -- 將投射體的使用狀態切換為true。
    self.Entity.Enable = true
end

@ExecSpace("ClientOnly")
@EventSender("Self")
handler HandleTriggerEnterEvent(TriggerEnterEvent event)
    -- 取得偵測到碰撞的實體。
    local trigger = event.TriggerBodyEntity
    -- 將用於儲存是否可碰撞的hitSuccess初始化為false。
    local hitSuccess = false
    -- 當玩家發射的投射體與怪物碰撞時, 進行處理。
    if isValid(trigger.MonsterState) and self.isShotPlayer then
        -- 若碰撞實體持有HitComponent, 且處於存活狀態, 則請求進行傷害計算。
        if trigger.HitComponent and trigger.MonsterState.isAlive then
            trigger.HitComponent:OnHit(self.Entity, self.atkValue, false,
self.hitEffectRUID, 1)
            hitSuccess = true
        end

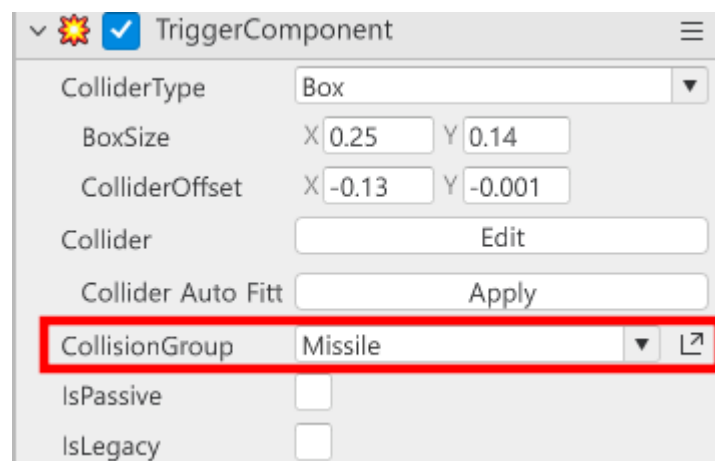
        -- 當怪物發射的投射體擊中玩家時, 進行處理。
    elseif isValid(trigger.PlayerState) and self.isShotPlayer == false then
        -- 檢查玩家是否持有HitComponent、是否為存活狀態, 以及目前是否為無敵狀態。
        if trigger.HitComponent and trigger.PlayerState.isAlive and not
trigger.PlayerState.isInvincibility then

```

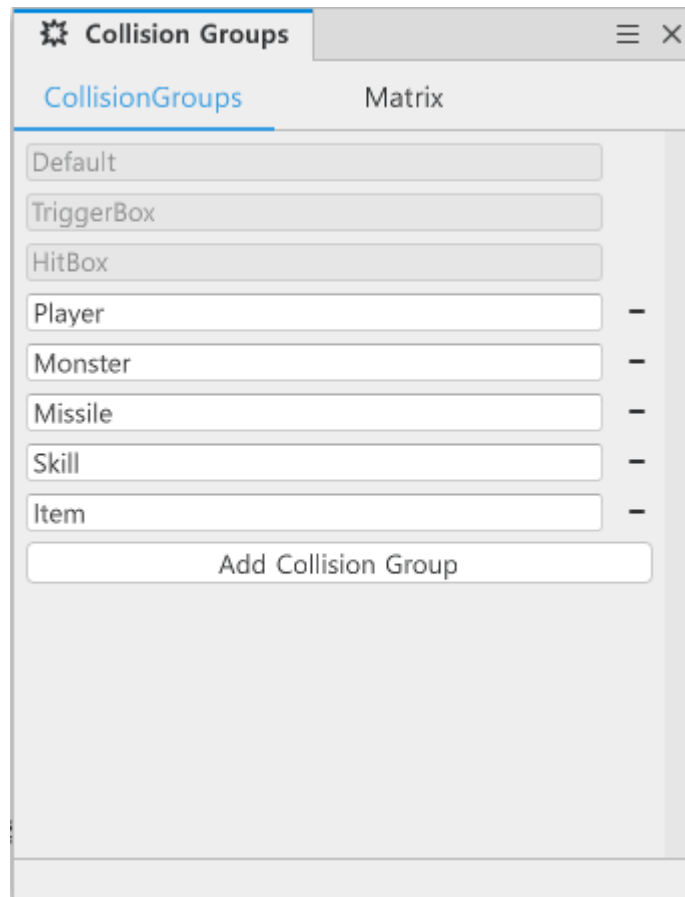
```
-- 若為可受傷狀態，則呼叫OnHit請求進行傷害計算。  
trigger.HitComponent:OnHit(self.Entity, self.atkValue, false,  
self.hitEffectRUID, 1)  
    hitSuccess = true  
end  
end  
  
-- 若已完成傷害計算處理，則再次停用實體。  
if hitSuccess then  
    self:ReturnSelf()  
end  
end  
  
end
```

設定TriggerComponent

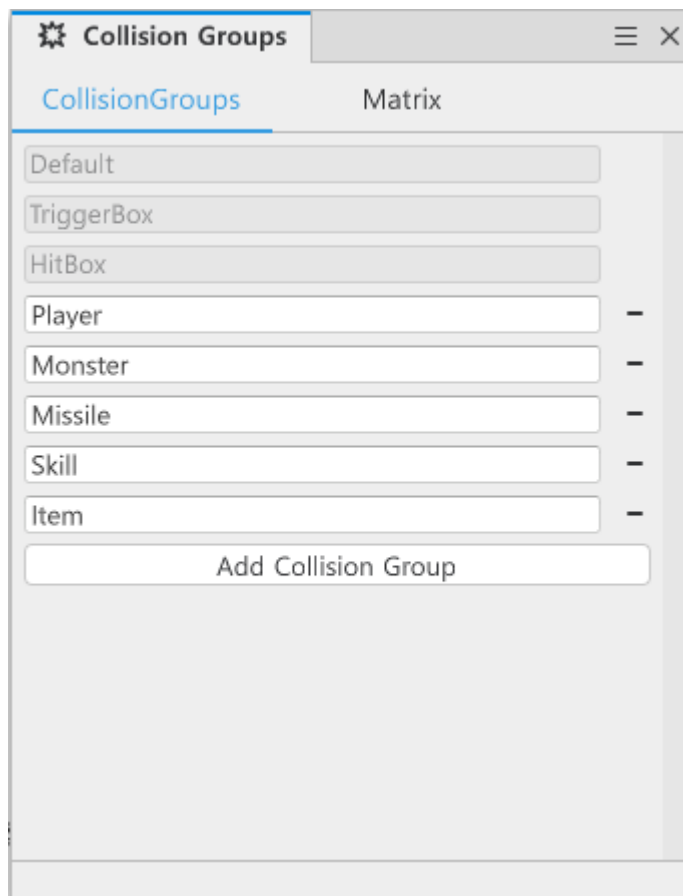
- 可透過變更TriggerComponent的組成要素，來變更Collision Group與Collision Group之間的碰撞關係。



- 找到已製作實體的TriggerComponent後，點擊CollisionGroup右側的開啟面板按鈕。



- 在Collision Groups視窗中點擊Add Collision Group以新增CollisionGroup。
 - **Player**: 用於指定玩家的Collision Group。
 - **Monster**: 用於指定敵人的Collision Group。
 - **Missile**: 用於指定投射體的Collision Group。
 - **Skill**: 用於指定技能的Collision Group。
 - **Item**: 用於指定會給玩家有利效果的Collision Group。
- 完成上述設定後，移動到Matrix。



- 如上圖所示設定Matrix。
- 該Matrix會形成如下碰撞關係。
 - Player會檢查與Monster、Missile、Item的碰撞。
 - Monster會檢查與Player、Missile、Skill的碰撞。
 - Missile會檢查與Player、Monster、Skill的碰撞。
 - Skill會檢查與Monster、Missile的碰撞。
 - Item會檢查與Player的碰撞。

指定Matrix的原因

- 碰撞運算是經常發生的檢查要素之一。
- 若新增不必要的CollisionGroup，並讓所有CollisionGroup都進行碰撞檢查，就會增加不必要的運算。
- 因此，為了最佳化效能，可以關閉不必要的碰撞檢查，只檢查必要碰撞，以提升效能。

最低實作標準

- 必須製作投射體Model，且至少要存在3個以上可使用的投射體。
- 必須透過投射體的InitData正常輸入投射體資料。
- 若以FireMissile發射透過InitData輸入數值的投射體，則投射體必須能正常發射。
- 投射體必須與Monster碰撞後消失。

應用與擴展方向

- 透過多樣化新增投射體移動方式來製作模式。
- 可透過各種投射體來建立新角色。
- 可讓每個敵人發射不同的投射體來製作模式。

[💡 基礎學習]建立物件池

影片時間軸

- 可在課程影片中以下時間點進行學習。
- 建立物件池：02:02:03

學習目標

- 學習物件池的原理。
- 可使用物件池來管理並使用投射體。
- 透過物件池管理記憶體使用量，以學習最佳化技術。

觀看該影片後可嘗試執行的內容

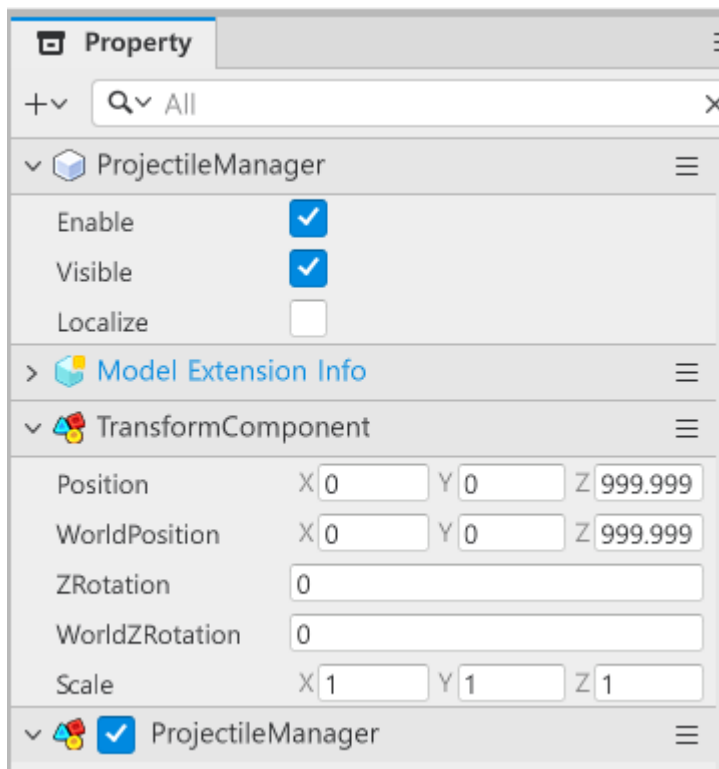
- 直接確認物件池的運作原理與實際執行結果。
- 透過這種方式思考哪些要素可以作為物件池使用。
- 設計結構，讓所想到的要素能以物件池方式進行管理。

什麼是物件池？

- 在進行遊戲時，會存在大量實體。
- 在這些實體中，像怪物這樣的單位有時會死亡並消滅。
- 但若單位數量很多，建立並消滅所有單位的行為會對電腦運算造成負擔。
- 但若事先建立實體，並不是消滅而只是停用，就能減少電腦負擔。
- 因為就像打開或關閉開關一樣，只要設定On、Off即可。
- 物件池會預先建立實體，再傳遞可使用的實體，以將不必要的產生與消滅工作降到最低。

製作物件池

- 為了製作物件池，必須在Map內建立實體。
- 以範例世界為基準，InGame地圖中的ProjectileManager被當作用來管理投射體的物件池使用。
- 物件池由如下Component構成。



- **TransformComponent**: 這是預設提供的Component。但為了能在地圖內正確位置出現，必須將Position值設為0。
- **ProjectileManager**: 這是為了製作物件池而新增的Component。由於該Component必須自行製作，因此必須透過New Component來製作。

設定ProjectileManager元件

- 必須如下撰寫程式碼。
- 該程式碼是在Plain Text環境下撰寫。

```
@Component
script ProjectileManager extends Component

property table projectilePool = {} -- 用於存放投射體實體的table。

@ExecSpace("ClientOnly")
method Entity GetMissile(string MissileKey)
-- 若沒有以請求的MissileKey作為Key值的要素，則嘗試初始化。
if self.projectilePool[MissileKey] == nil then
    self.projectilePool[MissileKey] = {}
end

-- 以MissileKey為Key，將projectilePool中的要素登錄到Pool中。
local pool = self.projectilePool[MissileKey]
-- 將用於儲存可使用實體的entity設為nil。
local entity = nil

-- 若pool中有要素存在，則開始搜尋。
if #pool > 0 then
    -- 搜尋pool中的要素。
    for i = 1, #pool do
```

```

        -- 若第i個實體存在, 且目前使用狀態為false, 則回傳該實體。
        if isvalid(pool[i]) and not pool[i].Enable then
            entity = pool[i]
            break
        end
    end
end

-- 若目前entity仍為nil, 表示不存在可用實體, 因此嘗試產生。
if entity == nil then
    -- 依據MissileKey值向DataTableLogic請求投射體資料。
    local missileData = _DataTableLogic:GetMissileDataByMissileID(MissileKey)
    -- 透過SpawnService建立投射體。
    entity = _SpawnService:SpawnByModelId(missileData.ModelRUID,
missileData.MissileID, Vector3.zero, self.Entity)
    -- 在建立的投射體中登錄投射體資料。
    entity.MissileComponent:InitData(missileData)
    -- 將自己登錄為投射體的poolManager。
    entity.MissileComponent.poolManager = self
    -- 將投射體的使用狀態變更為true。
    entity.Enable = true
    -- 將建立的實體登錄到projectilePool。
    table.insert(self.projectilePool[MissileKey], entity)
else
    -- 由於存在可使用的實體, 因此將找到的實體使用狀態變更為true。
    entity.Enable = true
end

-- 回傳找到或建立的實體。
return entity
end

@ExecSpace("ClientOnly")
method void ReturnMissile(Entity missile)
    -- 若傳入的實體為不可使用狀態, 則立即結束程式碼。
    if not isvalid(missile) then return end
    -- 將傳入實體的使用狀態設為將傳入實體的使用狀態設為false。
    missile.Enable = false
end

@ExecSpace("Client")
method void ResetPool()
    -- 以自己本身的實體為基準尋找子實體。
    local childList = self.Entity.Children
    -- 若沒有子實體, 則return。
    if childList == nil then return end
    -- 逐一巡覽找到的子實體清單, 將實體的Enable改為false。
    for i, child in pairs(childList) do
        if isvalid(child) then
            child.Enable = false
        end
    end
end

@ExecSpace("Client")

```

```

method Entity GetProjectileSkill(string SkillKey)
-- 若projectilePool中不存在以SkillKey為Value的值，則嘗試初始化。
if self.projectilePool[SkillKey] == nil then
    self.projectilePool[SkillKey] = {}
end

-- 將以SkillKey為Key的要素登錄到pool中。
local pool = self.projectilePool[SkillKey]
-- 將用於回傳的entity設為nil。
local entity = nil

-- 若pool中有要素存在，則開始搜尋。
if #pool > 0 then
    -- 搜尋pool中的要素。
    for i = 1, #pool do
        -- 若第i個實體存在，且目前使用狀態為false，則回傳該實體。
        if isValid(pool[i]) and not pool[i].Enable then
            entity = pool[i]
            break
        end
    end
end

-- 若pool中沒有可用的技能投射體實體，則新建實體。
if entity == nil then
    -- 以SkillKey為基準，向DataTableLogic請求資料。
    local skillData = _DataTableLogic:GetSkillDataBySkillID(SkillKey)
    -- 透過SpawnService請求建立投射體技能Model。
    entity = _SpawnService:SpawnByModelId(skillData.ModelRUID, skillData.SkillID,
    Vector3.zero, self.Entity)
    -- 請求輸入投射體技能的資料。
    entity.ProjectileSkillComponent:InitData(skillData)
    -- 將建立的實體登錄到projectilePool。
    table.insert(self.projectilePool[SkillKey], entity)
end

-- 將要回傳的實體使用狀態變更為true。
entity.Enable = true
-- 回傳找到或建立的實體。
return entity
end

end

```

- 先前撰寫的PlayerState與MonsterState都參照了ProjectileManager。
- PlayerState是透過ConnectMissilePool，MonsterState則是透過FindProjectileManager來尋找。
- 透過這種方式，Player與Monster都可以透過物件池發射投射體。

最低實作標準

- 利用物件池功能，實作讓投射體實體不會反覆建立與消滅。
- 投射體建立請求不由玩家或Monster自行提出。
- 所有投射體都以從物件池產生或回傳可使用實體的形式來實作。

應用與擴展方向

- 修改程式碼，讓其符合物件池**預先建立**後使用的原則。
- 以修改後的程式碼為基礎，確認投射體是否能正常使用。
- 實作功能，讓物件池只會預先建立必要的實體。

[📖 進階學習]實作技能

影片時間軸

- 可在課程影片中以下時間點進行學習。
- 製作技能：02:11:47

學習目標

- 學習TriggerEvent的種類。
- 理解各事件的觸發條件，並可製作所需功能。
- 理解TriggerEvent中的Stay具有什麼風險，並能說明應在什麼情況下使用該Event。

觀看該影片後可嘗試執行的內容

- 製作技能Model。
- 將影片中提供的投射體型技能製作成其他形式。
- 調整SkillData以調整與變更技能性能。
- 再製作一個投射體型技能後，變更玩家角色的技能。

製作DataSet

- 技能同樣也需要用來管理資料的DataSet。
- 在範例世界中，DataSet如下製作。

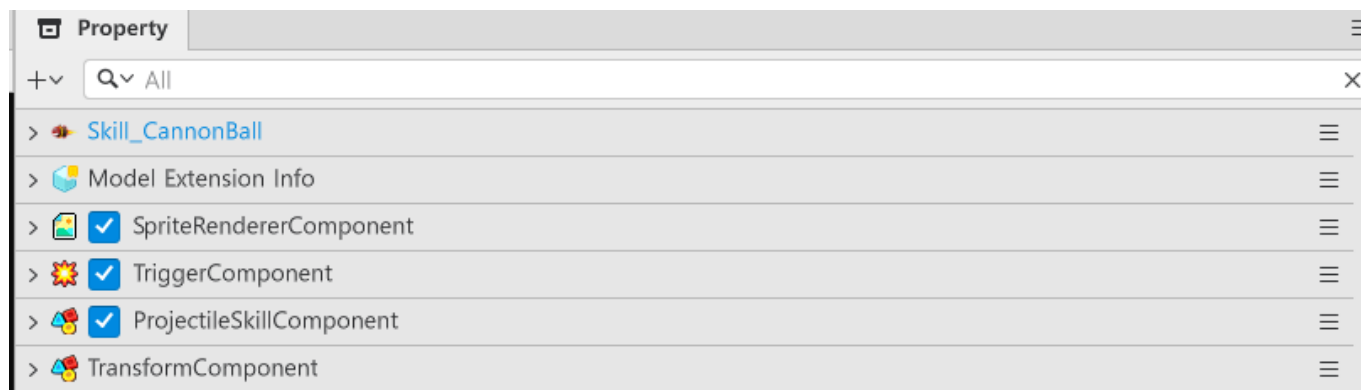
텍스트 입력										
	1	2	3	4	5	6	7	8	9	10
ID	SkillID	SkillType	IsPierce	AtkValue	MoveSpeed	LifeTime	MaxHitCount	HitInterval	ModelRUID	HitEffect
1	Skill_CannonBal	Projectile	true	20	1.0	5.0	5	0.2	model://a309b2	ee451f0e

姓名	說明
SkillID	這是用來區分技能的ID值。可依據該值指定玩家技能。
SkillType	這是用來區分技能種類的值。在範例世界中雖然只製作了投射體型，但這是為了日後製作其他型態技能而準備的值。

姓名	說明
IsPierce	在投射體技能的情況下，這是用來決定是否貫穿的值。若該值為false，則投射體技能在與1個敵人碰撞後即會消失。
AtkValue	這是用來指定技能攻擊力的值。
MoveSpeed	在投射體技能的情況下，這是用來指定移動速度的值。
LifeTime	在投射體技能的情況下，這是用來指定持續時間的值。
MaxHitCount	在多段Hit型技能的情況下，這是用來指定次數的值。
HitInterval	在多段Hit型技能的情況下，這是用來指定到下一次Hit所需時間的值。
ModelRUID	這是用來登錄技能Model的Entry ID的值。
HitEffectRUID	這是用來登錄被技能命中時應顯示的特效RUID的值。

構成技能實體

- 為了製作技能Model，必須構成實體。
- 如下在技能Entity上構成Component。



- SpriteRendererComponent**: 用來顯示技能圖片的Component。
- TriggerComponent**: 用來檢查技能是否發生碰撞的Component。必須將該Component的CollisionGroup設為Skill。
- ProjectileSkillComponent**: 用來控制投射體技能資料與行為的Component。必須透過Add Component來製作。

構成腳本

- 必須如下構成ProjectileSkillComponent。
- 以下程式碼是以Plain Text為基準撰寫。

```

@Component
script ProjectileSkillComponent extends Component

property table collideTable = {} -- 用來管理技能投射體正在碰撞中的實體的table。

property number atkValue = 0 -- 用來儲存技能投射體攻擊力的Property。

property number hitInterval = 0 -- 用來調整投射體技能多段Hit間隔的Property。

property number maxHitCount = 0 -- 用來調整投射體技能最大Hit間隔的Property。

property boolean isPierce = false -- 用來決定投射體技能是否可貫穿的Property。

property number lifeTime = 0 -- 用來決定投射體技能持續時間的Property。

property number moveSpeed = 0 -- 用來決定投射體技能移動速度的Property。

property string hitEffectRUID = "" -- 用來儲存被投射體技能命中時要顯示的特效ID的Property。

property number deltaTimer = 0 -- 用來計算投射體技能持續時間的Property。

property Vector2 fireDirection = Vector2(0,0) -- 用來儲存投射體應前進方向的Property。

@ExecSpace("Client")
method void InitData(table SkillData)
--檢查傳入的技能資料是否有效。
if not isValid(SkillData) or next(SkillData) == nil then
    log("傳入的SkillData是nil! ")
    return
end
-- 進行技能資料初始化。
--初始化用來檢查投射體技能壽命的deltaTimer。
self.deltaTimer = 0
-- 初始化存放已碰撞怪物的table。
self.collideTable = {}
-- 初始化技能攻擊力數值。
self.atkValue = SkillData.AtkValue
-- 初始化技能攻擊延遲數值。
self.hitInterval = SkillData.HitInterval
-- 初始化技能最大攻擊次數。
self.maxHitCount = SkillData.MaxHitCount
-- 初始化技能持續時間。
self.lifeTime = SkillData.LifeTime
-- 初始化技能移動速度。
self.moveSpeed = SkillData.MoveSpeed
-- 檢查技能是否可貫穿。
self.isPierce = SkillData.IsPierce
-- 初始化命中技能時要顯示的特效RUID。
self.hitEffectRUID = SkillData.HitEffectRUID
-- 先停用投射體技能的使用狀態。

```

```

self.Entity.Enable = false
end

@ExecSpace("Client")
method table GetOrCreateCollideEntity(Entity CollideEntity)
-- 若CollideEntity可使用, 則執行以下程式碼。
if invalid(CollideEntity) then
-- 取得CollideEntity的ID。
local entityID = CollideEntity.Id
-- 若collideTable中沒有持有該ID值的實體, 則新登錄到collideTable中。
if not invalid(self.collideTable[entityID]) or not
next(self.collideTable[entityID]) then
self.collideTable[entityID] = {
    Target = CollideEntity,
    HitCount = 0,
    LastHitTime = nil
}
end
-- 回傳持有CollideEntity ID的實體。
return self.collideTable[entityID]
end
end

@ExecSpace("Client")
method void ApplyDamage(table CheckData)
-- 若已死亡而不存在實體, 則將該ID的Table Value改為nil。
if not invalid(CheckData.Target) then
self.collideTable[CheckData.Target.Id] = nil
return
end

-- 檢查目標是否有MonsterState 元件後, 對該元件執行傷害計算。
if invalid(CheckData.Target.MonsterState) then
    CheckData.Target.HitComponent.OnHit(self.Entity, self.atkValue, false,
self.hitEffectRUID, 1)
end

-- 登錄最後命中時間。
CheckData.LastHitTime = _UtilLogic.ElapsedSeconds
-- 該實體可承受的傷害次數增加1。
CheckData.HitCount = CheckData.HitCount + 1
end

@ExecSpace("ClientOnly")
method void OnUpdate(number delta)
-- 若遊戲的Play狀態為false, 則中止執行以下程式碼。
if not _GameLogic.IsStillPlay then
return
end
-- 請求投射體技能移動。
self:Move(delta)
-- 請求計算投射體的持續時間。
self:RunLifeTimer(delta)

```

```

end

@ExecSpace("Client")
method void RunLifeTimer(number DeltaTime)
-- 將投射體技能的deltaTimer增加DeltaTime。
self.deltaTimer = self.deltaTimer + DeltaTime
-- 若deltaTimer超過lifeTime, 則停用使用狀態。
if self.deltaTimer >= self.lifeTime then
    self.Entity.Enable = false
end

end

@ExecSpace("Client")
method void Move(number DeltaTime)
-- 取得實體目前位置。
local transform = self.Entity.TransformComponent
-- 以投射體技能的移動速度與DeltaTime來計算位置。
local moveVec = Vector3(self.fireDirection.x, self.fireDirection.y, 0) *
self.moveSpeed * DeltaTime
-- 變更投射體技能的位置。
transform.Position = transform.Position + moveVec
end

@ExecSpace("Client")
method void FireSkill(Vector2 FireDirection, Vector3 FireStartPos)
-- 將用於計算投射體技能持續時間的deltaTimer值初始化為0。
self.deltaTimer = 0
-- 初始化投射體技能的發射方向。
self.fireDirection = FireDirection
-- 初始化投射體技能的發射位置。
local transform = self.Entity.TransformComponent
if transform then
    transform.Position = FireStartPos
end
-- 將投射體的使用狀態切換為true。
self.Entity.Enable = true
end

@EventSender("Self")
handler HandleTriggerStayEvent(TriggerStayEvent event)
-- 確認正在碰撞的Entity是否正常存在。
if invalid(event.TriggerBodyEntity) then
    --若正在碰撞的Entity是投射體, 則刪除敵方投射體。
    if invalid(event.TriggerBodyEntity.MissileComponent) and not
event.TriggerBodyEntity.MissileComponent.isShotPlayer then
        event.TriggerBodyEntity.MissileComponent.ReturnSelf()
        return
    end
    --若正在碰撞的實體是敵人, 則在滿足Hit次數的情況下執行傷害計算。
    if invalid(event.TriggerBodyEntity.MonsterState) then
        -- 從collideTable中回傳與正在碰撞的實體相同的實體。
        local collideTable =
self:GetOrCreateCollideEntity(event.TriggerBodyEntity)

```

```

-- 若回傳的實體不可使用，則中止執行程式碼。
if not isValid(collideTable) then
    return
end
-- 若HitCount超過最大值，則中止執行程式碼。
if collideTable.HitCount >= self.maxHitCount then
    return
end
-- 取得目前時間。
local currentTime = _UtilLogic.ElapsedSeconds
-- 若一次傷害計算都尚未進行，則立即造成傷害。
if collideTable.LastHitTime == nil then
    self:ApplyDamage(collideTable)
-- 若受傷時間大於多段Hit時間，則嘗試進行傷害計算。
elseif currentTime - collideTable.LastHitTime >= self.hitInterval then
    self:ApplyDamage(collideTable)
end
end
end
end

@EventSender("Self")
handler HandleTriggerEnterEvent(TriggerEnterEvent event)
-- 若event不正常，則輸出錯誤日誌。
if not isValid(event) then
    log("event不是正常的event! ")
    return
end
-- 若碰撞的實體持有MissileComponent，且發射主體不是玩家，則停用該碰撞實體。
if isValid(event.TriggerBodyEntity.MissileComponent) and not
event.TriggerBodyEntity.MissileComponent.isShotPlayer then
    event.TriggerBodyEntity.MissileComponent:ReturnSelf()
    return
end
-- 當碰撞的實體持有MonsterState時，執行以下程式碼。
if isValid(event.TriggerBodyEntity.MonsterState) then
    -- 檢查collideTable中是否有與碰撞實體相同的實體。
    local findList = self:GetOrCreateCollideEntity(event.TriggerBodyEntity)

    -- 若碰撞實體受到傷害的次數已超過最大值，則中止執行程式碼。
    if findList.HitCount >= self.maxHitCount then return end

    -- 立即請求對碰撞實體進行傷害處理。
    self:ApplyDamage(findList)
end
end

@EventSender("Self")
handler HandleTriggerLeaveEvent(TriggerLeaveEvent event)
-- 若離開碰撞範圍的實體為不可使用狀態，則中止執行程式碼。
if not isValid(event) then
    return
end
-- 尋找離開碰撞範圍的實體是否存在於collideTable內。

```

```
local record = self.collideTable[event.TriggerBodyEntity.Id]
-- 若該實體存在於collideTable內，則初始化最後命中時間，讓它再次進入碰撞範圍時可立即受到傷害。
if record then
    record.LastHitTime = nil
end
end

end
```

- 完成上述作業後，必須將技能實體模型化，並將Model的EntryID登錄到SkillData的ModelRUID中。

HandleTrigger事件的呼叫時機

- **EnterEvent**：當2個以上持有TriggerComponent的實體之碰撞範圍重疊的瞬間呼叫。
- **StayEvent**：當2個以上持有TriggerComponent的實體之碰撞範圍持續重疊時，會在每一幀呼叫。
- **LeaveEvent**：當2個以上持有TriggerComponent的實體原本正在碰撞，而碰撞範圍重疊解除的瞬間呼叫。

最低實作標準

- 玩家可以使用技能實體。
- 技能實體可以消除敵方投射體。
- 投射體技能可以攻擊多個敵人，並能依指定次數攻擊敵人。

應用與擴展方向

- 除了投射體型技能外，也可以製作其他形式的技能。
- 可以製作成讓投射體型技能不是直線移動，而是展現各種動作後再發射。

[📖 進階學習]實作道具

影片時間軸

- 可在課程影片中以下時間點進行學習。
- 實作道具：02:32:10

學習目標

- 學習利用Logic來使用負責特定功能的Logic的方法。
- 學習考量擴充性的設計，以便日後若想變更道具產生的功能時，只需變更Logic即可。

觀看該影片後可嘗試執行的內容

- 變更道具效果數值，並確認是否依變更後的數值套用。
- 除了已製作的道具外，也製作提供其他效果的Model。

製作DataSet

- 為了製作與道具掉落相關的功能，大致需要2種DataSet。
- 存在ItemData、ItemDropData這2種。
 - **ItemData**：用來決定道具效果或性能的DataSet。
 - **ItemDropData**：用來決定道具掉落機率的DataSet。

ItemDropData DataSet構成

	1	2	+
ID	ItemID	DropRate	
1	Item_LifeRecover	20	
2	Item_SkillCount	20	
3	Item_MoveSpeed	20	
4	Item_AttackInterval	20	
5	Item_ChangeMissile	20	
+			

姓名	說明
ItemID	這是用來區分是什麼道具的ID值。使用ItemData的ID值。
DropRate	這是用來決定道具掉落機率的值。

ItemData DataSet構成

	1	2	3	4	5	6	7	+
ID	ItemID	ValueType	TargetPlayerStat	NumberValue	StringValue	SpriteRUID	GainEffect	
1	Item_LifeRecover	1	currentHP	1		a2cfdb8831124f	32317a7b70484	
2	Item_SkillCount	1	canUseSkillCount	1		04640b1d108d4	196d4ef7652f44	
3	Item_MoveSpeed	1	moveSpeed	0.05		e223bdc2a6cd4	6802f1a489f94b	
4	Item_AttackInterval	1	attackInterval	-0.05		fafd61650bf146	3db73bd6398e4	
5	Item_ChangeMissile	2	currentMissile		Missile_Missile	fd8586b0c4c94c	0915b72bf4374	
+								

姓名	說明
ItemID	這是用來區分道具的ID值。該值會作為ItemDropData的ItemID值使用。
ValueType	這是用來區分道具效果值是數值型還是文字型的要素。

姓名	說明
TargetPlayerStateName	這是用來設定應影響PlayerState中哪個Property的值。必須與PlayerState的Property名稱完全相同。
NumberValue	若為具有數值型值的道具效果，則會依該值套用效果。
StringValue	若為具有文字型值的道具效果，則會依該值套用效果。在範例世界中，是以實作基本攻擊變更道具為目的而製作。
SpriteRUID	這是為了變更道具圖片而製作的值。透過該值變更道具圖片。
GainEffect	這是用來指定獲得道具時之道具獲得特效的值。

建立ItemSpawnManager

- 這是實際負責產生道具的Component。
- 該要素是以實體形式製作，並以範例世界為基準製作在InGame中。
- 在製作好實體與Component後，必須將以下程式碼登錄到ItemSpawnManager中。
- 程式碼是以Plain Text為基準撰寫。

```
@Component
script ItemSpawnManager extends Component

property string itemModelID = "model://0b6340f2-19d9-4a27-b18a-70feb94ce2ae" -- 這
是在Workspace內作為道具使用的Model ID。

property table itemObjectPool = {} -- 這是用來尋找並重複使用可用實體的table。

@ExecSpace("Client")
method void CreateItem(string ItemKey, Vector3 ItemSpawnPosition)
-- 檢查itemObjectPool是否為nil。
if self.itemObjectPool[ItemKey] == nil then
-- 若itemObjectPool為nil，則初始化為{}。
self.itemObjectPool[ItemKey] = {}
end

-- 將pool內傳入的ItemKey種類實體登錄到pool中。
local pool = self.itemObjectPool[ItemKey]
-- 為了登錄可用的實體，將returnEntity設為nil。
local returnEntity = nil

-- 當pool內要素至少有1個時，開始搜尋。
if #pool > 0 then
-- 依pool的要素數量開始搜尋。
for i = 1, #pool do
-- 若第i個實體可使用，則回傳第i個實體。
if isValid(pool[i]) and not pool[i].Enable then
returnEntity = pool[i]
```

```

        break
    end
end
end

-- 依據ItemKey值從DataTableLogic中尋找值。
local itemData = _DataTableLogic:GetItemDataByItemID(ItemKey)
-- 若直到目前returnEntity仍為nil, 則判定為沒有可用實體。
if returnEntity == nil then
    -- 透過SpawnService, 將已做成Model的道具實體建立到地圖中。
    returnEntity = _SpawnService:SpawnByModelId(self.itemModelID, itemData.ItemID,
        Vector3.zero, self.Entity)
    -- 為了讓已建立的道具可以再次被回收, 將自己登錄到ItemComponent的poolManager中。
    returnEntity.ItemComponent.poolManager = self
    -- 為了防止道具處於不可使用狀態, 將實體的Enable改為true。
    returnEntity.Enable = true
    -- 將建立的要素以ItemKey值作為Key、實體作為Value新增到itemObjectPool中。
    table.insert(self.itemObjectPool[ItemKey], returnEntity)
else
    -- 若returnEntity存在, 則將Enable改為true, 以便可再次使用。
    returnEntity.Enable = true
end

-- 初始化已建立的道具實體資料。
returnEntity.ItemComponent:InitData(itemData, ItemSpawnPosition)
end

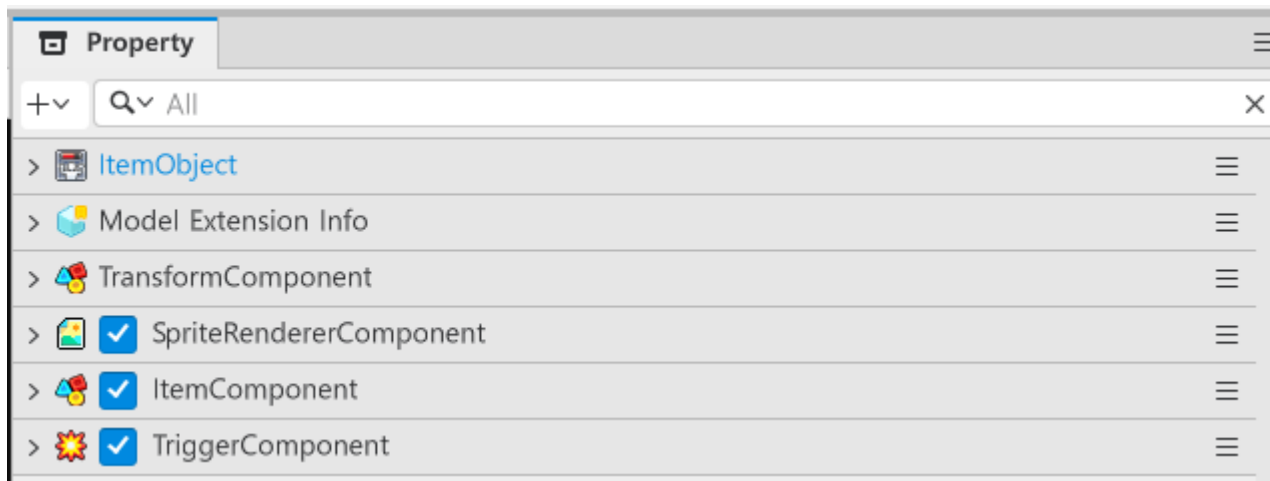
@ExecSpace("Client")
method void ReturnItemEntity(Entity ItemEntity)
    -- 若傳入的ItemEntity為不可使用的實體, 則透過return離開程式碼。
    if not isValid(ItemEntity) then return end
    -- 若ItemEntity可使用, 則停用Enable, 切換為可再次使用的待機狀態。
    ItemEntity.Enable = false
end

end

```

製作道具實體

- 必須製作用來構成道具的實體, 並將該實體登錄為Model。此時Model的EntryID值會作為ItemSpawnManager的itemModelID值使用。



- **SpriteRendererComponent**: 用來顯示道具圖片的Component。會將該Component的ImageRUID值套用為ItemData的SpriteRUID值來變更道具圖示。
- **TriggerComponent**: 用來偵測道具碰撞的Component。新增該Component後，必須將Collision Group 變更為Item。
- **ItemComponent**: 用來承載道具移動與效果的Component。由於該Component必須自行製作，因此必須透過Add Component、New Component來製作。

撰寫ItemComponent腳本

- 必須將以下程式碼撰寫到ItemComponent中。
- 程式碼是以Plain Text為基準撰寫。

```
@Component
script ItemComponent extends Component

property SpriteRendererComponent imageSpriteRenderer = nil

property number itemID = 0 -- 用來儲存ItemData的ID值的Property。

property number valueType = 0 -- 用來區分ItemData套用類型的Property。

property string targetPlayerStateName = "" -- 用來承載應套用於PlayerState 元件之
Property名稱的Property。

property number numberValue = 0 -- 若需套用數值型的值，用來承載數值的Property。

property string stringValue = "" -- 若需套用文字型的值，用來承載文字的Property。

property number moveSpeed = 3.0 -- 用來儲存道具在地圖中移動速度的Property。

property number lifeTime = 3.0 -- 用來承載道具可留存在地圖上的時間的Property。

property string gainEffect = "" -- 用來承載取得道具時要顯示之特效ID的Property。

property any poolManager = nil
```

```

property number deltaTime = 0 -- 用來計算道具持續時間的Property。

@ExecSpace("Client")
method void InitData(table ItemData, Vector3 SpawnPosition)
-- 若管理道具圖片Sprite的Property為nil, 則開始初始化。
if not invalid(self.imageSpriteRenderer) then
-- 在附加該元件的實體中尋找SpriteRendererComponent。
local findImageSpriteRenderer = self.Entity.SpriteRendererComponent
if not invalid(findImageSpriteRenderer) then
-- 若找不到SpriteRendererComponent, 則留下找不到的錯誤日誌。
log("道具Model中不存在SpriteRendererComponent! ")
return
end
-- 若找到了SpriteRenderer, 則將找到的元件登錄到Property中。
self.imageSpriteRenderer = findImageSpriteRenderer
end

-- 若傳入的ItemData為nil, 則回傳錯誤日誌且不嘗試初始化。
if not invalid(ItemData) or not next(ItemData) then
log("透過Init傳入的ItemData是nil! ")
return
end

-- 將deltaTimer初始化為0, 以計算持續時間。
self.deltaTime = 0
-- 將傳入的ItemData資料登錄到Component的Property中。
-- 登錄道具圖示圖片RUID。
self.imageSpriteRenderer.SpriteRUID = ItemData.SpriteRUID
-- 登錄決定道具影響力的Value值。
self.valueType = ItemData.ValueType
-- 登錄與PlayerState的Property同名的值。
self.targetPlayerStateName = ItemData.TargetPlayerStateName
-- 若為數值型變動道具, 則登錄增加量。
self.numberValue = ItemData.NumberValue
-- 若為像基本投射體一樣會變更String型Key值的道具, 則登錄String值。
self.stringValue = ItemData.StringValue
-- 登錄取得道具時要顯示的特效ID值。
self.gainEffect = ItemData.GainEffect
-- 接收怪物死亡的位置, 並在該位置建立。
self.Entity.TransformComponent.Position = SpawnPosition
-- 若實體處於停用狀態, 則將其啟用。
if not self.Entity.Enable then
self.Entity.Enable = true
end
end

@ExecSpace("ClientOnly")
method void OnUpdate(number delta)
-- 若實體處於停用狀態, 則立即透過Return不執行以下腳本。
if not self.Entity.Enable then
return
end

-- 若遊戲的Play狀態不是遊玩中, 則透過Return不執行以下腳本。

```

```

if not _GameLogic.IsStillPlay then
    return
end
-- 若deltaTimer超過道具生存時間，則停用道具。
if self.deltaTimer > self.lifeTime then
    self:ReturnSelf()
    return
else
    -- 若deltaTimer尚未超過lifeTime，則依已經過的時間增加deltaTimer。
    self.deltaTimer = self.deltaTimer + delta
end

-- 每幀呼叫道具的移動函式。
self:ItemMove(delta)
end

@ExecSpace("Client")
method void ItemMove(number DeltaTime)
-- 取得管理道具位置的TransformComponent。
local transform = self.Entity.TransformComponent

-- 計算道具應移動的距離。
local moveVec = Vector3(-1, 0, 0) * self.moveSpeed * DeltaTime

-- 將道具目前位置移動相應的距離。
transform.Position = transform.Position + moveVec
end

@ExecSpace("Client")
method void ReturnSelf()
-- 若LifeTime結束，則將道具的Enable改為false。
if self.poolManager and isvalid(self.poolManager) then
    self.poolManager:ReturnItemEntity(self.Entity)
else
    -- 若沒有poolManager，則刪除道具實體。
    _EntityService:Destroy(self.Entity)
end
end

@EventSender("Self")
handler HandleTriggerEnterEvent(TriggerEnterEvent event)
-- 當碰撞到的實體具有PlayerState時，進行使用道具處理。
if isvalid(event.TriggerBodyEntity.PlayerState) then
    -- 執行碰撞到的玩家實體之AppliedItem方法。
    event.TriggerBodyEntity.PlayerState:AppliedItem(self.valueType,
self.targetPlayerStateName, self.stringValue, self.numberValue, self.gainEffect)
    -- 將該實體設為停用狀態。
    self:ReturnSelf()
end
end

end

```

製作ItemHelper

- ItemHelper是在怪物死亡時，用來請求建立道具的Logic。
- 當ItemHelper收到請求後，會進行機率計算，若需掉落道具，則向ItemSpawnManager請求建立道具。
- 在範例世界中，是以ItemHelper這個名稱來製作Logic。
- 程式碼是以Plain Text為基準撰寫。

```
@Logic
script ItemHelper extends Logic

property ItemSpawnManager itemSpawnManager = nil

method void CalcDropItemData(number ItemDropRate, string MustDropItemKey, Vector3
SpawnPosition)
-- 由於道具掉落機率為0，因此不執行機率計算並回傳nil。
if not isValid(ItemDropRate) or ItemDropRate <= 0.0 then
    return
end

-- 其他情況則判定是否會掉落道具。
if ItemDropRate >= 1.0 then
    -- 由於是必定掉落道具的情況，因此不執行額外運算。
else
    -- 依照ItemDropRate的值進行隨機。
    local randomValue = _UtilLogic:RandomDouble()
    if ItemDropRate < randomValue then
        -- 因隨機值高於道具掉落機率，因此視為掉落失敗並回傳nil。
        return
    end
end

-- 若指定了MustDropItemKey，則必定回傳該道具。
if not _UtilLogic:IsNilorEmptyString(MustDropItemKey) then
    -- 依據MustDropItemKey向DataTableLogic請求資料。
    local findData = _DataTableLogic:GetItemDataByItemID(MustDropItemKey)
    -- 當存在具有MustDropItemKey值的資料時，建立道具。
    if isValid(findData) and next(findData) then
        self:SendSpawnItem(findData.ItemID, SpawnPosition)
        return
    else
        -- 若不存在，則留下找不到道具的錯誤日誌。
        log("找不到MustDropItemKey的道具！")
        return
    end
end

-- 若MustDropItemKey為空白，則進行隨機以提供隨機道具。
local getDropData = _DataTableLogic:GetItemDropData()
local totalValue = 0
-- 將道具掉落表中的所有機率相加後，在該機率中嘗試計算。
for _, data in pairs(getDropData) do
    totalValue = totalValue + data.DropRate
end
```

```

end
-- 在0到totalValue之間決定一個值後，從1開始累加DropRate，並在超過隨機值的時點將該道具視為中獎並回傳。
local randomValue = _UtilLogic:RandomIntegerRange(0, totalValue)
local resultValue = 0
for _, result in pairs(getDropData) do
    -- 將DropRate加到resultValue中。
    resultValue = resultValue + result.DropRate
    -- 若resultValue的值大於決定出的randomValue，則視為中獎。
    if resultValue >= randomValue then
        -- 在DataTableLogic中以ItemID值尋找ItemData。
        local findData = _DataTableLogic:GetItemDataByItemID(result.ItemID)
        -- 若正常取得資料，則回傳找到的資料。
        if isValid(findData) and next(findData) then
            self:SendSpawnItem(findData.ItemID, SpawnPosition)
            return
        else
            -- 若找到的資料為nil或不可使用的資料，則留下日誌並回傳nil。
            log("在ItemData中找不到名為"..result.ItemID.."的值!")
            return
        end
    end
end
end
end

method void SendSpawnItem(string ItemID, Vector3 SpawnPosition)
-- 若在InGame中找不到ItemSpawnManager，則輸出錯誤日誌。
if not isValid(self.itemSpawnManager) then
    log("ItemHelper的itemSpawnManager是nil!")
    return
end
-- 向InGame的ItemSpawnManager請求建立道具。
self.itemSpawnManager:CreateItem(ItemID, SpawnPosition)
end

method void InitData()
-- 若找不到InGame的ItemSpawnManager，則尋找InGame的ItemSpawnManager。
if not isValid(self.itemSpawnManager) then
    local findEntity =
_EntityService:GetEntityByPath("/maps/InGame/ItemSpawnManager")
    -- 若找不到ItemSpawnManager，則輸出錯誤日誌。
    if not isValid(findEntity.ItemSpawnManager) then
        log("InGame地圖中不存在ItemSpawnManager實體，或ItemSpawnManager 元件!")
        return
    end
    -- 將找到的ItemSpawnManager登錄到property中。
    self.itemSpawnManager = findEntity.ItemSpawnManager
end
end

method void ReleaseData()
-- 在Title畫面中解除對ItemSpawnManager的存取，以避免被使用。
self.itemSpawnManager = nil
end

```

end

最低實作標準

- ItemHelper必須能正常輸出隨機道具。
- ItemComponent必須能正常取得並套用ItemData的DataSet值。

應用與擴展方向

- 除了已製作的道具外，也要製作能提供其他效果的道具。
- 可依需求加入投射體強化功能與道具獲得效果，實作基本攻擊強化系統。

[📖 進階學習]製作MonsterSpawn

影片時間軸

- 此內容會在實作道具項目中一併說明。
- 實作道具：02:32:10

學習目標

- 學習利用TimerService在一定時間後執行想要動作的方法。
- 利用GameLogic構成功能，以便能檢查GameOver與Stage通關。

觀看該影片後可嘗試執行的內容

- 透過變更MonsterSpawnData的值，調整Monster出現的時間。
- 另外製作Monster後，修改MonsterSpawnData，讓該Monster能夠建立。
- 修改ItemDropRate與MustDropItem，以改變各Monster在被擊敗時掉落道具的機率。

製作DataSet

- 為了建立Monster，必須製作DataSet。
- 在範例世界中，是以MonsterSpawnData這個名稱製作。

	1	2	3	4	5	6	7	+
ID	StageID	MonsterID	SpawnPosX	SpawnPosY	EntranceTime	ItemDropRate	MustDropItem	
1	1	Bat_1	8.0	2.0	3.0	0.0		
2	1	Bat_1	8.0	-2.0	5.0	0.8		
3	1	Bat_1	6.0	0.0	8.0	1.0	Item_MoveSpee	
4	1	Bat_1	6.0	-5.0	8.0			
+								

姓名	說明
StageID	這是MonsterSpawnManager為了依照目前Stage編號區分應建立Monster種類的ID值。
MonsterID	這是用來尋找應該建立哪種Monster的ID值。會依據該值在MonsterDefaultData中以MonsterID為基準進行搜尋。
SpawnPosX, SpawnPosY	這是用來指定Monster應該在地圖哪個位置出現的值。分別是Position的X、Y值。
EntranceTime	這是用來指定該Monster在遊戲開始後幾秒出現的值。
ItemDropRate	這是用來指定Monster死亡時掉落道具機率的值。
MustDropItem	這是當Monster必須掉落道具時，想固定掉落特定道具時所使用的值。該值使用的是ItemData的ItemID值。

製作MonsterSpawnManager

- 為了製作MonsterSpawnManager，必須在InGame中新增實體。
- 此外還必須構成MonsterSpawnManager Component。
- 構成MonsterSpawnManager的程式碼如下。
- 該程式碼是以Plain Text為基準撰寫。

```
@Component
script MonsterSpawnManager extends Component

property number stageID = 0 -- 這是用來區分Stage的Property。

property number needKillCount = 0 -- 這是用來在Stage開始時，計算並儲存可擊殺怪物數量的Property。

property table timerTable = {} -- 這是用來儲存怪物出現時間的table。

@ExecSpace("Client")
method void StartGame()
-- 先將Stage通關所需的怪物擊殺數初始化為0。
self.needKillCount = 0
-- 初始化存放Stage中怪物出現時間的timerTable。
self.timerTable = {}
-- 若stageID為0，為了讓遊戲能正常執行，會暫時設為1。
if self.stageID == 0 then
    self.stageID = 1
    -- 此外也會以Log告知目前stageID被登錄為0。
    log("SpawnManager中未登錄StageID! ")
    return
```

```

end
-- 依照stageID, 從DataTableLogic取得怪物產生資料。
local getData = _DataTableLogic:GetMonsterSpawnDataTableByStageID(self.stageID)
-- 若getData不可使用, 則透過log輸出錯誤。
if not isValid(getData) or next(getData) == nil then
    log("無法取得資料表中"..self.stageID.."的值! ")
    return
end
-- 計算取得的getData數量, 並逐一處理getData的值。
for i = 1, #getData do
    -- 將getData的第i筆資料登錄到data中。
    local data = getData[i]
    -- 若data可使用, 則透過data的值嘗試產生怪物。
    if data then
        -- 由於存在可用怪物數量, 因此為了通關, 將需要擊殺的數量增加1。
        self.needKillCount = self.needKillCount + 1
        -- 設定怪物產生用的計時器。
        self.timerTable[i] = _TimerService:SetTimerOnce(function()
            -- 向DataTableLogic請求要產生之怪物的資料。
            local monsterData =
                _DataTableLogic:GetMonsterDefaultDataByMonsterID(data.MonsterID)

            -- 產生要建立的怪物。
            local monster = _SpawnService:SpawnByModelId(
                -- 透過怪物的ID尋找Model, 並產生該Model。
                monsterData.ModelRUID,
                -- 將實體名稱變更為MonsterID。
                monsterData.MonsterID,
                -- 設定要出現怪物的出現位置。
                Vector3(data.SpawnPosX, data.SpawnPosY, 0),
                -- 將要建立的實體在Hierarchy中設為自己的子實體。
                self.Entity
            )
            -- 若monster已正常建立, 則初始化其數值。
            if monster then
                -- 再次將怪物的位置初始化為MonsterSpawnData中的位置。
                monster.TransformComponent.Position = Vector3(data.SpawnPosX,
                    data.SpawnPosY, 0)
                -- 依照MonsterDefaultData初始化怪物資料。
                monster.MonsterState:InitData(monsterData, data.ItemDropRate,
                    data.MustDropItem)
            end
        end,
        -- 登錄時間, 使其依照MonsterSpawnData中登錄的出現時間產生。
        data.EntranceTime)
    end
end
-- 當所有怪物產生預約完成後, 將通關所需的怪物擊殺數傳遞給GameLogic。
_GameLogic:InitClearKillCount(self.needKillCount)
end

@ExecSpace("Client")
method void ClearAllChild()
    -- 尋找所有作為自己子實體的實體, 並登錄到chidList中。

```

```
local childList = self.Entity.Children
-- 若childList為nil, 則立即結束程式碼。
if childList == nil then return end
-- 由於childList的數量大於等於1, 因此開始依childList數量進行搜尋與處理。
for i, child in pairs(childList) do
    --若第i個實體可使用, 則移除要素child。
    if isValid(child) then
        _EntityService:Destroy(child)
    end
end
-- 若timerTable中存在要素, 則開始搜尋timerTable中的要素。
if self.timerTable ~= nil and next(self.timerTable) ~= nil then
    -- 解除第i個已預約的計時器。
    for i = 1, #self.timerTable do
        _TimerService:ClearTimer(self.timerTable[i])
    end
end
end
end

end
```

最低實作標準

- 可透過修改MonsterSpawnData來調整與Monster產生相關的部分。
- 必須能加入Monster, 讓各種Monster都能產生。
- 可透過ItemDropRate與MustDropItem調整每個怪物的道具掉落機率。

應用與擴展方向

- 在GameLogic中以number管理Stage進度, 並在InGame開始時變更MonsterSpawnManager的StageID值。
- 實作功能, 使依照MonsterSpawnData的StageID值而出現的Monster有所不同。

[💡 基礎學習]構成UI

影片時間軸

- 可在課程影片中以下時間點進行學習。
- 實作UI: 03:00:20

學習目標

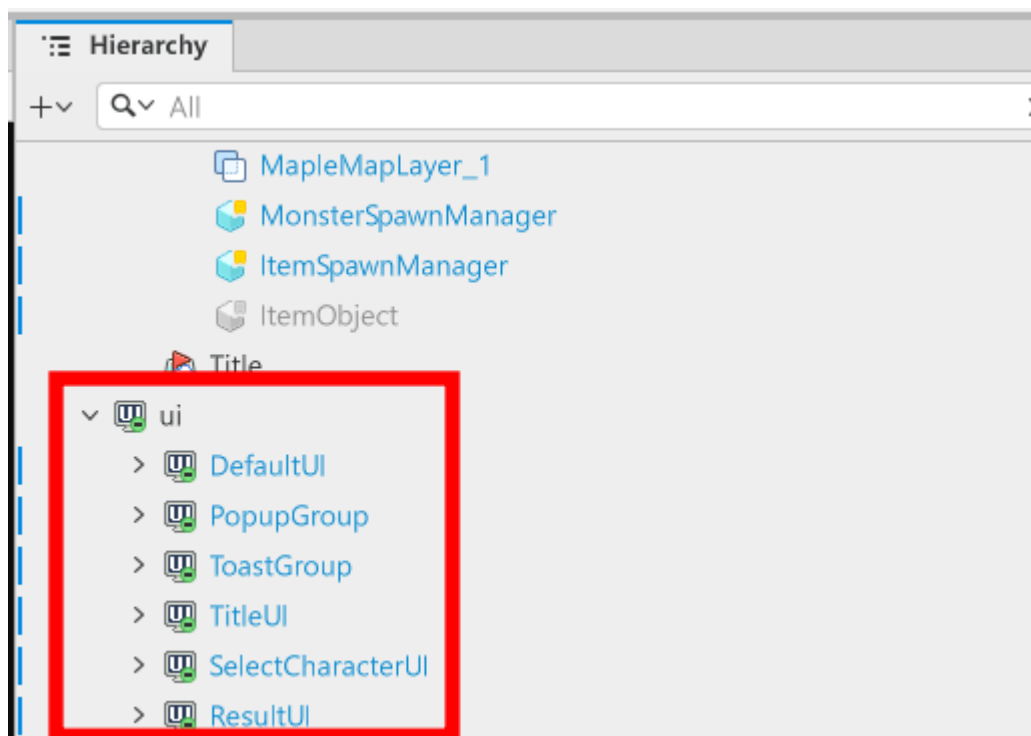
- 學習在楓之谷世界中構成UI的方法。

- 學習Localizing功能，並可學習套用Localizing的方法。
- 理解UIManageLogic的角色，並學習活用方法。
- 可參考範例世界的程式碼，製作符合欲製作遊戲的UI並賦予功能。

觀看該影片後可嘗試執行的內容

- 直接實作範例世界中已實作的功能。
- 在實作過程中，變更UI以展現所製作世界的特色。
- 製作並管理新的UIGroup，新增UI以傳遞所需資訊。

理解UI Group

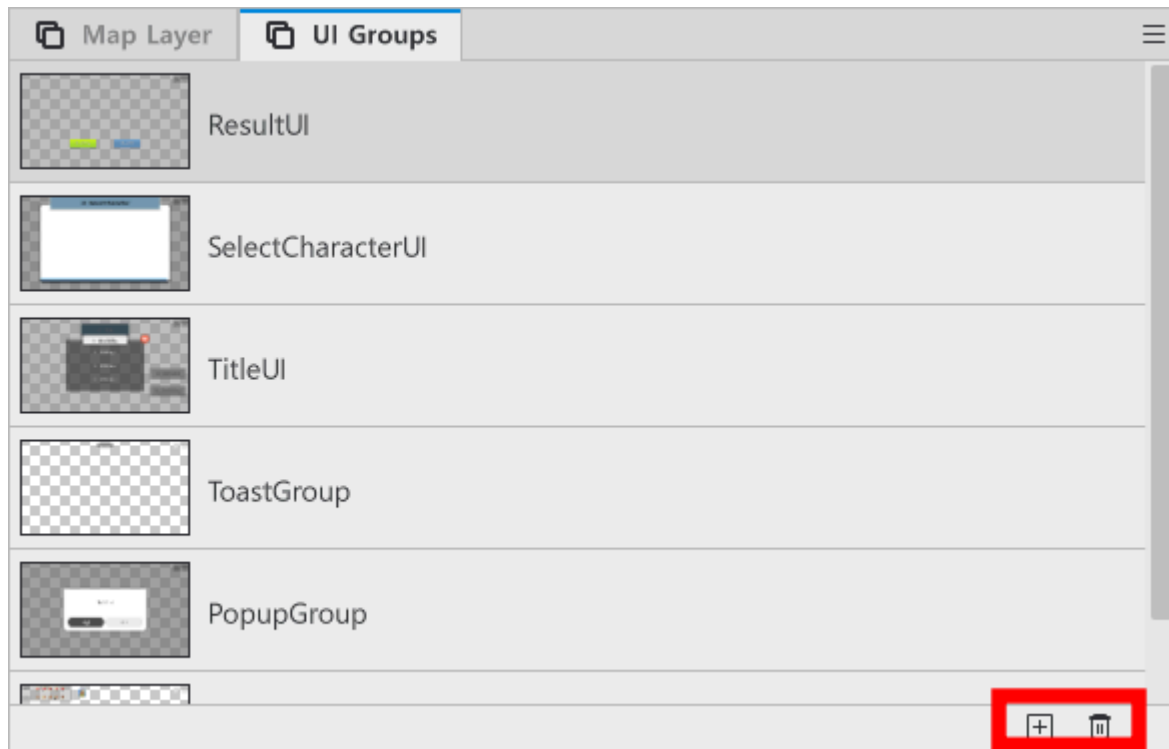


- 查看Hierarchy視窗下方，可以確認到以ui這個名稱構成的實體。
- 該實體是為了輸出ui而構成的ui group。
- 可透過修改ui group來變更或製作要輸出的ui。

建立UI Group



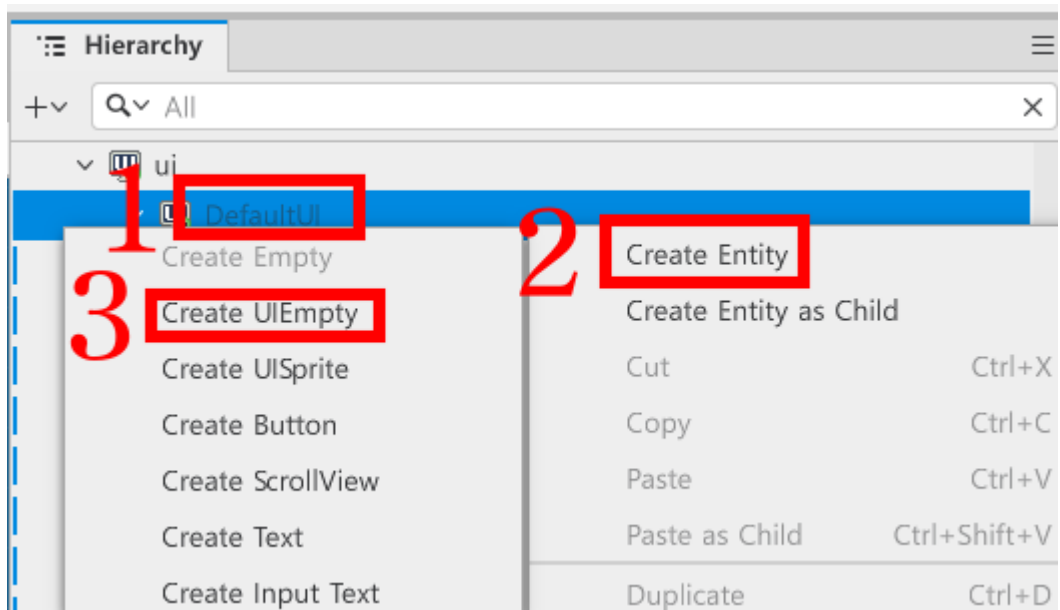
- 點擊編輯器上方的UI按鈕，切換為UI編輯狀態。



- 可在編輯器視窗下方的UI Groups中點擊+按鈕來新增新的UI Groups。
- 若有需要刪除的UI Groups，可點擊要刪除的UI Group後，再點擊垃圾桶按鈕進行刪除。

製作空的UI實體

- 在UI中也可以像製作一般實體一樣製作實體。
- 差異在於，建立空實體時不是使用Create Empty，而是使用Create UIEmpty。



- 在Hierarchy視窗中對要新增實體的ui group按右鍵。
- 點擊Create Entity。點擊Create UIEmpty即可建立空的UI實體。

為UI實體賦予Component

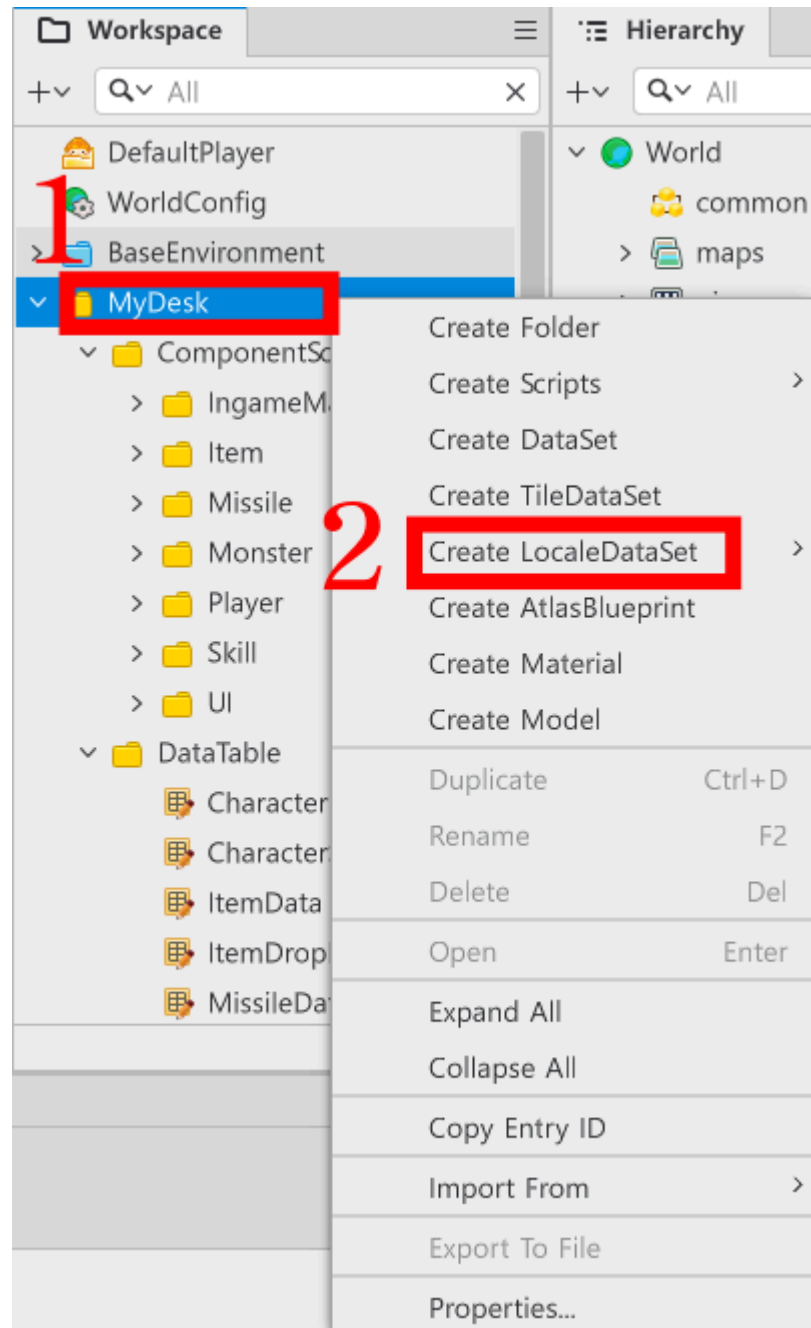
- UI實體也和一般實體一樣，可以賦予Component來執行額外功能。
- 賦予Component的方法與一般實體使用Add Component的方式相同。
- 在本單元中，將說明常用的Component。
 - **SpriteGUIRendererComponent**：這是用於在UI中顯示圖片的Component。在UI中輸出圖片時，比起SpriteRendererComponent，更建議使用該Component。
 - **TextGUIRendererComponent**：這是用於在UI中顯示文字的Component。
 - **ButtonComponent**：當UI需要執行按鈕功能時所使用的Component。

為UIGroup賦予Controller

- Controller這個概念，是透過製作Component來管理該UI的概念。
- 為每個UIGroup賦予Controller Component，並構成功能，讓該Component處理與UI相關的功能。
- 將各Controller登錄到UIManageLogic中，當呼叫UIManageLogic時，就能向其他UI請求與UI相關的功能。
- 在範例世界中，製作了如下的UIController。
 - **DefaultUIController**：這是用來管理DefaultUI的Component。主要負責更新玩家體力、技能使用次數，也負責在遊戲開始時將玩家的基本攻擊與技能使用功能登錄到各按鈕。
 - **TitleUIController**：這是用來構成標題畫面的Component。該Component是為了處理開始遊戲按鈕與輸出遊戲玩法UI的功能而製作。
 - **ResultUI**：當GameLogic將遊戲結果傳遞給UIManageLogic時，UIManageLogic會向該Component請求應顯示的結果UI。

製作Localizing DataSet

- Localizing功能是在需要服務其他國家時的必要功能。
- 楓之谷世界可利用Locale DataSet來製作該功能。



- 在Workspace中對MyDesk按右鍵。
- 點擊Create LocaleDataSet。
- 點擊Default來製作Locale DataSet。
- 在範例世界中，是以StringTable製作。

	1	2	3	4	5
ID	Key	Source	Note	ko	en
1	UI_Title	Shooting Game	타이틀에 사용	슈팅 게임 프레	Shooting Game
2	UI_StartGame	Start Game	타이틀 화면의	게임 시작	Start Game
3	UI_Pirate	Pirate	UI표기용 직업명	해적	Pirate
4	UI_SelectCharac	Select	캐릭터 선택 창	캐릭터 선택	Select Character
5	UI_Retry	Retry	결과창에서 다시	다시하기	Retry
6	UI_Exit	Exit	결과창의 나가기	나가기	Exit
7	UI_HowToPlay	How to play	게임 방법 버튼	플레이 방법	How to play
8	UI_InfoPlay1	Move	게임 방법 이동	이동 : WSAD	Move: WSAD
9	UI_InfoPlay2	Attack	게임 방법 공격	공격 : J	Attack : J
10	UI_InfoPlay3	Skill	게임 방법 스킬	필살기 : K	Skill : K

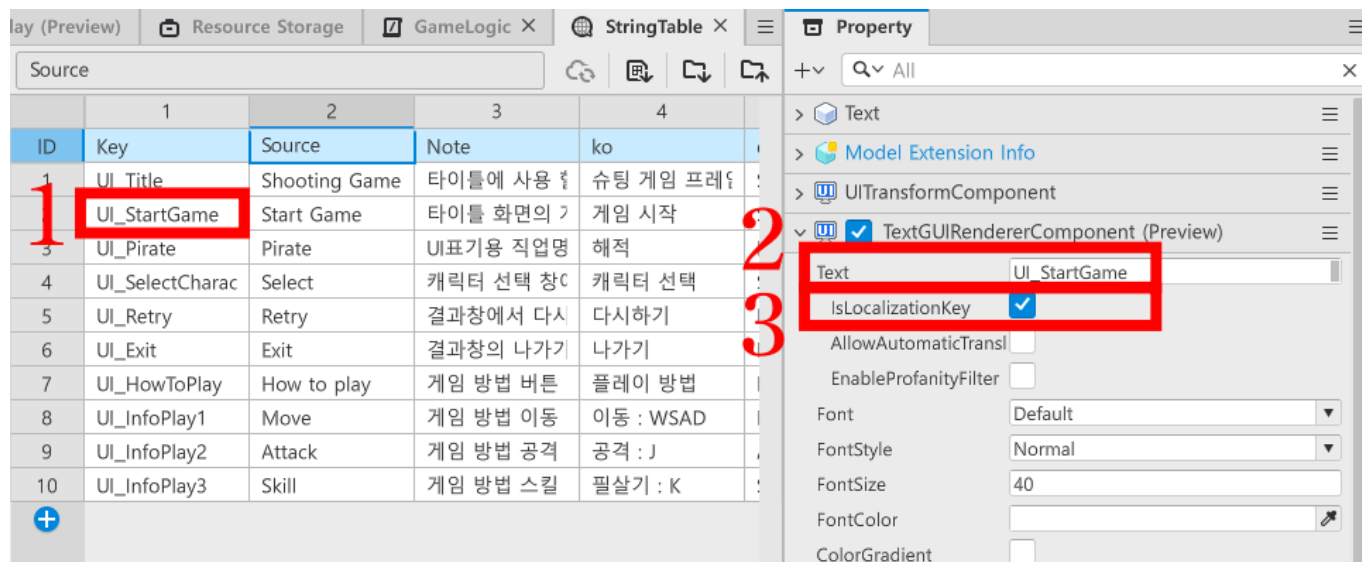
姓名	說明
Key	這是楓之谷世界在進行Localizing時，用來尋找需要Localizing文字的ID值。
Source	這是用來承載翻譯原文文字的值。若使用楓之谷世界提供的自動翻譯功能，便會依據該值尋找翻譯值。
Note	這是用來記載該文字是以什麼意義、用途撰寫的欄位。
ko	這是用來承載韓文輸出文字的值。
en	這是用來承載英文輸出文字的值。

- 此外，也可以加入楓之谷世界官方支援的各種語言來製作。

套用Localizing

- 套用Localizing功能的方法大致有2種。
- 一種是利用TextGUIRendererComponent的值，另一種則是透過腳本使用。

利用TextGUIRendererComponent的方法



- 在製作完成的Locale DataSet中輸入要套用Localizing的值
- 在TextGUIRendererComponent的Text值中輸入Locale DataSet的Key。
- 將TextGUIRendererComponent的isLocalizationKey值設為勾選狀態。
- 透過該狀態，可以讓Localizing功能自動套用。

利用腳本的方法

- 在範例世界中，為了方便快速使用，已先製作好LocalizeLogic。
- 以下是已製作的LocalizeLogic程式碼。
- 該程式碼以Plain Text撰寫。

```
@Logic
script LocalizeLogic extends Logic

@ExecSpace("ClientOnly")
method string GetLocalizeStringByKey(string StringKey)
-- 依據傳入的StringKey值，尋找並回傳StringTable中的值。
return _LocalizationService.GetText(StringKey)
end

end
```

- 當以參數StringKey傳入Locale DataSet的Key值時，會依照目前遊戲執行國家的語言回傳文字。
 - EX: 若遊玩國家為美國等英語圈，則會回傳en的值。

活用UIManageLogic

- 當所有UI設定都完成後，為了管理各Controller，必須將其登錄到UIManageLogic中並設定功能。
- 以下是UIManageLogic的程式碼。
- 該程式碼是在Plain Text環境中撰寫。

```

@Logic
script UIManageLogic extends Logic

property TitleUIController titleUIController = nil

property SelectCharacterUIController selectCharacterUIController = nil

property ResultUIController resultUIController = nil

property DefaultUIController defaultUIController = nil

@ExecSpace("ClientOnly")
method void OnBeginPlay()
-- 當遊戲開始時，尋找並登錄TitleUIController。
self:InitTitleUIController()
-- 當遊戲開始時，尋找並登錄SelectCharacterUIController。
self:InitSelectCharacterUIController()
-- 當遊戲開始時，尋找並登錄ResultUIController。
self:InitResultUIController()
-- 當遊戲開始時，尋找並登錄DefaultUIController。
self:InitDefaultUIController()
end

@ExecSpace("ClientOnly")
method void InitTitleUIController()
--若未登錄TitleUIController，則登錄該元件。
if not isvalid(self.titleUIController) then
    local titleUIEntity = _EntityService:GetEntityByPath("/ui/TitleUI")
    --若成功找到titleUIGroup，則登錄該實體的元件。
    if isvalid(titleUIEntity) then
        local component = titleUIEntity.TitleUIController
        -- 搜尋是否已正常登錄元件後，將其指派。
        if isvalid(component) then
            self.titleUIController = component
        end
    end
end
end

@ExecSpace("ClientOnly")
method void InitSelectCharacterUIController()
--若未登錄SelectCharacterUIController，則登錄該元件。
if not isvalid(self.selectCharacterUIController) then
    local selectCharacterUIEntity =
_EntityService:GetEntityByPath("/ui/SelectCharacterUI")
    --若成功找到SelectCharacterUIController，則登錄該實體的元件。
    if isvalid(selectCharacterUIEntity) then
        local component = selectCharacterUIEntity.SelectCharacterUIController
        -- 搜尋是否已正常登錄元件後，將其指派。
        if isvalid(component) then
            self.selectCharacterUIController = component
        end
    end
end

```

```
        end
    end
end

@ExecSpace("Client")
method void ShowTitle()
    -- 顯示標題畫面。
    self.titleUIController:Show()
    -- 其餘UI全部設為隱藏。
    self.selectCharacterUIController:Hide()
    self.resultUIController:Hide()
    self.defaultUIController:Hide()
end

@ExecSpace("Client")
method void CallSelectCharacterUI()
    -- 顯示角色選擇視窗。
    self.selectCharacterUIController:Show()
    -- 嘗試初始化角色選擇視窗內的角色清單。
    self.selectCharacterUIController:InitCharacterSelectFrame()
end

@ExecSpace("Client")
method void ShowInGame()
    -- 初始化作為遊戲畫面UI的defaultUI。
    self.defaultUIController:Show()
    -- 請求初始化自動攻擊UI。
    self.defaultUIController:UpdateAutoUI()
    -- 將其餘UI設為隱藏。
    self.titleUIController:Hide()
    self.selectCharacterUIController:Hide()
    self.resultUIController:Hide()
end

@ExecSpace("Client")
method void InitResultUIController()
    --若尚未登錄ResultUIController, 則登錄該元件。
    if not isvalid(self.resultUIController) then
        local resultUIEntity = _EntityService:GetEntityByPath("/ui/ResultUI")
        --若成功找到ResultUIController, 則登錄該實體的元件。
        if isvalid(resultUIEntity) then
            local component = resultUIEntity.ResultUIController
            -- 搜尋是否已正常登錄元件後, 將其指派。
            if isvalid(component) then
                self.resultUIController = component
            end
        end
    end
end

@ExecSpace("Client")
method void ShowResultUI(boolean IsClear)
    -- 顯示結果UI。
    self.resultUIController:Show(IsClear)
```

```

if IsClear then
    --播放祝賀效果音
    _SoundService:PlaySound("5d57b617cfc242cca347925e209c8e05", 1.0)
else
    --播放失敗效果音
    _SoundService:PlaySound("52dccfb8889b4181b9181780dc7e9040", 1.0)
end
end

@ExecSpace("Client")
method void InitDefaultUIController()
    --若尚未登錄defaultUIController, 則登錄該元件。
    if not isvalid(self.defaultUIController) then
        local titleUIEntity = _EntityService:GetEntityByPath("/ui/DefaultUI")
        --若成功找到DefaultUIGroup, 則登錄該實體的元件。
        if isvalid(titleUIEntity) then
            local component = titleUIEntity.DefaultUIController
            -- 搜尋是否已正常登錄元件後, 將其指派。
            if isvalid(component) then
                self.defaultUIController = component
            end
        end
    end
end
end

@ExecSpace("Client")
method void UpdateLifeCountUI(number PlayerLife)
    -- 依照PlayerLife數量更新DefaultUI的體力UI。
    self.defaultUIController:UpdateLifeUI(PlayerLife)
end

@ExecSpace("Client")
method void UpdateSkillCountUI(number LeftSkillCount)
    -- 依照LeftSkillCount的數量更新DefaultUI的技能UI。
    self.defaultUIController:UpdateSkillUI(LeftSkillCount)
end

end

```

- 已註冊所有UI Group的Controller, 並依照需要的UI更新製作對應方法。
- 當GameLogic或PlayerState有UI更新請求時, 呼叫負責的Controller來執行該作業。

最低實作標準

- 建構管理UI的Controller, 並建立UIManageLogic, 使Controller功能能在外部方便使用。
- 在UI中, 當按鈕等元素被點擊時, 必須產生互動。
- UI必須傳遞遊戲進行所需的資訊。

應用與擴展方向

- 當玩家受到攻擊時, 製作Event讓UI無需向UIManageLogic請求更新也能自動更新。
 - 此方式將成為日後製作Server與Client通訊時同步處理的基礎。

- 可依需求製作新的UI或在畫面上加入效果，讓使用者能直觀接收視覺資訊。

結尾

- UI會依據製作的遊戲類型與方向性而有很大的差異。
- 範例世界的UI是為了展示整體流程而製作，並不一定需要完全照此方式進行。
- 反而積極採用自己的想法並融入其中，能製作出更優秀的遊戲。